数据结构

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 iv

第一章 哈希表 1
1.1 GNU gp_hash_table 与 cc_hash_table . . . 1

第二章 并查集 4
2.1 并查集 . . . 4
2.1.1 并查集 . . . 4
2.1.2 可撤销并查集 . . . 6
2.1.3 带势并查集与差值约束 . . . 8
2.1.4 时间线段树分治：离线动态连通性 . . . 11

第三章 堆 17
3.1 配对堆 . . . 17
3.1.1 PBDS 可修改、可合并优先队列 . . . 17
3.2 左偏树 . . . 21
3.2.1 左偏树可并小根堆 . . . 21

第四章 块状数据结构 23
4.1 树分块 . . . 23
4.1.1 树分块与在线路径去重 . . . 23

第五章 ST 表 30
5.1 位掩码静态区间最值与位置 . . . 30
5.2 二维静态矩形最值与位置 . . . 34

第六章 树状数组 38
6.1 稀疏二维树状数组：点加与矩形和 . . . 38
6.2 四矩二维树状数组：矩形加与矩形和 . . . 41
6.3 树状数组第 k 小与组合依赖 . . . 43
6.4 离线静态矩形权值和 . . . 45

第七章 线段树 48
7.1 线段树基础 . . . 48
7.1.1 通用懒标记线段树与边界搜索 . . . 48
7.1.2 通用线段树与左右边界搜索 . . . 53
7.1.3 区间加与区间和 . . . 58
7.1.4 线段树：区间乘、加、赋值与模意义求和 . . . 59
7.1.5 线段树维护单调栈：区间加与接雨水 . . . 63
7.2 线段树合并 & 分裂 . . . 68
7.2.1 线段树合并与分裂 . . . 68
7.3 李超线段树 . . . 79
7.3.1 精确线段李超树：最大值编号 . . . 79
7.3.2 离散李超树与最优直线编号 . . . 85
7.4 区间最值操作 & 区间历史最值 . . . 88

7.4.1 Segment Tree Beats：区间截断、加法与和 . . . 88

第八章 划分树 **94**
8.1 划分树：静态区间第 k 小 . . . 94
8.2 压位小波矩阵：静态区间顺序统计 . . . 97

第九章 二叉搜索树 & 平衡树 **102**
9.1 二叉搜索树 & 平衡树 . . . 102
9.1.1 GNU rope 与版本共享 . . . 102
9.1.2 PBDS 有序树与顺序统计 . . . 104
9.2 Treap . . . 106
9.2.1 FHQ Treap：可重集合与顺序统计 . . . 106
9.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和 . . . 110
9.2.3 动态仿射序列：插删、翻转与区间和 . . . 115
9.2.4 状态序列 Treap：分组区间 GCD . . . 120
9.2.5 序列最大值 Treap . . . 125
9.3 Splay 树 . . . 129
9.3.1 Splay：可重集合与顺序统计 . . . 129
9.3.2 可合并 Splay：固定编号与集合顺序统计 . . . 135
9.3.3 按值分裂与整体平移 Splay . . . 143
9.3.4 区间翻转 Splay 与固定编号定位 . . . 147
9.4 替罪羊树 . . . 154
9.4.1 替罪羊树：可重集合与顺序统计 . . . 154
9.4.2 在线序列：插入、修改与区间第 k 小 . . . 158
9.4.3 序列替罪羊树：按位置插删与区间等值计数 . . . 163
9.5 笛卡尔树 . . . 169
9.5.1 笛卡尔树与相等值优先级 . . . 169

第十章 可持久化数据结构 **171**
10.1 可持久化线段树 . . . 171
10.1.1 主席树：静态区间第 k 小 . . . 171
10.1.2 可持久化数组：历史版本分叉与单点修改 . . . 173
10.1.3 可持久化区间加与版本拼接 . . . 175
10.1.4 树上路径点权第 k 小 . . . 183
10.1.5 静态区间不同数计数（前驱位置主席树） . . . 186
10.1.6 持久化计数与权值索引 . . . 189
10.1.7 持久化奇偶计数与降序选择 . . . 193
10.1.8 可持久化并查集 . . . 198
10.2 可持久化平衡树 . . . 201
10.2.1 可持久化有序 Treap：分叉版本与顺序统计 . . . 201
10.3 可持久化字典树 . . . 207
10.3.1 前缀可持久化 01 字典树 . . . 207

第十一章 树套树 **210**
11.1 树状数组套权值线段树 . . . 210
11.1.1 动态区间第 k 小（树状数组套权值线段树） . . . 210

第十二章 K-D Tree **214**
12.1 动态二维 K-D Tree：单点累加与矩形和 . . . 214
12.2 静态 K-D Tree 最近与最远点 . . . 219
12.3 动态 K-D Tree 矩形取点与删除 . . . 224
12.4 静态二维矩形最小权与删点 . . . 230

第十三章 动态树 **235**
13.1 Link Cut Tree . . . 235
13.1.1 Link-cut tree：动态森林与路径异或 . . . 235
13.1.2 动态换根路径乘积总和 . . . 240
13.1.3 点权修改与全树路径异或和 . . . 243
13.1.4 动态森林路径加与最大值 . . . 249

第十四章 配套知识与验证范围 **254**
14.1 可持久化区间拼接的懒标记 . . . 254
14.2 杭州 2023 K：跨版本拼接应用 . . . 255
14.3 并查集模板题的接口范围 . . . 256
14.4 动态树路径乘积：虚子树与双向信息 . . . 256
14.5 隐式 Treap 位翻转与顺序反转 . . . 256
14.6 替罪羊树：重复计数与墓碑清理 . . . 256
14.7 线段树维护单调栈：双向扫描与接雨水 . . . 257
14.8 动态 K-D Tree：矩形和与重建边界 . . . 257
14.9 压位小波矩阵与区间顺序统计 . . . 258

第一部分 跨类依赖 **259**

附录 A 数论 **260**
A.1 模算术简介 . . . 260
A.1.1 动态模整数与合数模逆元 . . . 260
A.1.2 静态模整数 . . . 263
A.2 模逆元 . . . 265
A.2.1 模整数类型的批量逆元 . . . 265
A.3 卢卡斯定理 . . . 266
A.3.1 Lucas：小素数模的大下标组合数 . . . 266

附录 B 组合数学 **268**
B.1 排列组合 . . . 268
B.1.1 阶乘组合数 . . . 268

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　哈希表

1.1　GNU gp_hash_table 与 cc_hash_table

GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp。gp_map<K,V> 为探测哈希表，cc_map<K,V> 为碰撞链接哈希表；K 为不超过 64 位的整数，复用下方 hash64。两者支持 operator[]、find、insert、erase(key)、clear、size、empty；insert 不覆盖已有键，operator[] 对不存在的键插入默认值。只查询时用 find 判断 end。通常查改摊还 O(1)，最坏 O(n)，空间 O(n)，不支持排名或有序前驱。纯键值查改可先试 gp_map；需要在插入/删除其他键后保留找到的元素引用时可考虑 cc_map，其节点分配和链接有成本，速度仍须按题实测。gp_map 修改后重新 find；cc_map 的点迭代器不等于可递增的遍历迭代器，删除该元素、clear 或销毁后不能保留引用。默认策略不直接调用 resize，也不照搬 unordered_map::reserve。hash64 使用进程内固定随机种子，无符号回绕；只针对整数键，不保证已知种子下的最坏碰撞。历史 associative_array AC 仅覆盖 gp_map，cc_map 另行验证。

需要排名与有序前驱时，PBDS 平衡树见第 104 页。

```
struct hash64
{
    static uint64_t mix(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const
    {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};

template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
template <class K, class V> using cc_map = __gnu_pbds::cc_hash_table<K, V, hash64>;
```

模板题使用：Library Checker associative_array

题意：维护整数键值映射，修改为覆盖赋值，查询不存在的键输出 0。gp_map 用 operator[] 写入，用 find 查询，避免查询插入新键。所需条目中包含 hash64。

所需模板：gp_map（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    gp_map<uint64_t, uint64_t> a;
    while (q--)
    {
        int op;
        uint64_t k, v;
        cin >> op >> k;
        if (op == 0)
        {
            cin >> v;
            a[k] = v;
        }
        else
        {
            auto it = a.find(k);
            cout << (it == a.end() ? 0 : it->second) << '\n';
        }
    }
}
```

模板题使用：Library Checker associative_array

题意：同一关联数组题的 cc_map 用法：覆盖赋值，查询不存在的键输出 0。cc_map 与 hash64 均在 gp_map 条目中；本例独立执行验证，不借用 gp_map 的运行结果。

所需模板：gp_map（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cc_map<uint64_t, uint64_t> a;
    while (q--)
    {
        int op;
        uint64_t k, v;
        cin >> op >> k;
        if (op == 0)
        {
            cin >> v;
            a[k] = v;
        }
        else
        {
            auto it = a.find(k);
            cout << (it == a.end() ? 0 : it->second) << '\n';
        }
    }
}
```

第二章　并查集

2.1　并查集

2.1.1　并查集

dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 4 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

2.1.2 可撤销并查集

RollbackDSU(n) 可使用 0..n（若题目 0-based 则不用 n）。find 不压缩路径，merge 按集合大小合并，已同集合返回 false 且不记历史；siz 仅根处有效。snapshot() 返回历史长度，退出递归分支时 rollback(t) 撤销此后成功合并。t 必须是当前历史的祖先快照，不能用已回滚分支的旧快照恢复版本。find/merge 为 O(log(n+1))，每次撤销成功合并 O(1)，空间 O(n)；适合版本树或时间分治。它按大小合并以保证回滚复杂度，接口与普通 dsu 的根连接方向不同。

```
struct RollbackDSU
{
    vector<int> fa, siz;
    vector<pair<int, int>> his;

    RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x) const
    {
        while (x != fa[x]) x = fa[x];
        return x;
    }

    int snapshot() const { return (int)his.size(); }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        his.push_back({y, siz[x]});
        fa[y] = x;
        siz[x] += siz[y];
        return true;
    }

    void rollback(int t)
    {
        assert(0 <= t && t <= snapshot());
        while (snapshot() > t)
        {
            auto [y, s] = his.back();
            his.pop_back();
            siz[fa[y]] = s;
            fa[y] = y;
        }
    }
};
```

模板题使用：Library Checker persistent_unionfind

题意：离线读入所有操作，按所依赖的版本建立树。版本 -1 映射到树根 0，操作 i 映射到 i+1；顶点仍是 0-based。递归进入版本前保存 snapshot，退出时 rollback，答案按原查询顺序输出。只有合并产生可引用版本；本例是离线解法，不提供在线任意版本访问。总时间 O(n+q log(n+1))，空间 O(n+q)，版本链递归深度可为 q。

所需模板：RollbackDSU（第 6 页，2.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_unionfind

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> type(q + 1), u(q + 1), v(q + 1), ans(q + 1);
    vector<vector<int>> g(q + 1);
    for (int i = 1; i <= q; i++)
    {
        int k;
        cin >> type[i] >> k >> u[i] >> v[i];
        g[k + 1].push_back(i);
    }
    RollbackDSU d(n);
    auto dfs = [&](auto &&self, int x) -> void
    {
        int t = d.snapshot();
        if (x)
        {
            if (type[x] == 0)
                d.merge(u[x], v[x]);
            else
                ans[x] = d.find(u[x]) == d.find(v[x]);
        }
        for (int y : g[x]) self(self, y);
        d.rollback(t);
    };
    dfs(dfs, 0);
    for (int i = 1; i <= q; i++)
        if (type[i] == 1) cout << ans[i] << '\n';
}
```

2.1.3 带势并查集与差值约束

PotentialDSU<T>(n)，n≥0，点号 0..n−1。merge(u,v,w) 尝试添加 potential[u]−potential[v]=w，返回是否与已接受信息一致；重复一致约束也返回 true，矛盾返回 false，已有逻辑关系不变（仍可能路径压缩）。diff(u,v) 返回 optional<T>，未连通为 nullopt，连通时为 u−v；合法零/负值不是失败标志。find 求根并压缩路径，size 返回连通块大小。T 须提供精确加法交换群的 +、−、== 及零 T(0)，不是非交换群或浮点近似比较；整数须保证所有中间运算不溢出，动态模数不可在实例存活时改变。n 点、每条输入差值绝对值 ≤W 时，(2n+1)W≤LLONG_MAX 是 long long 的一个保守充分条件。按大小合并，摊还 O(α(n)) 次 T 运算、O(n) 空间；递归 find 未压缩前深度 O(log n)。fa/sz/d 为内部状态，不直接改写。原普通并查集的指定根合并方向与 bool 语义不变。

```
template <class T = long long> struct PotentialDSU
{
    vector<int> fa, sz;
    vector<T> d;

    PotentialDSU(int n) : fa(n), sz(n, 1), d(n, T(0))
    {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int u)
    {
        assert(0 <= u && u < (int)fa.size());
        int p = fa[u];
        if (p != u)
        {
            fa[u] = find(p);
            d[u] = d[u] + d[p];
        }
        return fa[u];
    }
```

约束合并、差值查询与分量大小（接上页同一结构体）：

```
    // Accept potential[u] - potential[v] = w iff it is consistent.
    bool merge(int u, int v, T w)
    {
        int a = find(u), b = find(v);
        if (a == b) return d[u] - d[v] == w;
        T t = d[u] - d[v] - w;
        if (sz[a] < sz[b])
        {
            swap(a, b);
            t = T(0) - t;
        }
        fa[b] = a;
        d[b] = t;
        sz[a] += sz[b];
        return true;
    }

    optional<T> diff(int u, int v)
    {
        int a = find(u), b = find(v);
        if (a != b) return nullopt;
        return d[u] - d[v];
    }

    int size(int u) { return sz[find(u)]; }
};
```

模板题使用：Library Checker Unionfind with Potential

题意：未知序列 a，1≤N,Q≤200000，模 998244353。0 u v x 尝试加入 a[u]−a[v]=x；与此前已接受信息一致输出 1，否则输出 0 并拒绝，不把矛盾边加入状态。1 u v 查询 a[u]−a[v] 的规范模值，未连通输出 −1；点号 0-based，0≤x<mod。重复一致、自约束及零差值均有效。先设置模整数的模数再构造结构，存活期间不改变它；复用已有加减与相等比较，不需要乘法逆元。

所需模板：PotentialDSU（第 8 页，2.1.3 节）、mint（第 260 页，A.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/unionfind_with_potential

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    using Z = mint<>;
    Z::set_mod(998244353);
    PotentialDSU<Z> d(n);
    while (q--)
    {
        int t, u, v;
        cin >> t >> u >> v;
        if (t == 0)
        {
            int x;
            cin >> x;
            cout << d.merge(u, v, Z(x)) << '\n';
        }
        else
        {
            auto x = d.diff(u, v);
            cout << (x ? x->v : -1) << '\n';
        }
    }
}
```

2.1.4 时间线段树分治：离线动态连通性

代码依赖：RollbackDSU（第 6 页，2.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

TimeConnectivity(n,q)，点号 0..n（1-based 图不使用 0），时间 0..q-1。add(l,r,u,v) 登记边在半开区间 [l,r) 有效，允许空区间、重叠区间、重边和自环。run(visit) 按时间递增调用 visit(t,const RollbackDSU &d)，可用 find 判断连通、siz[find(u)] 取分量大小；只在回调内读取，不保留引用，不在遍历中改图。每次 run 重新建并查集，可重复调用，也可在两次 run 间加区间。q=0 不调用回调。n>=0 且 n<INT_MAX，0<=q<=INT_MAX/4，并须满足内存限制。K 个非空区间：存储 O(n+q+K log(q+1))；建区间 O(K log(q+1))，遍历 O(n+q+K log(q+1) log(n+2))，另加回调代价。依赖 RollbackDSU，不做路径压缩；DFS 保持递归。

```
struct TimeConnectivity
{
    int n, q;
    vector<vector<pair<int, int>>> edges;

    TimeConnectivity(int n, int q) : n(n), q(q), edges(4 * size_t(q) + 1)
    {
        assert(n >= 0 && n < INT_MAX && q >= 0 && q <= INT_MAX / 4);
    }

    // Edge (u,v) exists at times l <= t < r. Empty intervals are allowed.
    void add(int l, int r, int u, int v)
    {
        assert(0 <= l && l <= r && r <= q);
        assert(0 <= u && u <= n && 0 <= v && v <= n);
        if (l < r)
            insert(1, 0, q, l, r, {u, v});
    }

    template <class F> void run(F visit) const
    {
        RollbackDSU d(n);
        if (q > 0)
            dfs(1, 0, q, d, visit);
    }
```

时间区间插入与回滚递归（接上页同一结构体）：

```
private:
    void insert(int p, int l, int r, int a, int b, pair<int, int> e)
    {
        if (a <= l && r <= b)
        {
            edges[p].push_back(e);
            return;
        }
        int m = l + (r - l) / 2;
        if (a < m)
            insert(p * 2, l, m, a, b, e);
        if (m < b)
            insert(p * 2 + 1, m, r, a, b, e);
    }

    template <class F>
    void dfs(int p, int l, int r, RollbackDSU &d, F &visit) const
    {
        int saved = d.snapshot();
        for (auto [u, v] : edges[p])
            d.merge(u, v);
        if (r - l == 1)
            visit(l, static_cast<const RollbackDSU &>(d));
        else
        {
            int m = l + (r - l) / 2;
            dfs(p * 2, l, m, d, visit);
            dfs(p * 2 + 1, m, r, d, visit);
        }
        d.rollback(saved);
    }
};
```

应用题补充：P2147 洞穴勘测（离线解法）

题意：初始无边，读 Connect/Destroy/Query；合法增删且始终为森林。点号 1..n；加边时刻 a、删边时刻 b 配成 [a,b)，结束未删边配到 q。输出 Yes/No。竞赛应用，不是任意多重图完整接口的正式模板题。

所需模板：RollbackDSU（第 6 页，2.1.2 节）、TimeConnectivity（第 11 页，2.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2147

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    TimeConnectivity graph(n, q);
    map<pair<int, int>, int> start;
    vector<pair<int, int>> ask(q, {-1, -1});
    for (int t = 0; t < q; ++t)
    {
        string op;
        int u, v;
        cin >> op >> u >> v;
        if (u > v)
            swap(u, v);
        pair<int, int> e = {u, v};
        if (op == "Connect")
            start[e] = t;
        else if (op == "Destroy")
        {
            auto it = start.find(e);
            graph.add(it->second, t, u, v);
            start.erase(it);
        }
        else
            ask[t] = e;
    }
    for (auto [e, t] : start)
        graph.add(t, q, e.first, e.second);
    graph.run([&](int t, const RollbackDSU &d)
    {
        auto [u, v] = ask[t];
        if (u != -1)
            cout << (d.find(u) == d.find(v) ? "Yes" : "No") << '\n';
    });
    return 0;
}
```

接口用法演示：带初始边的多重图增删连通性（接口演示）

演示约定：EOF 多组 n,m,q，初始 m 条无向边，再读 q 行 + u v / - u v / ? u v。点号 1..n，允许重边和自环，删除须存在且只删一份；每次? 输出连通性 0/1 及 u 的分量大小。计数从 0 到 1 开启生存区间，从 1 到 0 关闭；初始边从 0 开始。n>=1，m+q<=INT_MAX。

所需模板：RollbackDSU（第 6 页，2.1.2 节）、TimeConnectivity（第 11 页，2.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    while (cin >> n >> m >> q)
    {
        TimeConnectivity graph(n, q);
        // value = {active multiplicity, start of positive multiplicity}
        map<pair<int, int>, pair<int, int>> active;
        for (int i = 0; i < m; ++i)
        {
            int u, v;
            cin >> u >> v;
            if (u > v)
                swap(u, v);
            ++active[{u, v}].first;
        }
```

继续处理操作（接上页同一 main）：

```
        vector<pair<int, int>> ask(q, {-1, -1});
        for (int t = 0; t < q; ++t)
        {
            char op;
            int u, v;
            cin >> op >> u >> v;
            if (op == '?')
            {
                ask[t] = {u, v};
                continue;
            }
            if (u > v)
                swap(u, v);
            pair<int, int> e = {u, v};
            auto &state = active[e];
            if (op == '+')
            {
                if (state.first == 0)
                    state.second = t;
                ++state.first;
            }
            else
            {
                assert(state.first > 0);
                --state.first;
                if (state.first == 0)
                    graph.add(state.second, t, u, v);
            }
        }
        for (auto [e, state] : active)
            if (state.first > 0)
                graph.add(state.second, q, e.first, e.second);
        graph.run([&](int t, const RollbackDSU &d)
        {
            auto [u, v] = ask[t];
            if (u != -1)
                cout << (d.find(u) == d.find(v)) << ' '
                     << d.siz[d.find(u)] << '\n';
        });
    }
    return 0;
}
```

模板题使用：LOJ121 动态图连通性

题意：初始空的无向简单图，点号 1..n；0 加不存在的边、1 删存在的边、2 查连通性，输出 Y/N。n≤5000，q≤500000；端点统一排序，加删配成半开生存区间，剩余边持续到 q。

所需模板：RollbackDSU（第 6 页，2.1.2 节）、TimeConnectivity（第 11 页，2.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/121

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    TimeConnectivity tree(n, q);
    map<pair<int, int>, int> start;
    vector<pair<int, int>> query(q, {-1, -1});
    for (int t = 0; t < q; t++)
    {
        int op, u, v;
        cin >> op >> u >> v;
        if (u > v) swap(u, v);
        pair<int, int> edge{u, v};
        if (op == 0)
            start[edge] = t;
        else if (op == 1)
        {
            auto it = start.find(edge);
            tree.add(it->second, t, u, v);
            start.erase(it);
        }
        else
            query[t] = edge;
    }
    for (auto [edge, l] : start)
        tree.add(l, q, edge.first, edge.second);
    tree.run([&](int t, const RollbackDSU &d)
    {
        auto [u, v] = query[t];
        if (u != -1)
            cout << (d.find(u) == d.find(v) ? 'Y' : 'N') << '\n';
    });
}
```

第三章　堆

3.1　配对堆

3.1.1　PBDS 可修改、可合并优先队列

pheap<T,C> 默认比较器为 less<T>，固定 pairing_heap_tag，形成大根堆；小根堆用 greater<T>。

包含 ext/pb_ds/priority_queue.hpp。push(x) 返回 point_iterator 句柄；modify(h,x) 改值，erase(h) 删除指定元素。top/pop 要求非空，size/empty/clear 沿用容器接口。a.join(b) 把 b 全部移入 a，b 变空；原 b 元素句柄仍指向同一元素，此后应交给 a。a.swap(b) 后句柄随元素交换归属。split(pred,b) 将满足谓词的元素移入 b，本模板约定 b 原先为空。join/split 两个堆须不同，比较规则一致。删除元素、pop 对应堆顶、clear 或销毁后，相应句柄失效；不要把原堆句柄用于深复制的副本。top、push、join 为 O(1)；pop 摊还 O(log n)，modify/erase 的单次最坏可达 O(n)，不当作最坏常数。split/复制/clear 可能扫描整个堆。适合需要修改已入堆元素或合并的任务；只要普通入队出队时可用 std::priority_queue。复杂度按比较、复制标量或编号对为常数计，T 须满足容器的复制要求。

```
template <class T, class C = less<T>>
using pheap = __gnu_pbds::priority_queue<T, C, __gnu_pbds::pairing_heap_tag>;
```

```
pheap<int> a, b;
auto h = a.push(7);
a.modify(h, 2);
b.join(a);
b.erase(h);
```

模板题使用：Library Checker Double-Ended Priority Queue

题意：多重集合支持插入和删除一个最小/最大值；0≤n≤500000、1≤q≤500000，值在 $[-10^9,10^9]$，删除时保证非空。用两个大根配对堆分别保存 (-值,id)/(值,id)，同次插入保存两份句柄；删除时先 erase 另一堆的句柄，再 pop 当前堆。id 每次新插入都递增，重复值也可区分。删除后两份句柄失效，禁止再次使用。本题取负在 int 范围内；全 int64 值不能直接照搬取负。

所需模板：pheap（第 17 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/double_ended_priority_queue

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using H = pheap<pair<int, int>>;
    array<H, 2> q;
    vector<array<H::point_iterator, 2>> h;
    h.reserve(n + m);
    auto add = [&](int x)
    {
        int id = h.size();
        h.push_back({q[0].push({-x, id}), q[1].push({x, id})});
    };
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        add(x);
    }
    while (m--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int x;
            cin >> x;
            add(x);
        }
        else
        {
            int k = t - 1;
            auto [x, id] = q[k].top();
            cout << (k ? x : -x) << '\n';
            q[k ^ 1].erase(h[id][k ^ 1]);
            q[k].pop();
        }
    }
}
```

模板题使用：Library Checker Shortest Path

题意：非负权简单有向图，0 起点号，n 和 m 不超过 500000，权 $\leq 10^9$，s≠t。输出最短距离和一条有序路径，无解输出-1。小根配对堆存 (距离, 点)，每个堆内点保存句柄；松弛时已入堆则 modify 降键，否则 push。pop 后置 in=false，不再访问旧句柄。距离用 long long，pre 还原原边；零权边允许。这个用法展示 push/top/pop/modify，不覆盖 join/split。

所需模板：pheap（第 17 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/shortest_path

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
    }
    using P = pair<long long, int>;
    pheap<P, greater<P>> q;
    vector<decltype(q)::point_iterator> h(n);
    const long long inf = LLONG_MAX;
    vector<long long> d(n, inf);
    vector<int> pre(n, -1);
    vector<bool> in(n);
    d[s] = 0;
    h[s] = q.push({0, s});
    in[s] = true;
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        in[u] = false;
        for (auto [v, w] : g[u])
        {
            if (du + w >= d[v]) continue;
            d[v] = du + w;
            pre[v] = u;
            if (in[v])
                q.modify(h[v], {d[v], v});
            else
            {
                h[v] = q.push({d[v], v});
                in[v] = true;
            }
        }
    }
```

继续处理操作（接上页同一 main）：

```
    if (d[t] == inf)
    {
        cout << -1 << '\n';
        return 0;
    }
    vector<pair<int, int>> path;
    for (int v = t; v != s; v = pre[v]) path.push_back({pre[v], v});
    reverse(path.begin(), path.end());
    cout << d[t] << ' ' << path.size() << '\n';
    for (auto [u, v] : path) cout << u << ' ' << v << '\n';
}
```

3.2 左偏树

3.2.1 左偏树可并小根堆

默认构造为空节点池；singleton(key) 创建单点堆并返回节点编号，编号从 1 开始，0 表示空堆。meld(x,y) 原地合并两个不相交堆并返回新根，调用后旧根不能再当独立堆使用；pop(root) 删除非空堆的最小节点并返回新根，必须保存返回值。相同 key 按节点编号小者优先，t[root].key 为最小值，root 本身就是最小节点编号。key 支持完整 long long；仅比较而不做减法。dist[空]=0，叶子 dist=1，每点左子树 dist 不小于右子树；递归沿右链合并，meld/pop 为 O(log(n+1))，其中 n 为参与堆节点数。singleton 均摊 O(1)，节点池空间 O(累计创建数)，节点不回收，不保留版本；pop 后删除节点已脱离堆，不应继续作为根操作。只保证根操作，不提供任意点所属堆查询，P3377 驱动另用并查集和删除标记维护。不要改写内部 l/r/dist，也不要合并有交集的堆；assert(x!=y) 不能检测所有非法重叠。

```
struct LeftistHeap
{
    struct Node
    {
        long long key;
        int l, r, dist;
    };

    vector<Node> t{{0, 0, 0, 0}};

    int singleton(long long key)
    {
        t.push_back({key, 0, 0, 1});
        return int(t.size()) - 1;
    }

    int meld(int x, int y)
    {
        if (!x || !y) return x ? x : y;
        assert(x != y);
        if (pair(t[x].key, x) > pair(t[y].key, y)) swap(x, y);
        t[x].r = meld(t[x].r, y);
        if (t[t[x].l].dist < t[t[x].r].dist) swap(t[x].l, t[x].r);
        t[x].dist = t[t[x].r].dist + 1;
        return x;
    }

    int pop(int x)
    {
        assert(x > 0);
        int root = meld(t[x].l, t[x].r);
        t[x].l = t[x].r = 0;
        t[x].dist = 1;
        return root;
    }
};
```

模板题使用：Luogu P3377

题意：合并指定元素所在的堆，或弹出其堆最小值；已删除元素的操作无效。同值先删先输入者。并查集代表不一定是堆根，root 单独保存；节点编号比输入数组下标大 1。

所需模板：LeftistHeap（第 21 页，3.2.1 节）、dsu（第 4 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3377

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LeftistHeap heap;
    dsu sets(n);
    vector<int> root(n);
    vector<bool> removed(n);
    for (int i = 0; i < n; i++)
    {
        long long key;
        cin >> key;
        root[i] = heap.singleton(key);
    }
    while (m--)
    {
        int op, x;
        cin >> op >> x;
        x--;
        if (op == 1)
        {
            int y;
            cin >> y;
            y--;
            if (removed[x] || removed[y]) continue;
            int a = sets.find(x), b = sets.find(y);
            if (a == b) continue;
            root[a] = heap.meld(root[a], root[b]);
            sets.merge(a, b);
        }
        else if (removed[x])
            cout << -1 << '\n';
        else
        {
            int a = sets.find(x), id = root[a];
            cout << heap.t[id].key << '\n';
            removed[id - 1] = true;
            root[a] = heap.pop(id);
        }
    }
}
```

第四章　块状数据结构

4.1　树分块

4.1.1　树分块与在线路径去重

TreeBlockDistinct(n), 1≤n<INT_MAX, 点 1..n; add(u,v) 加入无向边, 调用方保证恰为一棵树。build(values, b=512, root=1) 离散化 signed64 点颜色，values 长度 n+1 且忽略 0 号；b≥1，root 为合法点。query(u,v) 返回闭路径不同颜色数，可强制在线；无修改操作，改颜色或加边后必须重新 build 且仍保证树结构。支持换根、换块长、重建和值复制。

递归按残余子树大小选关键点并补根，关键点数 K≤floor(n/B)+1，每点到最近关键祖先距离 <B。预存每对关键点路径颜色位集，重复颜色用计数回溯。令 C 为颜色种数、W=ceil(C/64)，预处理 O(n log n+Kn+K^2W)，空间 O(n log n+K^2W+n)，查询 O(B+W+log n)。n=40000、B=512 时 K≤79，位集表最多约 31.2MB；B 过小会导致平方级关键点表，不保证任意块长都能装入内存。DFS 保持递归，按题目内存提供足够调用栈。

查询在 LCA 分成两条祖先链；长链选 LCA 下方 B-1 层的 x，以 near[x] 为上端关键点，保证它不会越过 LCA。不能直接使用 near[LCA]。最后显式加入 LCA 颜色。g、up、c、dep、id、near、keys、paths 等构建后只读；partition、mark、collect、leg 为内部过程，jump 要求 0≤steps≤dep[u]。

结构与关键点选择:

```
struct TreeBlockDistinct
{
    using ull = unsigned long long;
    int n, block, lg, words, k;
    vector<vector<int>> g, up;
    vector<int> c, dep, id, near, keys, count;
    vector<vector<ull>> paths;
    vector<ull> bits;

    TreeBlockDistinct(int n) : n(n), g(n + 1)
    {
        assert(n >= 1 && n < INT_MAX);
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int partition(int u, int p)
    {
        up[0][u] = p;
        for (int j = 1; j < lg; j++)
            up[j][u] = up[j - 1][up[j - 1][u]];
        int size = 1;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            size += partition(v, u);
        }
        if (size < block) return size;
        id[u] = keys.size();
        keys.push_back(u);
        return 0;
    }
```

最近关键祖先与路径位集（接上页同一结构体）：

```
    void mark(int u, int p, int last)
    {
        if (id[u] >= 0) last = u;
        near[u] = last;
        for (int v : g[u])
            if (v != p) mark(v, u, last);
    }

    void collect(int u, int p, int source)
    {
        int x = c[u];
        if (count[x]++ == 0) bits[x >> 6] |= 1ULL << (x & 63);
        if (id[u] >= 0) paths[size_t(source) * k + id[u]] = bits;
        for (int v : g[u])
            if (v != p) collect(v, u, source);
        if (--count[x] == 0) bits[x >> 6] &= ~(1ULL << (x & 63));
    }
```

分块界 每个按残余大小选出的关键点消耗至少 B 个尚未归入更深关键点的点，故数量不超过 floor(n/B)，补根后至多再加 1。连续 B 个非关键祖先会使其中最高点的残余大小至少 B，矛盾；因此 near 距离严格小于 B。

重复颜色 collect 沿当前简单路径维护颜色出现次数，回溯时只有计数归零才能清除对应位。每个关键点作为源递归遍历整棵树，遇到另一个关键点就保存当前位集。

离散化与从零重建（接上页同一结构体）：

```
    void build(const vector<long long> &values, int b = 512, int root = 1)
    {
        assert(values.size() == size_t(n + 1) && b >= 1);
        assert(1 <= root && root <= n);
        block = b;
        vector<long long> all(values.begin() + 1, values.end());
        sort(all.begin(), all.end());
        all.erase(unique(all.begin(), all.end()), all.end());
        c.resize(n + 1);
        for (int u = 1; u <= n; u++)
            c[u] = lower_bound(all.begin(), all.end(), values[u]) - all.begin();
        words = (all.size() + 63) / 64;
        count.assign(all.size(), 0);
        bits.assign(words, 0);
        dep.assign(n + 1, 0);
        near.assign(n + 1, 0);
        id.assign(n + 1, -1);
        keys.clear();
        lg = 1;
        while ((1LL << lg) <= n) lg++;
        up.assign(lg, vector<int>(n + 1));
        partition(root, 0);
        if (id[root] < 0)
        {
            id[root] = keys.size();
            keys.push_back(root);
        }
        mark(root, 0, root);
        k = keys.size();
        paths = vector<vector<ull>>(size_t(k) * k);
        for (int i = 0; i < k; i++) collect(keys[i], 0, i);
    }
```

倍增祖先与 LCA（接上页同一结构体）：

```
    int jump(int u, int steps) const
    {
        for (int j = 0; j < lg; j++)
            if (steps >> j & 1) u = up[j][u];
        return u;
    }

    int lca(int u, int v) const
    {
        if (dep[u] < dep[v]) swap(u, v);
        u = jump(u, dep[u] - dep[v]);
        if (u == v) return u;
        for (int j = lg - 1; j >= 0; j--)
        {
            if (up[j][u] != up[j][v])
            {
                u = up[j][u];
                v = up[j][v];
            }
        }
        return up[0][u];
    }
```

上端关键点不能越过 LCA 设当前祖先链终点为 p。长链中 x 在 p 下方 B-1 层，near[x] 距 x 至多 B-1，所以 b=near[x] 仍在 p 的子树内；a=near[u] 位于 b 下方或等于 b。关键路径 a 至 b 之外，两端各不足 B 步，枚举这些短段即可。短链直接枚举，最后补上 p 的颜色。

空间选型 令 W=ceil(C/64)。K 个源的遍历需 O(Kn)，保存 K 平方份位集需 $O(K^2W)$ 时间和空间；另有倍增表。默认 B=512 针对四万点量级，不能把 B 任意缩小后仍假定线性内存。

拼接路径与在线查询（接上页同一结构体）：

```
    void leg(int u, int ancestor, vector<ull> &ans) const
    {
        if (dep[u] - dep[ancestor] >= block)
        {
            int a = near[u];
            int x = jump(u, dep[u] - dep[ancestor] - block + 1);
            int b = near[x];
            const auto &path = paths[size_t(id[a]) * k + id[b]];
            for (int i = 0; i < words; i++) ans[i] |= path[i];
            while (u != a)
            {
                ans[c[u] >> 6] |= 1ULL << (c[u] & 63);
                u = up[0][u];
            }
            u = b;
        }
        while (u != ancestor)
        {
            ans[c[u] >> 6] |= 1ULL << (c[u] & 63);
            u = up[0][u];
        }
    }

    int query(int u, int v) const
    {
        int p = lca(u, v);
        vector<ull> ans(words);
        leg(u, p, ans);
        leg(v, p, ans);
        ans[c[p] >> 6] |= 1ULL << (c[p] & 63);
        int result = 0;
        for (ull x : ans) result += __builtin_popcountll(x);
        return result;
    }
};
```

模板题使用：P6177 【模板】树分块 / Count on a tree II

题意：静态点颜色，u 与上次答案异或解码；在线输出闭路径颜色种数。

所需模板：TreeBlockDistinct（第 23 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6177

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n + 1);
    for (int u = 1; u <= n; u++) cin >> a[u];
    TreeBlockDistinct g(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u, v);
    }
    g.build(a);
    int last = 0;
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        u ^= last;
        last = g.query(u, v);
        cout << last << '\n';
    }
}
```

第五章　ST 表

5.1　位掩码静态区间最值与位置

StaticRMQ<T,Cmp>(a,cmp) 复制数组；默认 T=long long、Cmp=less<T> 求最小，greater<T> 求最大，也可传有状态比较器。query(l,r) 返回 0-based 半开区间 [l,r) 内最优元素的原编号，并列总取最左；值读 rmq.a[编号]。要求 0≤l<r≤n；空数组可构造但不可查询；重新赋值新对象重建，不支持修改。比较器必须是稳定的严格弱序、const 可调用，元素类型可复制；浮点数据不能含 NaN。

单调栈位掩码处理块内区间，块间用 ST 表；块长为 n 的二进制位数（空数组取 1），n≤INT_MAX/2 且内存可分配，32 位掩码足够。在复制/比较/位操作 O(1) 的模型下，预处理 O(n)、查询 O(1)、空间 O(n)。所有成员约定只读，better/small 是内部辅助；small 仅供非空同块区间。只比较元素，不做元素算术，signed64 极值合法。

```
template <class T = long long, class Cmp = less<T>>
struct StaticRMQ
{
    int n, b;
    vector<T> a;
    vector<uint32_t> mask;
    vector<vector<int>> st;
    Cmp cmp;

    StaticRMQ(const vector<T> &v, Cmp cmp = Cmp()) : a(v), cmp(cmp)
    {
        assert(v.size() <= INT_MAX / 2);
        n = v.size();
        b = max(1, (int)bit_width((uint32_t)n));
        mask.resize(n);
        if (!n) return;
        st.push_back({});
        uint32_t s = 0;
        for (int i = 0; i < n; i++)
        {
            int start = i - i % b;
            if (i % b == 0) s = 0;
            while (s && cmp(a[i], a[start + bit_width(s) - 1]))
            {
                s ^= uint32_t(1) << (bit_width(s) - 1);
            }
            s |= uint32_t(1) << (i % b);
            mask[i] = s;
            if (i % b == b - 1 || i == n - 1)
            {
                st[0].push_back(start + countr_zero(s));
            }
        }
```

块间表构建、最优位置合并与查询（接上页同一结构体）：

```
        int m = st[0].size();
        for (int k = 1; (1 << k) <= m; k++)
        {
            st.push_back(vector<int>(m - (1 << k) + 1));
            for (int i = 0; i + (1 << k) <= m; i++)
            {
                st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    int better(int x, int y) const
    {
        if (cmp(a[x], a[y])) return x;
        if (cmp(a[y], a[x])) return y;
        return min(x, y);
    }

    int small(int l, int r) const
    {
        return l + countr_zero(mask[r - 1] >> (l % b));
    }

    int query(int l, int r) const
    {
        assert(0 <= l && l < r && r <= n);
        int x = l / b, y = (r - 1) / b;
        if (x == y) return small(l, r);
        int ans = better(small(l, (x + 1) * b), small(y * b, r));
        x++;
        if (x < y)
        {
            int k = bit_width((uint32_t)(y - x)) - 1;
            ans = better(ans, better(st[k][x], st[k][y - (1 << k)]));
        }
        return ans;
    }
};
```

模板题使用：Library Checker Static RMQ

题意：静态数组上查询 0-based 半开区间 [l,r) 最小值；n,q≤500000，值为 0..10^9。模板返回最左最小值位置，通过 rmq.a[位置] 输出值。

所需模板：StaticRMQ（第 30 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/staticrmq

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    StaticRMQ<int> rmq(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << rmq.a[rmq.query(l, r)] << '\n';
    }
}
```

模板题使用：Luogu P3865 【模板】ST 表 && RMQ 问题

题意：静态区间最大值；输入 1-based 闭区间 [l,r]，转换为 query(l-1,r)。用 greater<int> 选择最大值，再由返回位置读取值。

所需模板：StaticRMQ（第 30 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3865

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    StaticRMQ<int, greater<int>> rmq(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << rmq.a[rmq.query(l - 1, r)] << '\n';
    }
}
```

接口用法演示：静态区间最左最小/最大位置（接口演示）

演示约定：输入 n,q、signed64 数组和 q 个非空 0-based 半开区间，分别输出最左最小与最左最大位置。允许重复值；n=0 时要求 q=0。

所需模板：StaticRMQ（第 30 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/sparse-table/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    StaticRMQ<long long> mn(a);
    StaticRMQ<long long, greater<long long>> mx(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << mn.query(l, r) << ' ' << mx.query(l, r) << '\n';
    }
}
```

5.2 二维静态矩形最值与位置

StaticRMQ2D<T,Cmp>(a,cmp) 复制矩形矩阵；默认 T=long long、less 求最小，greater 求最大。

query(x1,y1,x2,y2) 查询 0-based 半开矩形 [x1,x2)×[y1,y2)，返回最优元素的 (row,col)；并列按行号再列号取最小，值读 rmq.a[row][col]。要求 0≤x1<x2≤n、0≤y1<y2≤m；空矩阵或零列可构造但不可查询，所有行必须等长。重新赋值新对象重建，不支持修改。

比较器须稳定严格弱序、const 可调用，支持有状态比较器和可复制元素，不接受 NaN；不做元素算术，signed64 极值合法。n,m 及 n*m≤INT_MAX/2 且内存可分配；所有成员只读，better 是内部辅助。四块 ST 查询 O(1)，非空矩阵的预处理和空间 O(nm log(n+1) log(m+1))（元素比较/复制 O(1) 时）。每个层对保存完整 n*m 个 int 位置，空间较大：305×305 时仅 st 的 int 数据约 28.7MiB，构造前需计算内存。

```
template <class T = long long, class Cmp = less<T>>
struct StaticRMQ2D
{
    int n, m;
    vector<vector<T>> a;
    vector<vector<vector<int>>> st;
    Cmp cmp;

    StaticRMQ2D(const vector<vector<T>> &v, Cmp cmp = Cmp()) : a(v), cmp(cmp)
    {
        assert(v.size() <= INT_MAX / 2);
        n = v.size();
        assert(n == 0 || v[0].size() <= INT_MAX / 2);
        m = n ? v[0].size() : 0;
        for (const auto &row : v) assert(row.size() == (size_t)m);
        assert(1LL * n * m <= INT_MAX / 2);
        if (!n || !m) return;
        int h = bit_width((unsigned)n), w = bit_width((unsigned)m);
        st.assign(h, vector<vector<int>>(w, vector<int>(n * m)));
        iota(st[0][0].begin(), st[0][0].end(), 0);
```

层表递推、位置比较与四块查询（接上页同一结构体）：

```
        for (int k = 0; k < h; k++)
        {
            for (int l = 0; l < w; l++)
            {
                if (k == 0 && l == 0) continue;
                for (int x = 0; x + (1 << k) <= n; x++)
                {
                    for (int y = 0; y + (1 << l) <= m; y++)
                    {
                        int id = x * m + y;
                        if (k)
                        {
                            st[k][l][id] = better(st[k - 1][l][id],
                                st[k - 1][l][id + (1 << (k - 1)) * m]);
                        }
                        else
                        {
                            st[k][l][id] = better(st[k][l - 1][id],
                                st[k][l - 1][id + (1 << (l - 1))]);
                        }
                    }
                }
            }
        }
    }

    int better(int x, int y) const
    {
        const auto &u = a[x / m][x % m], &v = a[y / m][y % m];
        if (cmp(u, v)) return x;
        if (cmp(v, u)) return y;
        return min(x, y);
    }

    pair<int, int> query(int x1, int y1, int x2, int y2) const
    {
        assert(0 <= x1 && x1 < x2 && x2 <= n);
        assert(0 <= y1 && y1 < y2 && y2 <= m);
        int k = bit_width((unsigned)(x2 - x1)) - 1;
        int l = bit_width((unsigned)(y2 - y1)) - 1;
        x2 -= 1 << k;
        y2 -= 1 << l;
        const auto &t = st[k][l];
        int top = better(t[x1 * m + y1], t[x1 * m + y2]);
        int bottom = better(t[x2 * m + y1], t[x2 * m + y2]);
        int id = better(top, bottom);
        return {id / m, id % m};
    }
};
```

接口用法演示：二维矩形最小/最大位置（接口演示）

演示约定：输入 n,m,q、矩阵与 q 个 0-based 非空半开矩形，依次输出最小值的行列和最大值的行列；并列按行再列取最小。值支持 signed64；空矩阵或零列时 q 必须为 0。

所需模板：StaticRMQ2D（第 34 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/sparse-table/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<vector<long long>> a(n, vector<long long>(m));
    for (auto &row : a)
    {
        for (auto &x : row) cin >> x;
    }
    StaticRMQ2D<long long> mn(a);
    StaticRMQ2D<long long, greater<long long>> mx(a);
    while (q--)
    {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        auto [u, v] = mn.query(x1, y1, x2, y2);
        auto [s, t] = mx.query(x1, y1, x2, y2);
        cout << u << ' ' << v << ' ' << s << ' ' << t << '\n';
    }
}
```

接口用法演示：矩形最大值是否出现在四角（kuangbin 源码模型）

演示约定：读到 EOF，每组 n,m、矩阵、q 和 q 个 1-based 闭矩形端点，分别交换逆序的行/列端点。输出最大值与 yes/no，表示四个角是否至少有一个等于最大值。演示 n,m 为 1..305，值为 signed64；不是已核验的原 OJ 协议、资源或正式模板题。

所需模板：StaticRMQ2D（第 34 页，5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        vector<vector<long long>> a(n, vector<long long>(m));
        for (auto &row : a)
        {
            for (auto &x : row) cin >> x;
        }
        StaticRMQ2D<long long, greater<long long>> rmq(a);
        int q;
        cin >> q;
        while (q--)
        {
            int x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            if (x1 > x2) swap(x1, x2);
            if (y1 > y2) swap(y1, y2);
            auto [x, y] = rmq.query(x1 - 1, y1 - 1, x2, y2);
            long long value = rmq.a[x][y];
            bool corner = value == a[x1 - 1][y1 - 1] || value == a[x1 - 1][y2 - 1] ||
                          value == a[x2 - 1][y1 - 1] || value == a[x2 - 1][y2 - 1];
            cout << value << ' ' << (corner ? "yes" : "no") << '\n';
        }
    }
}
```

第六章　树状数组

6.1　稀疏二维树状数组：点加与矩形和

Fenwick2D<T>(points)，T 默认 long long；预登记所有将来更新的 (x,y)，重复点自动去重，初始权值全 0。add(x,y,v) 在已登记点累加，删除可加负值；不是不预知坐标的在线数据结构。prefix(x,y) 求横坐标 <x 且纵坐标 <y 的权和，sum(l,d,r,u) 求 [l,r)×[d,u)，允许空矩形；坐标、端点均为 long long，不做端点 ±1。上界须可表示，因此不能用有限半开区间包含 LLONG_MAX 坐标。init(points) 重新登记并清空权值，空点集合法。K 为登记项数（含重复），P 为不同点数且 P<2^30：构造 O(K log K+P log P)，构造峰值空间 O(K+P log P)，更新与查询 $O(\log^2 P)$。按 y 有序分发到各横轴 Fenwick 节点，使内层坐标有序且去重。T 支持零值、加减，所有节点和与容斥中间值均须可表示。内部字段只读，不提供未经登记的动态插入。

```
template <class T = long long> struct Fenwick2D
{
    using ll = long long;
    int n = 0;
    vector<pair<ll, ll>> pt;
    vector<ll> xs;
    vector<vector<ll>> ys;
    vector<vector<T>> bit;

    Fenwick2D(vector<pair<ll, ll>> p = {})
    {
        init(move(p));
    }

    // Register every future update coordinate; all weights start at zero.
    void init(vector<pair<ll, ll>> p)
    {
        sort(p.begin(), p.end());
        p.erase(unique(p.begin(), p.end()), p.end());
        assert(p.size() < (1U << 30));
        pt = p;
        xs.clear();
        for (auto [x, y] : p)
            if (xs.empty() || xs.back() != x) xs.push_back(x);
        n = xs.size();
        ys.assign(n + 1, {});
        bit.assign(n + 1, {});
        sort(p.begin(), p.end(), [](auto a, auto b)
        {
            return a.second < b.second;
        });
        for (auto [x, y] : p)
        {
            int k = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
            for (int i = k; i <= n; i += i & -i)
                if (ys[i].empty() || ys[i].back() != y) ys[i].push_back(y);
        }
        for (int i = 1; i <= n; i++) bit[i].assign(ys[i].size() + 1, T{});
    }
```

更新与半开矩形求和（接上页同一结构体）：

```
    void add(ll x, ll y, T v)
    {
        assert(binary_search(pt.begin(), pt.end(), pair<ll, ll>{x, y}));
        int k = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
        for (int i = k; i <= n; i += i & -i)
        {
            int j = lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin() + 1;
            for (; j < (int)bit[i].size(); j += j & -j) bit[i][j] += v;
        }
    }

    // Sum over coordinates strictly less than x and y; no endpoint +/-1.
    T prefix(ll x, ll y) const
    {
        T ans{};
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        for (; i; i -= i & -i)
        {
            int j = lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin();
            for (; j; j -= j & -j) ans += bit[i][j];
        }
        return ans;
    }

    T sum(ll l, ll d, ll r, ll u) const
    {
        assert(l <= r && d <= u);
        if (l == r || d == u) return T{};
        return prefix(r, u) - prefix(l, u) - prefix(r, d) + prefix(l, d);
    }
};
```

模板题使用：Library Checker point_add_rectangle_sum

题意：N,Q≤100000，初始 N 个带权平面点。0 x y w 加入点权，重合点累加；1 l d r u 查询 [l,r)×[d,u) 内总权。坐标和权值 0..10^9，查询边界严格递增。先读完所有操作，预登记初始与未来增加点的坐标，再按原顺序执行；这不改变原题操作语义。

所需模板：Fenwick2D（第 38 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_add_rectangle_sum

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q;
    std::cin >> n >> q;
    using ll = long long;
    std::vector<std::array<ll, 3>> a(n);
    std::vector<std::pair<ll, ll>> p;
    for (auto &[x, y, w] : a)
    {
        std::cin >> x >> y >> w;
        p.push_back({x, y});
    }
    std::vector<std::array<ll, 5>> ops(q);
    for (auto &[op, x, y, r, u] : ops)
    {
        std::cin >> op >> x >> y >> r;
        if (op == 0) p.push_back({x, y});
        else std::cin >> u;
    }
    Fenwick2D tree(p);
    for (auto [x, y, w] : a) tree.add(x, y, w);
    for (auto [op, x, y, r, u] : ops)
    {
        if (op == 0) tree.add(x, y, r);
        else std::cout << tree.sum(x, y, r, u) << '\n';
    }
}
```

6.2 四矩二维树状数组：矩形加与矩形和

RectangleFenwick<T>(n,m)，T 默认 long long，0≤n,m<2^30，初始化全 0；稠密空间与构造 O((n+1)(m+1))，需先确认可分配。add(l,d,r,u,v) 给 0-based 半开矩形 [l,r)×[d,u) 加 v；sum(l,d,r,u) 查询同一区间，prefix(x,y) 查询 [0,x)×[0,y)。端点须在 [0,n]×[0,m]，允许空矩形及零维矩阵。差分四角维护 D、xD、yD、xyD 四个矩，前缀和为 xyS0-ySx-xSy+Sxy；每次操作 O(log n log m)，实际存储每格 4 个 T。单点操作令右上界各加 1。T 支持零值、加减、取负和乘 int；包括-v、各坐标乘积及容斥在内的中间值均须可表示，仅最终答案可表示不够。内部 bit 与 corner 不供直接修改。重新赋值 RectangleFenwick(n,m) 清空；不支持稀疏大坐标自动压缩，因为区间加仍取决于真实格子数量。

```
template <class T = long long> struct RectangleFenwick
{
    int n, m;
    vector<vector<array<T, 4>>> bit;

    RectangleFenwick(int n, int m) : n(n), m(m)
    {
        assert(0 <= n && n < (1 << 30) && 0 <= m && m < (1 << 30));
        bit.assign(n + 1, vector<array<T, 4>>(m + 1));
    }

    // Difference impulse at (x,y); boundary impulses do not affect the grid.
    void corner(int x, int y, T v)
    {
        if (x == n || y == m) return;
        array<T, 4> a{v, v * x, v * y, v * x * y};
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j)
                for (int k = 0; k < 4; k++) bit[i][j][k] += a[k];
    }

    // Add to [l,r) x [d,u), 0-based. Empty rectangles are allowed.
    void add(int l, int d, int r, int u, T v)
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return;
        corner(l, d, v);
        corner(r, d, -v);
        corner(l, u, -v);
        corner(r, u, v);
    }
```

更新与半开矩形求和（接上页同一结构体）：

```
    T prefix(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= m);
        array<T, 4> a{};
        for (int i = x; i; i -= i & -i)
            for (int j = y; j; j -= j & -j)
                for (int k = 0; k < 4; k++) a[k] += bit[i][j][k];
        return a[0] * x * y - a[1] * y - a[2] * x + a[3];
    }

    T sum(int l, int d, int r, int u) const
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return T{};
        return prefix(r, u) - prefix(l, u) - prefix(r, d) + prefix(l, d);
    }
};
```

模板题使用：Luogu P4514 / 上帝造题的七分钟

题意：输入 X n m 初始化零矩阵，n,m≤2048；读到 EOF，L a b c d delta 给 1-based 闭矩形加 delta，k a b c d 查询区域和（小写 k）。至多 200000 操作，|delta|≤500；转换为 [a-1,c)×[b-1,d)。最终答案在 32 位内，中间值未保证，使用 long long。

所需模板：RectangleFenwick（第 41 页，6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4514

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    char op;
    int n, m;
    std::cin >> op >> n >> m;
    RectangleFenwick tree(n, m);
    int a, b, c, d;
    while (std::cin >> op >> a >> b >> c >> d)
    {
        if (op == 'L')
        {
            long long v;
            std::cin >> v;
            tree.add(a - 1, b - 1, c, d, v);
        }
        else
            std::cout << tree.sum(a - 1, b - 1, c, d) << '\n';
    }
}
```

6.3 树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。$n \le 100000$，$|x| \le 10000000$；总复杂度 $O(n \log n)$，空间 $O(n)$。

所需模板：Fenwick（第 43 页，6.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

相关应用：离线二维扫描线应用，不代表整页覆盖

6.4 离线静态矩形权值和

代码依赖：Fenwick（第 43 页，6.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

offline_rectangle_sum(points,queries)：points 每项为 {x,y,w}，queries 每项为 {l,d,r,u}，返回按询问原顺序排列的 [l,r)×[d,u) 内权值和。坐标、端点与权值均为 long long；要求 l<=r、d<=u，允许空矩形、空点集、空询问、重合点和负权。传入点按值复制，两个输入均不改变，多次调用互不影响。坐标仅比较，不用端点 ±1，支持 LLONG_MIN/MAX；半开矩形无法包含位于 LLONG_MAX 的上边界坐标。

一次读完所有询问，按横坐标扫描，在纵轴离散化树状数组上加点。同横坐标事件只包含 x< 事件横坐标的点，保证左右开闭正确。时间 O((N+Q)log(N+Q+1))，空间 O(N+Q)，N,Q 均小于 2^30 且内存可分配。要求所有权值绝对值之和 <=LLONG_MAX，保证树状数组、容斥与符号乘法不溢出。依赖 Fenwick；若需逐条在线回答，改用持久化前缀版本。

```
vector<long long> offline_rectangle_sum(vector<array<long long, 3>> points,
                                        const vector<array<long long, 4>> &queries)
{
    using ll = long long;
    assert(points.size() < (1U << 30) && queries.size() < (1U << 30));
    vector<ll> ys;
    for (auto [x, y, w] : points)
        ys.push_back(y);
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    auto rank = [&](ll y)
    {
        return int(lower_bound(ys.begin(), ys.end(), y) - ys.begin());
    };
    struct Event
    {
        ll x;
        int d, u, id, sign;
    };
    vector<Event> events;
    events.reserve(2 * queries.size());
    for (int i = 0; i < (int)queries.size(); i++)
    {
        auto [l, d, r, u] = queries[i];
        assert(l <= r && d <= u);
        if (l == r || d == u)
            continue;
        events.push_back({l, rank(d), rank(u), i, -1});
        events.push_back({r, rank(d), rank(u), i, 1});
    }
    sort(points.begin(), points.end());
    sort(events.begin(), events.end(), [](const Event &a, const Event &b)
    {
        return a.x < b.x;
    });
    Fenwick<ll> bit(ys.size());
    vector<ll> answer(queries.size());
    int p = 0;
    for (auto e : events)
    {
```

```
        while (p < (int)points.size() && points[p][0] < e.x)
        {
            auto [x, y, w] = points[p];
            bit.add(rank(y) + 1, w);
            p++;
        }
        ll sum = bit.sum(e.u) - bit.sum(e.d);
        answer[e.id] += e.sign * sum;
    }
    return answer;
}
```

模板题使用：Library Checker Rectangle Sum（离线）

题意：N,Q<=200000，点坐标与权值在 [0,1e9]；读取全部询问后用扫描线返回半开矩形权值和。此例可离线处理，节省持久化节点的内存；需要在线查询时使用持久化版本（用法 432）。Library Checker C++20 线上 AC 410016，258ms、29.98MiB。

所需模板：Fenwick（第 43 页，6.3 节）、offline_rectangle_sum（第 45 页，6.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/rectangle_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<array<long long, 3>> points(n);
    vector<array<long long, 4>> queries(q);
    for (auto &p : points)
        cin >> p[0] >> p[1] >> p[2];
    for (auto &r : queries)
        cin >> r[0] >> r[1] >> r[2] >> r[3];
    for (auto answer : offline_rectangle_sum(points, queries))
        cout << answer << '\n';
}
```

第七章　线段树

7.1　线段树基础

7.1.1　通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。0≤n≤2^28-1，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 O(log n)，构造 O(n)、操作/搜索 O(log n)、all O(1)，空间 O(n)，假设运算和类型复制 O(1)。d/tag 及其他递归辅助方法仅供内部使用。

```
template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
struct lazy_segtree
{
    int n;
    vector<S> d;
    vector<F> tag;

    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}

    lazy_segtree(const vector<S> &a)
        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
    {
        if (n) build(1, 0, n, a);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<S> &a)
    {
        if (r - l == 1)
        {
            d[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

```
    void act(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        tag[p] = composition(f, tag[p]);
    }

    void push(int p)
    {
        act(p * 2, tag[p]);
        act(p * 2 + 1, tag[p]);
        tag[p] = id();
    }

    void set(int x, S v)
    {
        assert(0 <= x && x < n);
        put(1, 0, n, x, v);
    }

    void put(int p, int l, int r, int x, S v)
    {
        if (r - l == 1)
        {
            d[p] = v;
            tag[p] = id();
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (x < m)
            put(p * 2, l, m, x, v);
        else
            put(p * 2 + 1, m, r, x, v);
        pull(p);
    }

    void apply(int l, int r, F f)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) update(1, 0, n, l, r, f);
    }

    void update(int p, int l, int r, int a, int b, F f)
    {
        if (a <= l && r <= b)
        {
            act(p, f);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (a < m) update(p * 2, l, m, a, b, f);
        if (m < b) update(p * 2 + 1, m, r, a, b, f);
        pull(p);
    }
```

```
87      S all() const { return d[1]; }
88
89      S get(int x)
90      {
91          assert(0 <= x && x < n);
92          return prod(x, x + 1);
93      }
94
95      S prod(int l, int r)
96      {
97          assert(0 <= l && l <= r && r <= n);
98          return l == r ? e() : query(1, 0, n, l, r);
99      }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }
```

```
    template <class G> int min_left(int r, G ok)
    {
        assert(0 <= r && r <= n && ok(e()));
        S s = e();
        return r == 0 ? 0 : left(1, 0, n, r, s, ok);
    }

    template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
    {
        if (b <= l) return 0;
        if (r <= b && ok(op(d[p], s)))
        {
            s = op(d[p], s);
            return 0;
        }
        if (r - l == 1) return r;
        push(p);
        int m = (l + r) / 2;
        int x = left(p * 2 + 1, m, r, b, s, ok);
        return x != 0 ? x : left(p * 2, l, m, b, s, ok);
    }
};
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 x→b*x+c，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree（第 48 页，7.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
using S = pair<long long, int>;
using F = pair<long long, long long>;
const int mod = 998244353;

S op(S a, S b)
{
    return {(a.first + b.first) % mod, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(F f, S a)
{
    return {(f.first * a.first + f.second * a.second) % mod, a.second};
}

F composition(F f, F g)
{
    return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
}

F id()
{
    return {1, 0};
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, len] : a)
    {
        cin >> x;
        len = 1;
    }
    lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
    while (q--)
    {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 0)
        {
            long long b, c;
            cin >> b >> c;
            s.apply(l, r, {b, c});
        }
        else
            cout << s.prod(l, r).first << '\n';
    }
}
```

7.1.2 通用线段树与左右边界搜索

segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

```

```
    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 53 页，7.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

7.1.3 区间加与区间和

初值全零；add(l,r,v)、query(l,r)，1≤l≤r≤n。维护 sum 与加法 tag；所有中间和与长度乘积须在 long long 范围内。单次 O(log n)。

```
struct LazySeg
{
    int n;
    vector<long long> sum, tag;

    LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4) { assert(n > 0); }

    void apply(int p, int l, int r, long long v)
    {
        sum[p] += (r - l + 1) * v;
        tag[p] += v;
    }

    void push(int p, int l, int r)
    {
        int m = (l + r) / 2;
        apply(p * 2, l, m, tag[p]);
        apply(p * 2 + 1, m + 1, r, tag[p]);
        tag[p] = 0;
    }

    void add(int l, int r, long long v) { add(1, 1, n, l, r, v); }

    void add(int p, int l, int r, int ql, int qr, long long v)
    {
        if (ql <= l && r <= qr)
        {
            apply(p, l, r, v);
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) add(p * 2, l, m, ql, qr, v);
        if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }

    long long query(int l, int r) { return query(1, 1, n, l, r); }

    long long query(int p, int l, int r, int ql, int qr)
    {
        if (ql <= l && r <= qr) return sum[p];
        push(p, l, r);
        int m = (l + r) / 2;
        long long ans = 0;
        if (ql <= m) ans += query(p * 2, l, m, ql, qr);
        if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
        return ans;
    }
};
```

模板题使用：Luogu P3372 【模板】线段树 1

题意：给出长度 n 的正整数序列与 m 次操作，1≤n,m≤100000，支持闭区间加正整数 k 与闭区间求和；任意时刻全序列之和不超过 2×10^18。LazySeg(n) 零初始化后用 n 次单点 add(i,i,a) 装入初值；修改调用 add(l,r,k)，询问调用 query(l,r)。初始化 O(n log n)，每次操作 O(log n)，空间 O(n)；初值、增量、区间和均用 long long。

所需模板：LazySeg（第 58 页，7.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3372

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LazySeg seg(n);
    for (int i = 1; i <= n; i++)
    {
        long long a;
        cin >> a;
        seg.add(i, i, a);
    }
    while (m--)
    {
        int op, l, r;
        cin >> op >> l >> r;
        if (op == 1)
        {
            long long k;
            cin >> k;
            seg.add(l, r, k);
        }
        else
            cout << seg.query(l, r) << '\n';
    }
}
```

7.1.4 线段树：区间乘、加、赋值与模意义求和

AffineSegTree(a,mod) 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。update(l,r,mul,add) 将区间元素 x 改为 mul*x+add；乘法用 (mul,0)，加法用 (1,add)，赋值用 (0,value)。query(l,r) 返回区间和。1≤mod≤INT_MAX，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 (m,b) 后应用新标记 (a,c)，复合为 (m*a,b*a+c)，乘法取模后再相加。建树 O(n)，单次 O(log n)，空间 O(n)。内部 build、apply 等函数不作为公共接口；动态版重建新对象。

```
struct AffineSegTree
{
    using ll = long long;

    struct Node
    {
        ll sum = 0, mul = 1, add = 0;
    };

    int n, mod;
```

```
    vector<Node> t;

    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
    {
        assert(n > 0 && mod > 0);
        build(1, 1, n, a);
    }

    ll norm(ll x) const
    {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }

    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        t[p] = Node{};
        if (l == r)
        {
            t[p].sum = norm(a[l - 1]);
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }

    // New transform follows the old pending transform: x -> x*a+b.
    void apply(int p, int len, ll a, ll b)
    {
        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
        t[p].mul = t[p].mul * a % mod;
        t[p].add = (t[p].add * a % mod + b) % mod;
    }

    void push(int p, int l, int r)
    {
        if (t[p].mul == 1 && t[p].add == 0) return;
        int m = (l + r) / 2;
        apply(p * 2, m - l + 1, t[p].mul, t[p].add);
        apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
        t[p].mul = 1;
        t[p].add = 0;
    }

    void update(int l, int r, ll a, ll b)
    {
        assert(1 <= l && l <= r && r <= n);
        update(1, 1, n, l, r, norm(a), norm(b));
    }

    void update(int p, int l, int r, int ql, int qr, ll a, ll b)
```

```
    {
        if (ql <= l && r <= qr)
        {
            apply(p, r - l + 1, a, b);
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
        if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
        pull(p);
    }

    ll query(int l, int r)
    {
        assert(1 <= l && l <= r && r <= n);
        return query(1, 1, n, l, r);
    }

    ll query(int p, int l, int r, int ql, int qr)
    {
        if (ql <= l && r <= qr) return t[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        ll answer = 0;
        if (ql <= m) answer += query(p * 2, l, m, ql, qr);
        if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
        return answer % mod;
    }
};
```

模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree（第 59 页，7.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3373

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q, mod;
    std::cin >> n >> q >> mod;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    AffineSegTree tree(a, mod);
    while (q--)
    {
        int op, l, r;
        std::cin >> op >> l >> r;
        if (op == 3)
            std::cout << tree.query(l, r) << '\n';
        else
        {
            long long k;
            std::cin >> k;
            if (op == 1)
                tree.update(l, r, k, 0);
            else
                tree.update(l, r, 1, k);
        }
    }
}
```

相关应用：线段树维护单调栈的双向极大值贡献；区间加接雨水应用，不将普通单调栈直接视为覆盖。

7.1.5 线段树维护单调栈：区间加与接雨水

MonotoneStackSeg(a) 要求非空 long long 数组；add(l,r,delta) 对 0-based 半开区间加值，空区间不变。water() 返回整个数组的接雨水体积。scan(l,r,height,reverse=false) 按左到右（reverse=true 则右到左）扫描，将 height 与当前元素取 max 并累加 height-a[i]，返回 (累加值, 最后的 height)；空区间返回 (0,height)。支持负高度与负增量，无固定零边界。节点缓存双向前缀极大值减原值的和；区间平移不改变两者差。建树 O(n)，add/scan 最坏 $O(\log^2(n+1))$，water 为 O(1)，空间 O(n)；递归深度 O(log n)。4n 须可由 int 表示；高度、和、懒标记、外部 height 与长度的乘积以及所有中间运算均须在 long long 范围内。t 与 extra/pull/push 及带节点编号的函数属内部实现。

```
struct MonotoneStackSeg
{
    using ll = long long;

    struct Node
    {
        int len = 0;
        ll mx = 0, sum = 0, lazy = 0;
        array<ll, 2> fill{};
    };

    int n;
    vector<Node> t;

    MonotoneStackSeg(const vector<ll> &a) : n(a.size()), t(4 * n)
    {
        assert(n > 0);
        build(1, 0, n, a);
    }

    void apply(int p, ll delta)
    {
        t[p].mx += delta;
        t[p].sum += delta * t[p].len;
        t[p].lazy += delta;
    }

    void push(int p)
    {
        if (!t[p].lazy) return;
        apply(p * 2, t[p].lazy);
        apply(p * 2 + 1, t[p].lazy);
        t[p].lazy = 0;
    }
```

单路扫描、合并与建树（接上页同一结构体）：

```
    ll extra(int p, ll height, int dir)
    {
        if (height >= t[p].mx) return height * t[p].len - t[p].sum;
        if (t[p].len == 1) return 0;
        push(p);
        int first = p * 2 + dir, second = p * 2 + 1 - dir;
        if (height >= t[first].mx)
            return height * t[first].len - t[first].sum + extra(second, height, dir);
        return extra(first, height, dir) + (t[p].fill[dir] - t[first].fill[dir]);
    }

    void pull(int p)
    {
        int l = p * 2, r = l + 1;
        t[p].len = t[l].len + t[r].len;
        t[p].mx = max(t[l].mx, t[r].mx);
        t[p].sum = t[l].sum + t[r].sum;
        t[p].fill[0] = t[l].fill[0] + extra(r, t[l].mx, 0);
        t[p].fill[1] = t[r].fill[1] + extra(l, t[r].mx, 1);
    }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            t[p].len = 1;
            t[p].mx = a[l];
            t[p].sum = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

区间加、双向查询与雨水体积（接上页同一结构体）：

```
    void add(int p, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr) return apply(p, delta);
        push(p);
        int m = (l + r) / 2;
        if (ql < m) add(p * 2, l, m, ql, qr, delta);
        if (m < qr) add(p * 2 + 1, m, r, ql, qr, delta);
        pull(p);
    }

    void add(int l, int r, ll delta)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) add(1, 0, n, l, r, delta);
    }

    ll scan(int p, int l, int r, int ql, int qr, ll &height, int dir)
    {
        if (ql <= l && r <= qr)
        {
            ll answer = extra(p, height, dir);
            height = max(height, t[p].mx);
            return answer;
        }
        push(p);
        int m = (l + r) / 2;
        ll answer = 0;
        if (dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        if (ql < m) answer += scan(p * 2, l, m, ql, qr, height, dir);
        if (!dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        return answer;
    }

    // Return (sum of running maximum - a[i], final running maximum).
    pair<ll, ll> scan(int l, int r, ll height, bool reverse = false)
    {
        assert(0 <= l && l <= r && r <= n);
        ll answer = 0;
        if (l < r) answer = scan(1, 0, n, l, r, height, reverse);
        return {answer, height};
    }

    ll water() const
    {
        ll gap = t[1].mx * n - t[1].sum;
        return t[1].fill[0] - (gap - t[1].fill[1]);
    }
};
```

应用题补充：NERC 2023 G / Luogu P12438

题意：Great City Saint Petersburg：先输出初始雨水量，每次输入 1-based 闭区间使高度加一，再输出全局雨水量。调用时只减 l。该题为比赛应用，不计正式模板题覆盖；一般负增量和 scan 接口另有本地验证。

所需模板：MonotoneStackSeg（第 63 页，7.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P12438

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MonotoneStackSeg s(a);
    cout << s.water() << '\n';
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        s.add(l - 1, r, 1);
        cout << s.water() << '\n';
    }
}
```

应用题补充：带外部高度的双向扫描（本地接口示例）

题意：输入 n,q 和高度数组，再输入 op,l,r,x；区间均为 0-based 半开。op=0 对区间加 x；op=1 从左向右、op=2 从右向左，初始最高值为 x，输出逐步最高值与当前元素之差的和，以及最终最高值。空区间保持 x，支持负数。

所需模板：MonotoneStackSeg（第 63 页，7.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MonotoneStackSeg s(a);
    while (q--)
    {
        int op, l, r;
        long long x;
        cin >> op >> l >> r >> x;
        if (op == 0)
            s.add(l, r, x);
        else
        {
            auto [cost, height] = s.scan(l, r, x, op == 2);
            cout << cost << ' ' << height << '\n';
        }
    }
}
```

7.2 线段树合并 & 分裂

7.2.1 线段树合并与分裂

MergeSplitTree(n) 建空集合 0，或传 vector<long long> 建初始重数；n>0，值域 [0,n)。add(s,x,delta) 点增减，结果重数非负；sum(s,l,r) 查询半开值域；kth(s,k) 按重数返回第 k 小的 0-based 值，不存在返回-1。split(s,l,r) 移走区间并返回新集合号，空区间也创建集合；split_rank(s,k,reverse=false) 移走最小 k 个元素到新集合；reverse=true 移走最大 k 个，k 在 [0, 集合总重数] 内，同值的重数可拆开，k=0 也创建空集合。merge(dst,src) 消耗不同的 src，src 变空后可重用。所有集合总量分别须 ≤LLONG_MAX，节点/集合索引须适合 int。仅通过集合号操作，root/t/free 只读，其余递归函数内部使用，禁止浅拷贝根。初始化 O(n)，增减/分裂/查询 O(log(n+1))；单次合并非 O(log n)，遍历重叠节点并回收，总合并开销可摊到节点创建。完整对象值复制独立。

```
struct MergeSplitTree
{
    using ll = long long;
    struct Node
    {
        int l = 0, r = 0;
        ll sum = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0}, free;

    MergeSplitTree(int n) : n(n)
    {
        assert(n > 0);
    }

    MergeSplitTree(const vector<ll> &a) : MergeSplitTree((int)a.size())
    {
        root[0] = build(0, n, a);
    }

    int make()
    {
        if (free.empty())
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free.back();
        free.pop_back();
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    void release(int p)
    {
        t[p] = Node{};
        free.push_back(p);
    }

    void pull(int p)
    {
        assert(t[t[p].l].sum <= LLONG_MAX - t[t[p].r].sum);
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    int build(int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(a[l] >= 0);
            if (!a[l]) return 0;
            int p = make();
            t[p].sum = a[l];
            return p;
        }
        int m = l + (r - l) / 2;
        int x = build(l, m, a);
        int y = build(m, r, a);
        if (!x && !y) return 0;
        int p = make();
        t[p].l = x;
        t[p].r = y;
        pull(p);
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int add_node(int p, int l, int r, int x, ll v)
    {
        if (!p) p = make();
        if (r - l == 1)
        {
            assert(v >= -t[p].sum);
            t[p].sum += v;
        }
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = add_node(t[p].l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = add_node(t[p].r, m, r, x, v);
                t[p].r = q;
            }
            pull(p);
        }
        if (t[p].sum) return p;
        release(p);
        return 0;
    }

    // Sets and coordinates are 0-based; resulting multiplicities are nonnegative.
    void add(int s, int x, ll v)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= x && x < n);
        assert(v <= 0 || t[root[s]].sum <= LLONG_MAX - v);
        if (!v) return;
        root[s] = add_node(root[s], 0, n, x, v);
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int meld(int p, int q, int l, int r)
    {
        if (!p || !q) return p ? p : q;
        if (r - l == 1)
            t[p].sum += t[q].sum;
        else
        {
            int m = l + (r - l) / 2;
            t[p].l = meld(t[p].l, t[q].l, l, m);
            t[p].r = meld(t[p].r, t[q].r, m, r);
            pull(p);
        }
        release(q);
        return p;
    }

    // Consume src; its set ID remains valid and becomes empty.
    void merge(int dst, int src)
    {
        assert(0 <= dst && dst < (int)root.size());
        assert(0 <= src && src < (int)root.size() && dst != src);
        assert(t[root[dst]].sum <= LLONG_MAX - t[root[src]].sum);
        root[dst] = meld(root[dst], root[src], 0, n);
        root[src] = 0;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    pair<int, int> cut(int p, int l, int r, int a, int b)
    {
        if (!p || b <= l || r <= a) return {p, 0};
        if (a <= l && r <= b) return {0, p};
        int m = l + (r - l) / 2;
        auto [x, u] = cut(t[p].l, l, m, a, b);
        auto [y, v] = cut(t[p].r, m, r, a, b);
        t[p].l = x;
        t[p].r = y;
        pull(p);
        if (!t[p].sum)
        {
            release(p);
            p = 0;
        }
        int q = 0;
        if (u || v)
        {
            q = make();
            t[q].l = u;
            t[q].r = v;
            pull(q);
        }
        return {p, q};
    }

    // Move [l,r) into a new set, even when the extracted set is empty.
    int split(int s, int l, int r)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        auto [p, q] = cut(root[s], 0, n, l, r);
        root[s] = p;
        root.push_back(q);
        return (int)root.size() - 1;
    }

    pair<int, int> cut_rank(int p, int l, int r, ll k, bool reverse)
    {
        if (!p || k == 0)
            return {p, 0};
        if (k == t[p].sum)
            return {0, p};
        int q = make();
        if (r - l == 1)
        {
            t[p].sum -= k;
            t[q].sum = k;
            return {p, q};
        }
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        ll right = t[t[p].r].sum;
        ll take_left = k < left ? k : left;
```

```
187         if (reverse)
188             take_left = k > right ? k - right : 0;
189         auto [a, b] = cut_rank(t[p].l, l, m, take_left, reverse);
190         auto [c, d] = cut_rank(t[p].r, m, r, k - take_left, reverse);
191         t[p].l = a;
192         t[p].r = c;
193         t[q].l = b;
194         t[q].r = d;
195         pull(p);
196         pull(q);
197         return {p, q};
198     }
199 
200     // Move the k smallest (or largest) occurrences into a new set.
201     int split_rank(int s, ll k, bool reverse = false)
202     {
203         assert(0 <= s && s < (int)root.size());
204         assert(0 <= k && k <= t[root[s]].sum);
205         auto [p, q] = cut_rank(root[s], 0, n, k, reverse);
206         root[s] = p;
207         root.push_back(q);
208         return (int)root.size() - 1;
209     }
```

线段树合并与分裂（接上页同一结构体）：

```
    ll query(int p, int l, int r, int a, int b) const
    {
        if (!p || b <= l || r <= a) return 0;
        if (a <= l && r <= b) return t[p].sum;
        int m = l + (r - l) / 2;
        return query(t[p].l, l, m, a, b) + query(t[p].r, m, r, a, b);
    }

    ll sum(int s, int l, int r) const
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        return query(root[s], 0, n, l, r);
    }

    int select(int p, int l, int r, ll k) const
    {
        if (r - l == 1) return l;
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        if (k <= left) return select(t[p].l, l, m, k);
        return select(t[p].r, m, r, k - left);
    }

    int kth(int s, ll k) const
    {
        assert(0 <= s && s < (int)root.size());
        if (k <= 0 || k > t[root[s]].sum) return -1;
        return select(root[s], 0, n, k);
    }
};
```

模板题使用：Luogu P5494 / 线段树分裂

题意：n,m≤200000，初始集合 1 含每种值 1..n 的重数 ai≤m。0 p x y 移走闭区间到新集合；1 p t 合并并清空 t，之后不再使用 t；2 p x q 增加 x 个值 q；3 p x y 查区间重数；4 p k 查第 k 小，不存在-1。集合和值均转 0-based，闭区间转 [x-1,y)，重数用 long long。

所需模板：MergeSplitTree（第 68 页，7.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5494

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplitTree seg(a);
    for (int i = 0; i < m; i++)
    {
        int op, p;
        cin >> op >> p;
        p--;
        if (op == 0 || op == 3)
        {
            int x, y;
            cin >> x >> y;
            if (op == 0)
                seg.split(p, x - 1, y);
            else
                cout << seg.sum(p, x - 1, y) << '\n';
        }
        else if (op == 1)
        {
            int t;
            cin >> t;
            seg.merge(p, t - 1);
        }
        else if (op == 2)
        {
            long long x;
            int q;
            cin >> x >> q;
            seg.add(p, q - 1, x);
        }
        else
        {
            long long k;
            cin >> k;
            int x = seg.kth(p, k);
            cout << (x == -1 ? -1 : x + 1) << '\n';
        }
    }
}
```

应用题补充：UVA1479 / 删边改权与连通块第 k 大

题意：多组无向图；D e 删边（每边至多一次），C x v 改点权，Q x k 查所在连通块第 k 大，非法 k 贡献 0；E 结束当前组，0 0 结束输入，输出查询均值六位小数。先读完操作保存每次修改前的旧值，倒序处理；删边变合并，改权还原旧值。所有初值/新值离散化，重复权值按频数累计。split(0,0,0) 连续创建空集合，编号对应顶点；合并根 u,v 时 t.merge(u,v) 与 d.merge(u,v) 顺序一致。k 的换算先转 long long，避免 INT_MIN 溢出。

所需模板：MergeSplitTree（第 68 页，7.2.1 节）、dsu（第 4 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/14/1479.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    struct Op
    {
        char op;
        int x, y;
    };
    int n, m, tc = 0;
    while (cin >> n >> m && n)
    {
        vector<int> a(n), values;
        for (auto &x : a) cin >> x;
        values = a;
        vector<pair<int, int>> edges(m);
        for (auto &[u, v] : edges)
        {
            cin >> u >> v;
            u--;
            v--;
        }
        vector<int> deleted(m);
        vector<Op> ops;
        char op;
        while (cin >> op && op != 'E')
        {
            int x, y = 0;
            cin >> x;
            x--;
            if (op == 'D') deleted[x] = 1;
            else cin >> y;
            if (op == 'C')
            {
                values.push_back(y);
                swap(a[x], y);
            }
            ops.push_back({op, x, y});
        }
```

继续处理操作（接上页同一 main）：

```
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        auto index = [&](int x)
        {
            return lower_bound(values.begin(), values.end(), x) - values.begin();
        };
        MergeSplitTree t(values.size());
        for (int u = 0; u < n; u++)
        {
            if (u) t.split(0, 0, 0);
            t.add(u, index(a[u]), 1);
        }
        dsu d(n);
        auto join = [&](int e)
        {
            auto [u, v] = edges[e];
            u = d.find(u);
            v = d.find(v);
            if (u == v) return;
            t.merge(u, v);
            d.merge(u, v);
        };
        for (int e = 0; e < m; e++)
            if (!deleted[e]) join(e);
        long long sum = 0;
        int count = 0;
        for (int i = (int)ops.size() - 1; i >= 0; i--)
        {
            auto [op, x, y] = ops[i];
            if (op == 'D') join(x);
            else if (op == 'C')
            {
                int r = d.find(x);
                t.add(r, index(a[x]), -1);
                a[x] = y;
                t.add(r, index(a[x]), 1);
            }
            else
            {
                int r = d.find(x);
                long long k = (long long)d.size(r) - y + 1;
                int p = t.kth(r, k);
                if (p != -1) sum += values[p];
                count++;
            }
        }
        cout << "Case " << ++tc << ": " << fixed << setprecision(6)
             << (long double)sum / count << '\n';
    }
}
```

接口用法演示：按重数升序/降序分裂 API

演示约定：读 n q，初始集合 0 为空，值域 [0,n)。1 s x delta 增减重数且结果非负；2 s k reverse 移走 k 个最小值（reverse=0）或最大值（1）并输出新集合号；3 s l r 输出 [l,r) 重数和；4 s other 把不同的 other 合并到 s，other 清空可复用；5 s k 输出 1-based 第 k 小的值，无结果-1。k 可等于 0 或集合总重数，重复值可分到两边。自定义 API 协议，非 OJ 判定。

所需模板：MergeSplitTree（第 68 页，7.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    MergeSplitTree tree(n);
    while (q--)
    {
        int op, s;
        cin >> op >> s;
        if (op == 1)
        {
            int x;
            long long delta;
            cin >> x >> delta;
            tree.add(s, x, delta);
        }
        else if (op == 2)
        {
            long long k;
            bool reverse;
            cin >> k >> reverse;
            cout << tree.split_rank(s, k, reverse) << '\n';
        }
        else if (op == 3)
        {
            int l, r;
            cin >> l >> r;
            cout << tree.sum(s, l, r) << '\n';
        }
        else if (op == 4)
        {
            int other;
            cin >> other;
            tree.merge(s, other);
        }
        else
        {
            long long k;
            cin >> k;
            cout << tree.kth(s, k) << '\n';
        }
    }
}
```

7.3 李超线段树

7.3.1 精确线段李超树：最大值编号

构造传入非空的所有查询横坐标，排序去重后数量 M。add(x1,y1,x2,y2) 插入闭线段并返回从 1 开始的编号；可交换端点，竖直线段在该横坐标取较高端点。query(x) 返回交点最高的线段编号，同值取较早插入者；无交点返回 0，x 必须在预登记坐标内。value(id,x) 返回未约分的 (分子, 正分母)，要求 id>0 且 x 在线段内；不直接相除即可保留精度。横坐标绝对值 ≤10^9，端点纵坐标绝对值 ≤2×10^18，int128 交叉乘积在这些界内安全；不使用浮点或 epsilon。最小值问题可先把纵坐标取负，得到编号后再还原。整条直线用原 LiChao 更短；这里插入仅作用于端点范围。构造 O(M log M)，插入 O($\log^2$(M+1))，查询 O(log(M+1))，空间 O(M+ 线段数)；要求 M<INT_MAX/4，累计插入线段数 <INT_MAX。

```
struct SegmentLiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll x1, y1, x2, y2;
    };

    vector<ll> xs;
    vector<Line> lines{{0, 0, 0, 0}};
    vector<int> t;

    SegmentLiChao(vector<ll> x) : xs(move(x))
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty() && xs.front() >= -1000000000 && xs.back() <= 1000000000);
        t.resize(4 * xs.size());
    }

    pair<I, I> value(int id, ll x) const
    {
        auto [a, b, c, d] = lines[id];
        if (a == c) return {max(b, d), 1};
        return {I(b) * (c - x) + I(d) * (x - a), c - a};
    }
```

精确比较与线段插入（接上页同一结构体）：

```
    bool better(int u, int v, ll x) const
    {
        if (!u || !v) return u != 0;
        auto [a, b] = value(u, x);
        auto [c, d] = value(v, x);
        return a * d == c * b ? u < v : a * d > c * b;
    }

    void insert(int p, int l, int r, int ql, int qr, int id)
    {
        int m = (l + r) / 2;
        if (ql <= l && r <= qr)
        {
            if (better(id, t[p], xs[m])) swap(id, t[p]);
            if (!id || l == r) return;
            if (better(id, t[p], xs[l]))
                insert(p * 2, l, m, l, m, id);
            else if (better(id, t[p], xs[r]))
                insert(p * 2 + 1, m + 1, r, m + 1, r, id);
            return;
        }
        if (ql <= m) insert(p * 2, l, m, ql, qr, id);
        if (qr > m) insert(p * 2 + 1, m + 1, r, ql, qr, id);
    }

    int add(ll x1, ll y1, ll x2, ll y2)
    {
        assert(-1000000000 <= x1 && x1 <= 1000000000);
        assert(-1000000000 <= x2 && x2 <= 1000000000);
        assert(-2000000000000000000LL <= y1 && y1 <= 2000000000000000000LL);
        assert(-2000000000000000000LL <= y2 && y2 <= 2000000000000000000LL);
        if (x1 > x2)
        {
            swap(x1, x2);
            swap(y1, y2);
        }
        int id = lines.size();
        lines.push_back({x1, y1, x2, y2});
        int l = lower_bound(xs.begin(), xs.end(), x1) - xs.begin();
        int r = upper_bound(xs.begin(), xs.end(), x2) - xs.begin() - 1;
        if (l <= r) insert(1, 0, (int)xs.size() - 1, l, r, id);
        return id;
    }
```

查询最优线段编号（接上页同一结构体）：

```
    int query(ll x) const
    {
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(i < (int)xs.size() && xs[i] == x);
        int p = 1, l = 0, r = (int)xs.size() - 1, ans = 0;
        while (true)
        {
            if (better(t[p], ans, x)) ans = t[p];
            if (l == r) return ans;
            int m = (l + r) / 2;
            if (i <= m)
            {
                p *= 2;
                r = m;
            }
            else
            {
                p = p * 2 + 1;
                l = m + 1;
            }
        }
    }
};
```

模板题使用：Library Checker segment_add_get_min

题意：在线段有效区间内查询最小值。预读所有查询横坐标以离散化，再按原序处理；没有查询时直接退出。题目横坐标都是整数，因此半开区间 [l,r) 换成闭区间 [l,r-1]。端点值取负，把最小值转为最大值；无结果输出 INFINITY，返回分数在本题必为整数。原线性式的端点计算在题目范围内可用 long long。

所需模板：SegmentLiChao（第 79 页，7.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/segment_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long l, r, a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        if (o.type == 0)
            cin >> o.l >> o.r >> o.a >> o.b;
        else
        {
            cin >> o.l;
            xs.push_back(o.l);
        }
    }
```

继续处理操作（接上页同一 main）：

```
    if (xs.empty()) return 0;
    SegmentLiChao t(xs);
    for (auto o : ops)
        if (o.type == 0)
            t.add(o.l, -o.a * o.l - o.b, o.r - 1, -o.a * (o.r - 1) - o.b);
        else
        {
            int id = t.query(o.l);
            if (!id)
                cout << "INFINITY\n";
            else
            {
                auto [num, den] = t.value(id, o.l);
                cout << (long long)(-num / den) << '\n';
            }
        }
}
```

应用题补充：Luogu P4097 / HEOI2013 Segment

题意：强制在线插入整数端点线段，查询最高交点的最早线段编号。预登记全部 1..39989 横坐标；每个操作用上次查询答案解码，插入不会修改 last。竖直线段取较高端点，无交点输出并保存 0。此题源自 HEOI，作为应用补充单列。

所需模板：SegmentLiChao（第 79 页，7.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4097

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    vector<long long> xs(39989);
    iota(xs.begin(), xs.end(), 1);
    SegmentLiChao t(xs);
    int q, last = 0;
    cin >> q;
    while (q--)
    {
        int op, x1, x2;
        long long y1, y2;
        cin >> op >> x1;
        x1 = (x1 + last - 1) % 39989 + 1;
        if (op == 0)
        {
            last = t.query(x1);
            cout << last << '\n';
        }
        else
        {
            cin >> y1 >> x2 >> y2;
            x2 = (x2 + last - 1) % 39989 + 1;
            y1 = (y1 + last - 1) % 1000000000 + 1;
            y2 = (y2 + last - 1) % 1000000000 + 1;
            t.add(x1, y1, x2, y2);
        }
    }
}
```

7.3.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标，add({k,b,id}) 插入直线，query(x) 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 int128，k、b、x 为有符号 64 位且中间值可表示。单次 O(log n)。

```
struct LiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll k = 0, b = 0;
        int id = -1;

        I value(ll x) const { return I(k) * x + b; }
    };

    struct Node
    {
        Line line;
        int left = -1, right = -1;
    };

    vector<ll> xs;
    vector<Node> a{Node{}};

    LiChao(vector<ll> x) : xs(x)
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty());
    }

    bool better(Line u, Line v, ll x) const
    {
        return v.id == -1 ||
               (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
    }

    void add(Line line)
    {
        assert(line.id >= 0);
        add(0, 0, (int)xs.size() - 1, line);
    }
```

插入与查询（接上页同一结构体）：

```
    void add(int p, int l, int r, Line line)
    {
        int m = (l + r) / 2;
        if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
        if (l == r || line.id == -1) return;
        bool left = better(line, a[p].line, xs[l]);
        bool right = better(line, a[p].line, xs[r]);
        if (!left && !right) return;
        int next = left ? a[p].left : a[p].right;
        if (next == -1)
        {
            next = (int)a.size();
            a.push_back(Node{});
            if (left)
                a[p].left = next;
            else
                a[p].right = next;
        }
        if (left)
            add(next, l, m, line);
        else
            add(next, m + 1, r, line);
    }

    pair<I, int> query(ll x) const
    {
        int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(idx < (int)xs.size() && xs[idx] == x);
        int p = 0, l = 0, r = (int)xs.size() - 1;
        Line best;
        while (p != -1)
        {
            if (better(a[p].line, best, x)) best = a[p].line;
            if (l == r) break;
            int m = (l + r) / 2;
            if (idx <= m)
            {
                p = a[p].left;
                r = m;
            }
            else
            {
                p = a[p].right;
                l = m + 1;
            }
        }
        assert(best.id != -1);
        return {best.value(x), best.id};
    }
};
```

模板题使用：Library Checker line_add_get_min

题意：插入整条直线并查询最小值。先收集查询横坐标，再按输入顺序处理；初始至少一条线，查询必有结果。编号由调用者递增指定，本题只输出值。题目系数与横坐标范围保证最终值可转为 long long；模板内部用 int128 比较。

所需模板：LiChao（第 85 页，7.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/line_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        cin >> o.a;
        if (o.type == 0)
            cin >> o.b;
        else
            xs.push_back(o.a);
    }
    if (xs.empty()) return 0;
    LiChao t(xs);
    int id = 0;
    for (auto o : ops)
        if (o.type == 0)
            t.add({o.a, o.b, id++});
        else
            cout << (long long)t.query(o.a).first << '\n';
}
```

7.4 区间最值操作 & 区间历史最值

7.4.1 Segment Tree Beats：区间截断、加法与和

SegmentBeats(a) 接受 0-based 数组，chmin/chmax/add(l,r,x)、sum(l,r) 均用半开区间。空数组/空区间允许，空和为 0，重新构造用于多测。维护最大/严格第二大/次数及最小对称项；阈值等于第二极值必须递归，先下传加法再按父极值截断。n≤(INT_MAX-4)/4 且内存可用；真实值、阈值、有限延迟极值、加法标记及先加后截断的中间值须严格在 ±2^60 内，返回和须可用 long long 表示。内部和与差值乘次数用 int128，避免尚未同步的孩子临时总和溢出；不能由最终值有界推断中间和安全。建树/空间 O(n)，sum/add 为 O(log n)，单次 chmin/chmax 最坏 O(n)，混合操作总摊还 $O((n+q)\log^2(n+1))$。sum 会下传，不是 const。内部辅助函数与 t 不得直接调用/改写；不维护历史最值。

```
struct SegmentBeats
{
    using ll = long long;
    static constexpr ll inf = 1LL << 60;

    struct Node
    {
        __int128 sum = 0;
        ll mx = -inf, mx2 = -inf, mn = inf, mn2 = inf, tag = 0;
        int cmx = 0, cmn = 0;
    };

    int n;
    vector<Node> t;

    SegmentBeats(const vector<ll> &a) : n(a.size()), t(4 * a.size() + 4)
    {
        assert(a.size() <= (INT_MAX - 4) / 4);
        if (n) build(1, 0, n, a);
    }
```

极值合并与递归建树（接上页同一结构体）：

```
    void pull(int p)
    {
        auto &u = t[p];
        const auto &a = t[p * 2], &b = t[p * 2 + 1];
        u.sum = a.sum + b.sum;
        u.mx = max(a.mx, b.mx);
        u.mx2 = max(a.mx == u.mx ? a.mx2 : a.mx,
                    b.mx == u.mx ? b.mx2 : b.mx);
        u.cmx = (a.mx == u.mx ? a.cmx : 0) + (b.mx == u.mx ? b.cmx : 0);
        u.mn = min(a.mn, b.mn);
        u.mn2 = min(a.mn == u.mn ? a.mn2 : a.mn,
                    b.mn == u.mn ? b.mn2 : b.mn);
        u.cmn = (a.mn == u.mn ? a.cmn : 0) + (b.mn == u.mn ? b.cmn : 0);
        u.tag = 0;
    }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(-inf < a[l] && a[l] < inf);
            t[p].sum = t[p].mx = t[p].mn = a[l];
            t[p].cmx = t[p].cmn = 1;
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

整段加法和单极值组截断（接上页同一结构体）：

```
    void apply_add(int p, int len, ll x)
    {
        auto &u = t[p];
        assert(-(__int128)inf < (__int128)u.mn + x &&
               (__int128)u.mx + x < inf);
        assert(-(__int128)inf < (__int128)u.tag + x &&
               (__int128)u.tag + x < inf);
        u.sum += (__int128)x * len;
        u.mx += x;
        u.mn += x;
        if (u.mx2 != -inf) u.mx2 += x;
        if (u.mn2 != inf) u.mn2 += x;
        u.tag += x;
    }

    // Only the maximum changes; equality with mx2 must descend instead.
    void apply_min(int p, ll x)
    {
        auto &u = t[p];
        if (u.mx <= x) return;
        assert(u.mx2 < x);
        u.sum += ((__int128)x - u.mx) * u.cmx;
        if (u.mn == u.mx) u.mn = x;
        else if (u.mn2 == u.mx) u.mn2 = x;
        u.mx = x;
    }

    void apply_max(int p, ll x)
    {
        auto &u = t[p];
        if (u.mn >= x) return;
        assert(x < u.mn2);
        u.sum += ((__int128)x - u.mn) * u.cmn;
        if (u.mx == u.mn) u.mx = x;
        else if (u.mx2 == u.mn) u.mx2 = x;
        u.mn = x;
    }
```

标记下传与有条件递归更新（接上页同一结构体）：

```
    void push(int p, int l, int r)
    {
        int m = (l + r) / 2;
        if (t[p].tag)
        {
            apply_add(p * 2, m - l, t[p].tag);
            apply_add(p * 2 + 1, r - m, t[p].tag);
            t[p].tag = 0;
        }
        for (int c : {p * 2, p * 2 + 1})
        {
            apply_min(c, t[p].mx);
            apply_max(c, t[p].mn);
        }
    }

    // type: 0 = chmin, 1 = chmax, 2 = add.
    void update(int p, int l, int r, int ql, int qr, ll x, int type)
    {
        if (qr <= l || r <= ql) return;
        if (type == 0 && t[p].mx <= x) return;
        if (type == 1 && t[p].mn >= x) return;
        if (ql <= l && r <= qr)
        {
            if (type == 0 && t[p].mx2 < x)
            {
                apply_min(p, x);
                return;
            }
            if (type == 1 && x < t[p].mn2)
            {
                apply_max(p, x);
                return;
            }
            if (type == 2)
            {
                apply_add(p, r - l, x);
                return;
            }
        }
        push(p, l, r);
        int m = (l + r) / 2;
        update(p * 2, l, m, ql, qr, x, type);
        update(p * 2 + 1, m, r, ql, qr, x, type);
        pull(p);
    }
```

公开修改与区间和查询（接上页同一结构体）：

```
    void chmin(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 0);
    }

    void chmax(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 1);
    }

    void add(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 2);
    }

    __int128 query(int p, int l, int r, int ql, int qr)
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        return query(p * 2, l, m, ql, qr) + query(p * 2 + 1, m, r, ql, qr);
    }

    ll sum(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        __int128 ans = l < r ? query(1, 0, n, l, r) : 0;
        assert(LLONG_MIN <= ans && ans <= LLONG_MAX);
        return (ll)ans;
    }
};
```

模板题使用：Library Checker Range Chmin Chmax Add Range Sum

题意：维护数组，操作 0/1/2/3 分别为区间 chmin、chmax、加法、求和。全部使用 0-based 非空半开区间。N,Q≤200000；所有时刻元素绝对值 ≤10^12，修改参数绝对值 ≤2×10^12。因此延迟值满足本模板的范围要求，公开区间和也可用 long long 表示。输入区间不需要加减 1，按操作号调用对应方法。

所需模板：SegmentBeats（第 88 页，7.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_chmin_chmax_add_range_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    SegmentBeats t(a);
    while (q--)
    {
        int type, l, r;
        cin >> type >> l >> r;
        if (type == 3)
        {
            cout << t.sum(l, r) << '\n';
            continue;
        }
        long long x;
        cin >> x;
        if (type == 0) t.chmin(l, r, x);
        else if (type == 1) t.chmax(l, r, x);
        else t.add(l, r, x);
    }
}
// END USAGE
```

第八章　划分树

8.1　划分树：静态区间第 k 小

DivisionTree(v)，v 为 0-based vector<long long>，完整 signed64 位、可重复；0≤n<INT_MAX。kth(l,r,k) 返回半开区间 [l,r) 内 0-based 第 k 小，要求 0≤l<r≤n、0≤k<r-l；第 k 大改传 r-l-1-k。允许构造空数组但不能查询；输入不改变，建后无修改接口。按中位数稳定划分，重复中位值只给左边所需配额。构造和空间 O(n log(n+1))，单次 O(log(n+1))；build/query 为递归内部函数，深度 O(log(n+1))。sorted/a/pre 只读。与压位小波矩阵同功能但存储更多、代码更短；原书第 k 大注释与实现不一致，本接口明确为第 k 小。

```
struct DivisionTree
{
    int n;
    vector<long long> sorted;
    vector<vector<long long>> a;
    vector<vector<int>> pre;

    DivisionTree(const vector<long long> &v) : n(v.size()), sorted(v)
    {
        sort(sorted.begin(), sorted.end());
        int h = bit_width((unsigned)max(1, n) - 1) + 1;
        a.assign(h, vector<long long>(n));
        pre.assign(h, vector<int>(n + 1));
        a[0] = v;
        if (n) build(0, n, 0);
    }
```

稳定划分与区间查询（接上页同一结构体）：

```
    void build(int l, int r, int d)
    {
        if (r - l == 1) return;
        int m = l + (r - l) / 2, same = m - l;
        for (int i = l; i < r; i++) same -= a[d][i] < sorted[m - 1];
        int x = l, y = m;
        for (int i = l; i < r; i++)
        {
            bool left = a[d][i] < sorted[m - 1];
            if (a[d][i] == sorted[m - 1] && same > 0)
            {
                left = true;
                same--;
            }
            if (left)
                a[d + 1][x++] = a[d][i];
            else
                a[d + 1][y++] = a[d][i];
            pre[d][i + 1] = pre[d][i] + left;
        }
        build(l, m, d + 1);
        build(m, r, d + 1);
    }

    long long query(int L, int R, int l, int r, int d, int k) const
    {
        if (r - l == 1) return a[d][l];
        int m = L + (R - L) / 2;
        int before = pre[d][l] - pre[d][L], cnt = pre[d][r] - pre[d][l];
        if (k < cnt) return query(L, m, L + before, L + before + cnt, d + 1, k);
        int x = m + (l - L - before);
        return query(m, R, x, x + (r - l - cnt), d + 1, k - cnt);
    }

    long long kth(int l, int r, int k) const
    {
        assert(0 <= l && l < r && r <= n && 0 <= k && k < r - l);
        return query(0, n, l, r, 0, k);
    }
};
```

模板题使用：Luogu P3834

题意：静态区间第 k 小；n、q 至 20 万，非负值至 10^9。输入为 1-based 闭区间与排名，调用 kth(l-1,r,k-1)。本程序使用中位数划分树，重复数仍按出现次数排名。

所需模板：DivisionTree（第 94 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    DivisionTree tree(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << tree.kth(l - 1, r, k - 1) << '\n';
    }
}
```

模板题使用：Library Checker range_kth_smallest

题意：半开区间 [l,r) 内第 k 小，位置和 k 均为 0-based，直接调用 kth；n、q 至 20 万，值在 $0..10^9$。

所需模板：DivisionTree（第 94 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_kth_smallest

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    DivisionTree tree(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << tree.kth(l, r, k) << '\n';
    }
}
```

相关应用：小波矩阵使用逐位稳定划分和压位 rank，解决静态区间顺序统计；在固定导航的划分树下按相关结构收录，不等同于原文划分树实现。

8.2 压位小波矩阵：静态区间顺序统计

WaveletMatrix(a)，a 为 0-based vector<long long>，支持完整有符号 64 位和重复值，建后不可修改；n 小于 INT_MAX。接口都用半开区间 [l,r)。kth(l,r,k) 取 0-based 第 k 小，要求 0≤k<r-l；第 k 大改传 r-l-1-k。less(l,r,x) 统计严格小于 x，freq 统计等于 x；两计数允许空数组和空区间，kth 不允许。构造 O(n log(n+1))，查询 O(log(s+1))，s 为不同值数。常驻空间 O(n+ceil(n/64)log(s+1)) 机器字，构造另用 O(n) 空间。rank 为内部方法，成员只读。字边界、区间值域计数及证明见本册补充说明。

```
struct WaveletMatrix
{
    int n, h;
    vector<long long> val;
    vector<int> mid;
    vector<vector<unsigned long long>> bit;
    vector<vector<int>> sum;

    WaveletMatrix(const vector<long long> &a) : n(a.size()), val(a)
    {
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        h = bit_width((unsigned)max(1, (int)val.size()) - 1);
        mid.resize(h);
        bit.assign(h, vector<unsigned long long>(n / 64 + 1));
        sum.assign(h, vector<int>(n / 64 + 1));
        vector<int> v(n), w(n);
        for (int i = 0; i < n; i++)
            v[i] = lower_bound(val.begin(), val.end(), a[i]) - val.begin();
```

逐层稳定划分（接上页构造函数）、rank 与第 k 小（接上页同一结构体）：

```
        for (int d = h - 1; d >= 0; d--)
        {
            for (int i = 0; i < n; i++)
            {
                if (v[i] >> d & 1)
                    bit[d][i / 64] |= 1ULL << (i % 64);
                else
                    mid[d]++;
            }
            for (int i = 1; i < (int)sum[d].size(); i++)
                sum[d][i] = sum[d][i - 1] + popcount(bit[d][i - 1]);
            int l = 0, r = mid[d];
            for (int x : v)
            {
                if (x >> d & 1)
                    w[r++] = x;
                else
                    w[l++] = x;
            }
            v.swap(w);
        }
    }

    // Internal: number of one bits before position i at level d.
    int rank(int d, int i) const
    {
        return sum[d][i / 64] + popcount(bit[d][i / 64] & ((1ULL << (i % 64)) - 1));
    }

    // [l, r), k is zero-based; requires 0 <= k < r-l.
    long long kth(int l, int r, int k) const
    {
        assert(0 <= l && l <= r && r <= n && 0 <= k && k < r - l);
        int x = 0;
        for (int d = h - 1; d >= 0; d--)
        {
            int a = rank(d, l), b = rank(d, r);
            int z = r - l - b + a;
            if (k < z)
            {
                l -= a;
                r -= b;
            }
            else
            {
                k -= z;
                x |= 1 << d;
                l = mid[d] + a;
                r = mid[d] + b;
            }
        }
        return val[x];
    }
```

严格小于计数与频次（接上页同一结构体）：

```
    // Count values < x; empty intervals and absent values are allowed.
    int less(int l, int r, long long x) const
    {
        assert(0 <= l && l <= r && r <= n);
        int q = lower_bound(val.begin(), val.end(), x) - val.begin();
        if (q == (int)val.size()) return r - l;
        int ans = 0;
        for (int d = h - 1; d >= 0; d--)
        {
            int a = rank(d, l), b = rank(d, r);
            if (q >> d & 1)
            {
                ans += r - l - b + a;
                l = mid[d] + a;
                r = mid[d] + b;
            }
            else
            {
                l -= a;
                r -= b;
            }
        }
        return ans;
    }

    int freq(int l, int r, long long x) const
    {
        assert(0 <= l && l <= r && r <= n);
        int q = lower_bound(val.begin(), val.end(), x) - val.begin();
        if (q == (int)val.size() || val[q] != x) return 0;
        for (int d = h - 1; d >= 0; d--)
        {
            int a = rank(d, l), b = rank(d, r);
            if (q >> d & 1)
            {
                l = mid[d] + a;
                r = mid[d] + b;
            }
            else
            {
                l -= a;
                r -= b;
            }
        }
        return r - l;
    }
};
```

模板题使用：Library Checker range_kth_smallest

题意：静态数组，查询半开区间 [l,r) 内第 k 小，l/r/k 均按题面 0-based；直接调用 kth。n、q 至 20 万，值在 $0..10^9$。

所需模板：WaveletMatrix（第 97 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_kth_smallest

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    WaveletMatrix wm(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << wm.kth(l, r, k) << '\n';
    }
}
```

模板题使用：Library Checker static_range_frequency

题意：静态数组，统计半开区间 [l,r) 中 x 的出现次数；允许空数组、空查询区间、q=0 和未出现的 x。n、q 至 50 万，值在 $0..10^9$。

所需模板：WaveletMatrix（第 97 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/static_range_frequency

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    WaveletMatrix wm(a);
    while (q--)
    {
        int l, r;
        long long x;
        cin >> l >> r >> x;
        cout << wm.freq(l, r, x) << '\n';
    }
}
```

模板题使用：Luogu P3834

题意：静态区间第 k 小；题目为 1-based 闭区间和 1-based k，调用时转换成 kth(l-1,r,k-1)。n、q 至 20 万，题面值在 $0..10^9$；本程序采用小波矩阵，不建立历史版本。

所需模板：WaveletMatrix（第 97 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    WaveletMatrix wm(a);
    while (q--)
    {
        int l, r, k;
        cin >> l >> r >> k;
        cout << wm.kth(l - 1, r, k - 1) << '\n';
    }
}
```

第九章　二叉搜索树 & 平衡树

9.1　二叉搜索树 & 平衡树

9.1.1　GNU rope 与版本共享

实际类型为 ___gnu_cxx::rope<T>，不是 std::rope；它是 GNU 扩展序列，包含 ext/rope。本工具链随后重新包含 cassert，保留断言接口。rp<int>、rp<char> 表示序列，默认空。size()、empty()、push_back(x)、insert(p,x)、erase(p,len)、replace(p,x)、substr(p,len) 直接沿用 rope；位置为 0-based，len 是长度而非右端点。约定 0≤p≤size，删除/截取要求 len≤size-p，读写单点要求 p<size。r[i] 读取第 i 项；substr 返回新序列。rp<T> b=a 共享底层树，随后通过容器接口修改 b 不影响 a，可用 vector<rp<int» 保存分支版本。复制构造共享根为 O(1)；随机访问沿树路径，正常平衡时 O(log n)。插入/删除/截取涉及路径、叶块复制和可能的再平衡，不能把每次操作都写成严格最坏 O(log n)；释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 O(n) 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值；函数生成型 rope 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本：Q 个零单元，复制根与 l/r，入队 replace(r++,x)，出队读取 [l++]；直接 push_back/erase 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB，具体失败与通过范围见 docs/ROPE-NOTES.md。适合需要序列中间插删、截取和少量分支快照的场景；只按下标读写通常使用 vector 更直接。需要区间和、翻转等懒标记聚合时使用隐式 Treap，rope 本身不提供这些接口。下方示例通过修改 b 演示 a 保持原版本。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 1 页。

```
template <class T> using rp = __gnu_cxx::rope<T>;
```

```
rp<int> a;
a.push_back(3);
a.insert(0, 7);
rp<int> b = a;
b.replace(1, 9);
b.erase(0, 1);
auto c = a.substr(0, 1);
int x = a[1];
```

模板题使用：Library Checker persistent_queue

题意：维护可持久化队列，每次从指定版本入队或出队并建立新版本。题目的 -1 号空版本映射为下标 0；rope 副本和头尾指针一起复制，replace 写入不会改变历史版本，出队只推进该版本队头。

所需模板：rp（第 102 页，9.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<rp<int>> s(q + 1);
    vector<int> l(q + 1), r(q + 1);
    s[0] = rp<int>(q, 0);
    for (int i = 1; i <= q; i++)
    {
        int op, t, x;
        cin >> op >> t;
        ++t;
        s[i] = s[t];
        l[i] = l[t];
        r[i] = r[t];
        if (op == 0)
        {
            cin >> x;
            s[i].replace(r[i]++, x);
        }
        else
        {
            cout << s[i][l[i]++] << '\n';
        }
    }
}
```

9.1.2 PBDS 有序树与顺序统计

GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp 和 ext/pb_ds/tree_policy.hpp。ost<T> 为去重有序集合；insert(x)、erase(x)、find(x)、lower_bound(x)、upper_bound(x) 沿用集合接口。order_of_key(x) 为严格小于 x 的元素数，find_by_order(k) 返回 0-based 第 k 个的迭代器，越界返回 end。默认 less<T>；改用其他严格弱序比较器时，排名和边界均按该比较器解释。单次 O(log n)，空间 O(n)，假设比较为 O(1)。重复值使用 ost<pair<T,int»，插入 {x,id} 且 id 全局唯一、1≤id<INT_MAX。order_of_key({x,0}) 统计小于 x，order_of_key({x,INT_MAX}) 统计不大于 x；find_by_order(k)->first 取值，须先检查 end。删除一个 x：it=lower_bound({x,0})，检查 it!=end 且 it->first==x 后 erase(it)。不可用 less_equal 模拟多重集合。前驱先检查迭代器是否为 begin，再递减；空集合和无解不得解引用 end。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 1 页。

序列中间插删、截取与版本共享的 GNU rope 见第 102 页。

```
template <class T, class C = less<T>>
using ost = __gnu_pbds::tree<T,
                             __gnu_pbds::null_type,
                             C,
                             __gnu_pbds::rb_tree_tag,
                             __gnu_pbds::tree_order_statistics_node_update>;
```

模板题使用：Library Checker ordered_set

题意：维护无重复整数集合，支持第 k 小、计数及含等号的前驱/后继。find_by_order 使用 0-based 排名；order_of_key 只计严格小于 x，故不大于 x 的数量需补上相等元素。越界查询按题意输出 -1。

所需模板：ost（第 104 页，9.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/ordered_set

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, x;
    cin >> n >> q;
    ost<int> s;
    while (n--)
    {
        cin >> x;
        s.insert(x);
    }
    while (q--)
    {
        int op;
        cin >> op >> x;
        if (op == 0) s.insert(x);
        if (op == 1) s.erase(x);
        if (op == 2)
        {
            auto it = s.find_by_order(x - 1);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
        if (op == 3) cout << s.order_of_key(x) + (s.find(x) != s.end()) << '\n';
        if (op == 4)
        {
            auto it = s.upper_bound(x);
            cout << (it == s.begin() ? -1 : *--it) << '\n';
        }
        if (op == 5)
        {
            auto it = s.lower_bound(x);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
    }
}
```

9.2 Treap

9.2.1 FHQ Treap：可重集合与顺序统计

insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在；rank(x) 为严格小于 x 的元素数加一，x 不必存在；kth(k) 要求 1≤k≤size()。prev/next 为严格前驱/后继，不存在时返回 nullopt。支持重复值、LLONG_MIN 和 LLONG_MAX，分裂不使用 x-1。随机优先级下单次期望 O(log n)，并非确定性最坏界。节点池不回收已删除节点，空间按累计插入次数计算；可在构造时指定随机种子。

```
struct OrderedTreap
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0;
        unsigned long long pri = 0;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }

    int node(ll x)
    {
        int p = a.size();
        a.push_back({0, 0, 1, x, rng()});
        return p;
    }

    // left: val < x, or val <= x when equal is true. No x-1 overflow.
    void split(int p, ll x, int &l, int &r, bool equal = false)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        if (a[p].val < x || (equal && a[p].val == x))
        {
            l = p;
            split(a[p].r, x, a[p].r, r, equal);
        }
        else
        {
            r = p;
            split(a[p].l, x, l, a[p].l, equal);
        }
        pull(p);
    }

```

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    void insert(ll x)
    {
        int p = node(x), l, r;
        split(root, x, l, r);
        root = merge(merge(l, p), r);
    }

    bool erase(ll x)
    {
        int l, mid, r;
        split(root, x, l, mid);
        split(mid, x, mid, r, true);
        bool found = mid != 0;
        if (mid) mid = merge(a[mid].l, a[mid].r);
        root = merge(l, merge(mid, r));
        return found;
    }

    int size() const { return a[root].siz; }

    int less(ll x, bool equal = false) const
    {
        int p = root, ans = 0;
        while (p)
        {
            if (a[p].val < x || (equal && a[p].val == x))
            {
                ans += a[a[p].l].siz + 1;
                p = a[p].r;
            }
            else
                p = a[p].l;
        }
        return ans;
    }

    int rank(ll x) const { return less(x) + 1; }

    ll kth(int k) const
    {
        assert(1 <= k && k <= size());
        int p = root;
```

```
        while (true)
        {
            int left = a[a[p].l].siz;
            if (k == left + 1) return a[p].val;
            if (k <= left)
                p = a[p].l;
            else
            {
                k -= left + 1;
                p = a[p].r;
            }
        }
    }

    optional<ll> prev(ll x) const
    {
        int k = less(x);
        if (!k) return nullopt;
        return kth(k);
    }

    optional<ll> next(ll x) const
    {
        int k = less(x, true);
        if (k == size()) return nullopt;
        return kth(k + 1);
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedTreap（第 106 页，9.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedTreap g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

9.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和

insert(k,value) 在前 k 个元素后插入，0≤k≤size()；区间接口 erase、reverse、flip_bits、add、query 均使用 1-based 非空闭区间。reverse 反转元素顺序；flip_bits 要求区间内全为 0/1，逐位取反并保持顺序，不检查这一前提。query 返回和，二进制区间即 1 的个数。values() 递归中序返回完整序列，O(n)。其余操作期望 O(log n)，不是最坏界。符号与加法标记表示 s*x+b，位翻转通过取负后加 1 实现，下传必须先符号再加法；顺序反转另行维护。所有值、和、标记、乘法及取负中间结果须在 long long 范围内。删除不回收节点，空间按累计插入次数计算；辅助函数和节点数组只供内部使用。

```
struct SequenceTreap
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0, sum = 0, tag = 0;
        unsigned long long pri = 0;
        bool rev = false, neg = false;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    void pull(int p)
    {
        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
    }

    void apply_add(int p, ll x)
    {
        if (!p) return;
        a[p].val += x;
        a[p].sum += x * a[p].siz;
        a[p].tag += x;
    }

    void apply_negate(int p)
    {
        if (!p) return;
        a[p].val = -a[p].val;
        a[p].sum = -a[p].sum;
        a[p].tag = -a[p].tag;
        a[p].neg = !a[p].neg;
    }
```

顺序反转、懒标记下传与分裂（接上页同一结构体）：

```
    void apply_reverse(int p)
    {
        if (!p) return;
        swap(a[p].l, a[p].r);
        a[p].rev = !a[p].rev;
    }

    void push(int p)
    {
        if (a[p].rev)
        {
            apply_reverse(a[p].l);
            apply_reverse(a[p].r);
            a[p].rev = false;
        }
        if (a[p].neg)
        {
            apply_negate(a[p].l);
            apply_negate(a[p].r);
            a[p].neg = false;
        }
        if (a[p].tag)
        {
            apply_add(a[p].l, a[p].tag);
            apply_add(a[p].r, a[p].tag);
            a[p].tag = 0;
        }
    }

    // Split into the first k elements and the rest.
    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        push(p);
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }
```

合并、插入、删除与顺序反转（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            push(l);
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        push(r);
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    // Insert after the first k elements, 0 <= k <= size().
    void insert(int k, ll value)
    {
        assert(0 <= k && k <= size());
        int p = a.size(), l, r;
        a.push_back({0, 0, 1, value, value, 0, rng(), false, false});
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    // All range operations use 1-based inclusive intervals.
    void erase(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        root = merge(x, z);
    }

    void reverse(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_reverse(y);
        root = merge(merge(x, y), z);
    }
```

位翻转、加法、查询与输出（接上页同一结构体）：

```
    // Requires every value in [l,r] to be 0 or 1; preserves their order.
    void flip_bits(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_negate(y);
        apply_add(y, 1);
        root = merge(merge(x, y), z);
    }

    void add(int l, int r, ll value)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_add(y, value);
        root = merge(merge(x, y), z);
    }

    ll query(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        ll ans = a[y].sum;
        root = merge(merge(x, y), z);
        return ans;
    }

    void collect(int p, vector<ll> &out)
    {
        if (!p) return;
        push(p);
        collect(a[p].l, out);
        out.push_back(a[p].val);
        collect(a[p].r, out);
    }

    vector<ll> values()
    {
        vector<ll> answer;
        collect(root, answer);
        return answer;
    }
};
```

模板题使用：Luogu P3391

题意：初始序列为 1..n，多次翻转闭区间 [l,r]，最后输出序列。插入参数为前面已有元素数，翻转区间为 1-based 闭区间。

所需模板：SequenceTreap（第 110 页，9.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3391

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SequenceTreap g;
    for (int i = 1; i <= n; i++) g.insert(i - 1, i);
    while (m--)
    {
        int l, r;
        cin >> l >> r;
        g.reverse(l, r);
    }
    for (long long x : g.values()) cout << x << ' ';
    cout << '\n';
}
```

应用题补充：二进制序列区间取反（本地接口示例）

题意：输入 n,q、长度 n 的 01 字符串，再输入 q 个 1-based 闭区间。每次逐位取反，最后输出字符串。保持位置顺序；要反转顺序应调用 reverse。按插入构造，保证随机优先级堆序；附件未提供完整牛客题号，仅登记本地应用。

所需模板：SequenceTreap（第 110 页，9.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    string s;
    cin >> n >> q >> s;
    SequenceTreap tree;
    for (int i = 0; i < n; i++) tree.insert(i, s[i] - '0');
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        tree.flip_bits(l, r);
    }
    for (auto bit : tree.values()) cout << bit;
    cout << '\n';
}
```

9.2.3 动态仿射序列：插删、翻转与区间和

代码依赖：ModInt（第 263 页，A.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

AffineSequenceTreap<mod>，默认模数 998244353，正 int 模数也可为 1 或合数，不做除法。insert(k,x) 在前 k 项后插入；erase、reverse、affine、query 均用 1-based 非空闭区间，affine(l,r,b,c) 令值变为 b*x+c，乘数 0 合法。query 返回 ModInt，values 递归导出序列；二者会改变内部标记或树形，不保证并发读。新标记复合为 (b*mul,b*add+c)，不能颠倒；顺序反转与统一仿射可交换。沿用递归 split/merge，逐项构造期望 O(N log(N+1))，操作期望 O(log(n+1))、最坏 O(n)，values 为 O(n)。固定种子可复现但无对抗最坏保证。删除不回收，空间按累计插入计；节点索引、大小须在 int 内。可 reserve 但不影响正确性；辅助函数及节点数组仅供内部使用。

```
template <int mod = 998244353> struct AffineSequenceTreap
{
    static_assert(mod > 0);
    using Z = ModInt<mod>;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        Z val = 0, sum = 0, mul = 1, add = 0;
        unsigned long long pri = 0;
        bool rev = false;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    AffineSequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    void pull(int p)
    {
        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
    }

    void apply(int p, Z b, Z c)
    {
        if (!p) return;
        a[p].val = b * a[p].val + c;
        a[p].sum = b * a[p].sum + c * Z(a[p].siz);
        a[p].mul = b * a[p].mul;
        a[p].add = b * a[p].add + c;
    }
```

反转、仿射下传与递归分裂（接上页同一结构体）：

```
    void flip(int p)
    {
        if (!p) return;
        swap(a[p].l, a[p].r);
        a[p].rev = !a[p].rev;
    }

    void push(int p)
    {
        if (a[p].rev)
        {
            flip(a[p].l);
            flip(a[p].r);
            a[p].rev = false;
        }
        if (a[p].mul.v != Z(1).v || a[p].add.v)
        {
            apply(a[p].l, a[p].mul, a[p].add);
            apply(a[p].r, a[p].mul, a[p].add);
            a[p].mul = 1;
            a[p].add = 0;
        }
    }

    // Split into the first k elements and the rest.
    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        push(p);
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }
```

合并、插入与删除（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            push(l);
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        push(r);
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    // Insert after k elements; deleted nodes are not recycled.
    void insert(int k, Z value)
    {
        assert(0 <= k && k <= size());
        int p = a.size(), l, r;
        a.push_back({0, 0, 1, value, value, 1, 0, rng(), false});
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    // All range operations use nonempty 1-based inclusive intervals.
    void cut(int l, int r, int &x, int &y, int &z)
    {
        assert(1 <= l && l <= r && r <= size());
        split(root, r, y, z);
        split(y, l - 1, x, y);
    }

    void erase(int l, int r)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        root = merge(x, z);
    }
```

区间操作与递归输出（接上页同一结构体）：

```
    void reverse(int l, int r)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        flip(y);
        root = merge(merge(x, y), z);
    }

    void affine(int l, int r, Z b, Z c)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        apply(y, b, c);
        root = merge(merge(x, y), z);
    }

    Z query(int l, int r)
    {
        int x, y, z;
        cut(l, r, x, y, z);
        Z ans = a[y].sum;
        root = merge(merge(x, y), z);
        return ans;
    }

    void collect(int p, vector<Z> &out)
    {
        if (!p) return;
        push(p);
        collect(a[p].l, out);
        out.push_back(a[p].val);
        collect(a[p].r, out);
    }

    vector<Z> values()
    {
        vector<Z> out;
        collect(root, out);
        return out;
    }
};
```

模板题使用：Library Checker Dynamic Sequence Range Affine Range Sum

题意：维护模 998244353 的动态序列，支持按位置插入、删除单项、顺序翻转、仿射修改和区间和。初始长度与操作数各至多 500000，乘数可以为 0。题目区间为 0-based 半开，转成模板的闭区间 [l+1,r]；插入位置不偏移，删除位置加 1。预留 N+Q+1 个节点，删除不回收。

所需模板：ModInt（第 263 页，A.1.2 节）、AffineSequenceTreap（第 115 页，9.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dynamic_sequence_range_affine_range_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    AffineSequenceTreap<> t;
    t.a.reserve(n + q + 1);
    for (int i = 0, x; i < n; i++)
    {
        cin >> x;
        t.insert(i, x);
    }
    while (q--)
    {
        int op, l, r, b, c;
        cin >> op >> l;
        if (op == 0)
        {
            cin >> b;
            t.insert(l, b);
        }
        else if (op == 1)
            t.erase(l + 1, l + 1);
        else
        {
            cin >> r;
            if (op == 2)
                t.reverse(l + 1, r);
            else if (op == 3)
            {
                cin >> b >> c;
                t.affine(l + 1, r, b, c);
            }
            else
                cout << t.query(l + 1, r).v << '\n';
        }
    }
}
// END USAGE
```

9.2.4 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树 O(n) 次 GCD，随机优先级下单次期望 O(log n) 次 GCD；删除另需 O(删除长度) 回收。一次 GCD 为 O(log U)。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```
struct GcdSequenceTreap
{
    using U = unsigned long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0, tag = 0;
        int cnt[2]{};
        U val = 0, g[2]{}, pri = 0;
    };

    vector<Node> a{Node{}};
    vector<int> recycled;
    mt19937_64 rng;
    int root = 0;

    GcdSequenceTreap(U seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    int node(U value, int tag)
    {
        assert(tag == 0 || tag == 1);
        int p;
        if (recycled.empty())
        {
            p = a.size();
            a.push_back(Node{});
        }
        else
        {
            p = recycled.back();
            recycled.pop_back();
            a[p] = Node{};
        }
        a[p].siz = 1;
        a[p].val = value;
        a[p].tag = tag;
        a[p].cnt[tag] = 1;
        a[p].g[tag] = value;
        a[p].pri = rng();
        return p;
    }
```

聚合与递归分裂合并（接上页同一结构体）：

```
    void pull(int p)
    {
        int l = a[p].l, r = a[p].r;
        a[p].siz = a[l].siz + a[r].siz + 1;
        for (int t = 0; t < 2; t++)
        {
            a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
            a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
            if (a[p].tag == t) a[p].g[t] = gcd(a[p].g[t], a[p].val);
        }
    }

    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = 0;
            r = 0;
            return;
        }
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }

    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }
```

构建、重算与节点回收（接上页同一结构体）：

```
    void rebuild(int p)
    {
        if (!p) return;
        rebuild(a[p].l);
        rebuild(a[p].r);
        pull(p);
    }

    void release(int p)
    {
        if (!p) return;
        release(a[p].l);
        release(a[p].r);
        recycled.push_back(p);
    }

    // O(n) GCD calls; replaces the current sequence.
    void build(const vector<pair<U, int>> &v)
    {
        a.assign(1, Node{});
        recycled.clear();
        root = 0;
        vector<int> st;
        for (auto [value, tag] : v)
        {
            int p = node(value, tag), last = 0;
            while (!st.empty() && a[st.back()].pri < a[p].pri)
            {
                last = st.back();
                st.pop_back();
            }
            a[p].l = last;
            if (!st.empty()) a[st.back()].r = p;
            st.push_back(p);
        }
        if (st.empty()) return;
        root = st.front();
        rebuild(root);
    }
```

插删、单点修改与分组查询（接上页同一结构体）：

```
    // Insert after the first k entries. Range operations below are 1-based.
    void insert(int k, U value, int tag)
    {
        assert(0 <= k && k <= size());
        int p = node(value, tag), l, r;
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    void erase(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        root = merge(x, z);
        release(y);
    }

    void set(int k, U value)
    {
        assert(1 <= k && k <= size());
        int x, y, z;
        split(root, k, y, z);
        split(y, k - 1, x, y);
        a[y].val = value;
        pull(y);
        root = merge(merge(x, y), z);
    }

    void toggle(int k)
    {
        assert(1 <= k && k <= size());
        int x, y, z;
        split(root, k, y, z);
        split(y, k - 1, x, y);
        a[y].tag ^= 1;
        pull(y);
        root = merge(merge(x, y), z);
    }

    optional<U> query(int l, int r, int tag)
    {
        assert(1 <= l && l <= r && r <= size());
        assert(tag == 0 || tag == 1);
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        optional<U> answer;
        if (a[y].cnt[tag]) answer = a[y].g[tag];
        root = merge(merge(x, y), z);
        return answer;
    }
};
```

接口用法演示：自定义带标签 GCD 序列操作

演示约定：自定义协议，非 OJ 题；n≤200000、q≤100000 均可为 0，是本演示约定。先读 n 组 (value,tag)，值为非负 unsigned long long，标签 0 或 1。I k v t 在前 k 项后插入；E l r 删 1-based 闭区间；S k v 改单值；T k 翻单标签；Q l r t 查询该标签 GCD，无该标签输出 NONE，全零输出 0。位置必须合法，可删空再插入，不支持翻转或负值。

所需模板：GcdSequenceTreap（第 120 页，9.2.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/src/compact/gcd_sequence.hpp

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<pair<GcdSequenceTreap::U, int>> a(n);
    for (auto &[v, t] : a) cin >> v >> t;
    GcdSequenceTreap tr;
    tr.build(a);
    while (q--)
    {
        char op;
        int k, l, r, t;
        GcdSequenceTreap::U v;
        cin >> op;
        if (op == 'I')
        {
            cin >> k >> v >> t;
            tr.insert(k, v, t);
        }
        else if (op == 'E')
        {
            cin >> l >> r;
            tr.erase(l, r);
        }
        else if (op == 'S')
        {
            cin >> k >> v;
            tr.set(k, v);
        }
        else if (op == 'T')
        {
            cin >> k;
            tr.toggle(k);
        }
        else if (op == 'Q')
        {
            cin >> l >> r >> t;
            auto ans = tr.query(l, r, t);
            if (ans) cout << *ans << '\n';
            else cout << "NONE\n";
        }
    }
}
```

相关应用：按位置分裂拼接与最大值剪枝，不代表整页覆盖

9.2.5 序列最大值 Treap

空构造，外部根变量 0 为空；同一对象可管理多棵互不相交的序列。insert(root,pos,value) 在 0-based 位置 pos 前插入，pos 可等于长度，通过引用更新根，返回按插入顺序递增的稳定节点 ID。split(root,k) 消耗旧根，返回前 k 项及其余项；merge(left,right) 消耗两根并按顺序拼接，无值域大小限制，但两根必须互不相交且属于同一对象。last_greater(root,value) 返回最右严格大于 value 的 0-based 位置，无结果为-1；后缀中 <=value 的个数等于 size(root)-1-last_greater(root,value)。ids(root) 按序返回稳定 ID。

插入、分裂、拼接和查询期望 O(log(N+1))，随机优先级下单次最坏 O(N)；输出 O(N)。空间 O(N)，节点不回收，递归最坏需 O(N) 栈。value 允许任意 signed64 及重复值，不作值加减。节点总数须小于 INT_MAX。消耗的根不可再作为独立树使用；root 引用必须为外部变量，不能引用 vector 内字段。t、rng 及辅助函数仅供内部实现。CF38G 使用互异正整数，重复值扩展由独立测试覆盖；当前无线上判定。

```
struct SequenceMaxTreap
{
    using ll = long long;
    struct Node
    {
        int left = 0, right = 0, size = 0;
        ll value = LLONG_MIN, maximum = LLONG_MIN;
        unsigned long long priority = 0;
    };
    vector<Node> t{Node{}};
    mt19937_64 rng;

    SequenceMaxTreap(unsigned long long seed = 712367821) : rng(seed)
    {
    }

    int size(int root) const
    {
        return t[root].size;
    }

    void pull(int p)
    {
        t[p].size = size(t[p].left) + size(t[p].right) + 1;
        t[p].maximum = max({t[p].value, t[t[p].left].maximum,
                            t[t[p].right].maximum});
    }

    pair<int, int> split(int p, int k)
    {
        assert(0 <= k && k <= size(p));
        if (!p)
            return {0, 0};
        if (k <= size(t[p].left))
        {
            auto [left, right] = split(t[p].left, k);
            t[p].left = right;
            pull(p);
            return {left, p};
        }
        auto [left, right] = split(t[p].right, k - size(t[p].left) - 1);
```

```
        t[p].right = left;
        pull(p);
        return {p, right};
    }

    int merge(int left, int right)
    {
        if (!left || !right)
            return left ? left : right;
        if (t[left].priority > t[right].priority)
        {
            t[left].right = merge(t[left].right, right);
            pull(left);
            return left;
        }
        t[right].left = merge(left, t[right].left);
        pull(right);
        return right;
    }

    int insert(int &root, int pos, ll value)
    {
        assert(0 <= pos && pos <= size(root) && t.size() < INT_MAX);
        int id = t.size();
        t.push_back({0, 0, 1, value, value, rng()});
        auto [left, right] = split(root, pos);
        root = merge(merge(left, id), right);
        return id;
    }

    int last_greater(int p, ll value) const
    {
        if (!p || t[p].maximum <= value)
            return -1;
        int right = last_greater(t[p].right, value);
        if (right != -1)
            return size(t[p].left) + 1 + right;
        if (t[p].value > value)
            return size(t[p].left);
        return last_greater(t[p].left, value);
    }

    void collect(int p, vector<int> &order) const
    {
        if (!p)
            return;
        collect(t[p].left, order);
        order.push_back(p);
```

```
        collect(t[p].right, order);
    }

    vector<int> ids(int root) const
    {
        vector<int> order;
        collect(root, order);
        return order;
    }
};
```

模板题使用：Codeforces 38G Queue

题意：n<=100000；重要度是 1..n 的排列，0<=c_i<=n。每人先入队尾，最多 c_i 次与重要度更低的前一个人交换，输出最终输入编号。最右重要度更大的位置是不可越过的屏障，插入位置 max(i-c_i,barrier+1)。期望 O(nlogn)、空间 O(n)。三组题面样例、独立逐次交换及最大规模边界通过；尚无线上 AC。

所需模板：SequenceMaxTreap（第 125 页，9.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/38/G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    SequenceMaxTreap tree;
    int root = 0;
    for (int i = 0; i < n; i++)
    {
        int importance, limit;
        cin >> importance >> limit;
        int barrier = tree.last_greater(root, importance);
        int pos = max(i - limit, barrier + 1);
        tree.insert(root, pos, importance);
    }
    auto order = tree.ids(root);
    for (int i = 0; i < n; i++)
        cout << order[i] << (i + 1 == n ? '\n' : ' ');
}
```

9.3 Splay 树

9.3.1 Splay：可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义，严格前驱后继不存在返回 nullopt。重复键计数存储；失败查询也伸展最后访问节点，单次均摊 O(log n)。不回收节点，空间按累计新建节点数计算。支持 long long 全范围；不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
struct OrderedSplay
{
    using ll = long long;

    struct Node
    {
        int ch[2]{}, fa = 0, cnt = 0, siz = 0;
        ll val = 0;
    };

    vector<Node> a{Node{}};
    int root = 0;

    int size() const { return a[root].siz; }

    void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }

    int node(ll value, int parent)
    {
        int x = a.size();
        a.push_back({{0, 0}, parent, 1, 1, value});
        return x;
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (z)
            a[z].ch[a[z].ch[1] == y] = x;
        else
            root = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

伸展与查找（接上页同一结构体）：

```
    void splay(int x)
    {
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    // Splay the last visited node even when the value does not exist.
    int find(ll value)
    {
        int x = root, last = 0;
        while (x && a[x].val != value)
        {
            last = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
            splay(x);
        else if (last)
            splay(last);
        return x;
    }
```

插入与删除（接上页同一结构体）：

```
    void insert(ll value)
    {
        int x = root, parent = 0;
        while (x && a[x].val != value)
        {
            parent = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
        {
            ++a[x].cnt;
            pull(x);
        }
        else
        {
            x = node(value, parent);
            if (parent)
                a[parent].ch[value > a[parent].val] = x;
            else
                root = x;
        }
        splay(x);
    }

    bool erase(ll value)
    {
        int x = find(value);
        if (!x) return false;
        if (a[x].cnt > 1)
        {
            --a[x].cnt;
            pull(x);
            return true;
        }
        int l = a[x].ch[0], r = a[x].ch[1];
        if (l) a[l].fa = 0;
        if (r) a[r].fa = 0;
        if (!l)
        {
            root = r;
            return true;
        }
        root = l;
        x = l;
        while (a[x].ch[1]) x = a[x].ch[1];
        splay(x);
        a[x].ch[1] = r;
        if (r) a[r].fa = x;
        pull(x);
        return true;
    }
```

排名与第 k 小（接上页同一结构体）：

```
    int rank(ll value)
    {
        int x = root, last = 0, ans = 1;
        while (x)
        {
            last = x;
            if (value <= a[x].val)
                x = a[x].ch[0];
            else
            {
                ans += a[a[x].ch[0]].siz + a[x].cnt;
                x = a[x].ch[1];
            }
        }
        if (last) splay(last);
        return ans;
    }

    ll kth(int k)
    {
        assert(1 <= k && k <= size());
        int x = root;
        while (true)
        {
            int left = a[a[x].ch[0]].siz;
            if (k <= left)
                x = a[x].ch[0];
            else if (k <= left + a[x].cnt)
            {
                splay(x);
                return a[x].val;
            }
            else
            {
                k -= left + a[x].cnt;
                x = a[x].ch[1];
            }
        }
    }
```

严格前驱与后继（接上页同一结构体）：

```
    optional<ll> prev(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val < value)
            {
                answer = x;
                x = a[x].ch[1];
            }
            else
                x = a[x].ch[0];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }

    optional<ll> next(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val > value)
            {
                answer = x;
                x = a[x].ch[0];
            }
            else
                x = a[x].ch[1];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedSplay（第 129 页，9.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedSplay g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

9.3.2 可合并 Splay：固定编号与集合顺序统计

MergeSplay(v) 为 0-based 元素建立独立单点集合，v 为 vector<long long>，允许空数组，元素数 <INT_MAX 且内存足够。merge(u,v) 合并两个元素所在集合，已经同集合返回 false，否则 true；same(u,v) 判同集合。set(id,val) 改该元素的值而不改编号或集合；value(id) 读值，size(id) 查集合大小。rank(id) 是集合内按 (值, 原编号) 排序的 1-based 名次；kth(id,k) 返回所在集合第 k 小的原编号 optional<int>，k 为 long long，非法时 nullopt，读值用 value(*ans)。重复值保留独立元素，signed64 极值只比较不加减。

初建 O(n)、空间 O(n)，始终仅 n+1 个节点槽；无独立插删、分裂或删除元素接口。合并递归遍历较小集合，将原节点逐个插入较大集合，不用显式栈；递归深度最坏 O(n)。从 n 个单点集合出发，全部成功合并的总成本 $O(n \log^2(n+1))$，其余操作除 value 外均摊 O(log(n+1))，value 为 O(1)，单次非合并操作最坏 O(n)；合并复杂度按前述总界计。节点池与辅助函数内部使用；公开查询会伸展树，根不稳定但原编号稳定。支持独立值拷贝，重建用新对象赋值。

```
struct MergeSplay
{
    using ll = long long;
    struct Node
    {
        int ch[2]{}, fa = 0, siz = 1;
        ll val = 0;
    };
    int n;
    vector<Node> a;

    // Each original ID starts in its own singleton set.
    MergeSplay(const vector<ll> &v)
    {
        assert(v.size() < INT_MAX);
        n = v.size();
        a.resize(n + 1);
        a[0].siz = 0;
        for (int i = 0; i < n; i++) a[i + 1].val = v[i];
    }

    void pull(int x)
    {
        a[x].siz = a[a[x].ch[0]].siz + a[a[x].ch[1]].siz + 1;
    }

    bool less(int x, int y) const
    {
        return pair{a[x].val, x} < pair{a[y].val, y};
    }
```

旋转、伸展与固定编号（接上页同一结构体）：

```
    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
        if (z) a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    int expose(int id)
    {
        assert(0 <= id && id < n);
        int x = id + 1;
        splay(x);
        return x;
    }
```

节点插入与小集合合并（接上页同一结构体）：

```
    int insert(int root, int x)
    {
        if (!root) return x;
        int u = root;
        while (true)
        {
            int k = less(u, x);
            if (!a[u].ch[k])
            {
                a[u].ch[k] = x;
                a[x].fa = u;
                break;
            }
            u = a[u].ch[k];
        }
        splay(x);
        return x;
    }

    void transfer(int u, int &root)
    {
        if (!u) return;
        int l = a[u].ch[0], r = a[u].ch[1];
        a[u] = Node{{0, 0}, 0, 1, a[u].val};
        root = insert(root, u);
        transfer(l, root);
        transfer(r, root);
    }

    bool same(int u, int v)
    {
        int x = expose(u), y = expose(v);
        return x == y || a[x].fa != 0;
    }

    bool merge(int u, int v)
    {
        int x = expose(u), y = expose(v);
        if (x == y || a[x].fa) return false;
        if (a[x].siz < a[y].siz) swap(x, y);
        transfer(y, x);
        return true;
    }

    int size(int id)
    {
        return a[expose(id)].siz;
    }

    ll value(int id) const
    {
        assert(0 <= id && id < n);
        return a[id + 1].val;
    }
```

保持编号的改值、名次与第 k 小（接上页同一结构体）：

```
    // Change this element's value; its ID and set membership do not change.
    void set(int id, ll val)
    {
        int x = expose(id);
        int l = a[x].ch[0], r = a[x].ch[1];
        if (l) a[l].fa = 0;
        if (r) a[r].fa = 0;
        int root = r;
        if (l)
        {
            root = l;
            while (a[root].ch[1]) root = a[root].ch[1];
            splay(root);
            a[root].ch[1] = r;
            if (r) a[r].fa = root;
            pull(root);
        }
        a[x] = Node{{0, 0}, 0, 1, val};
        insert(root, x);
    }

    // 1-based rank among (value, original ID) in this element's set.
    int rank(int id)
    {
        int x = expose(id);
        return a[a[x].ch[0]].siz + 1;
    }

    // Return an original ID, not its value. Invalid k returns nullopt.
    optional<int> kth(int id, ll k)
    {
        int x = expose(id);
        if (k <= 0 || k > a[x].siz) return nullopt;
        while (true)
        {
            int l = a[a[x].ch[0]].siz;
            if (k == l + 1)
            {
                splay(x);
                return x - 1;
            }
            if (k <= l) x = a[x].ch[0];
            else
            {
                k -= l + 1;
                x = a[x].ch[1];
            }
        }
    }
};
```

应用题补充：UVA1479 / 可合并 Splay 与离线图查询

题意：D e 删边、C x v 改点权、Q x k 查连通块第 k 大，非法 k 贡献 0 且计入均值；E 结束组，0 0 结束输入。读入 C 时保存旧值，最终图按未删除边合并，倒序将 D 还原为 merge、C 还原为 set。kth 返回元素编号，value 取权值；第 k 大换算先转 long long，避免 INT_MIN 溢出。无需额外 dsu 和离散化；输出各组查询均值六位小数。

所需模板：MergeSplay（第 135 页，9.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.org/external/14/1479.pdf

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    struct Op
    {
        char op;
        int x, y;
    };
    int n, m, tc = 0;
    while (cin >> n >> m && n)
    {
        vector<long long> a(n);
        for (auto &x : a) cin >> x;
        vector<pair<int, int>> edges(m);
        for (auto &[u, v] : edges)
        {
            cin >> u >> v;
            u--;
            v--;
        }
        vector<int> deleted(m);
        vector<Op> ops;
        char op;
        while (cin >> op && op != 'E')
        {
            int x, y = 0;
            cin >> x;
            x--;
            if (op == 'D') deleted[x] = 1;
            else cin >> y;
            if (op == 'C')
            {
                int old = a[x];
                a[x] = y;
                y = old;
            }
            ops.push_back({op, x, y});
        }
```

继续处理操作（接上页同一 main）：

```
        MergeSplay t(a);
        auto join = [&](int e)
        {
            auto [u, v] = edges[e];
            t.merge(u, v);
        };
        for (int e = 0; e < m; e++)
            if (!deleted[e]) join(e);
        long long sum = 0;
        int count = 0;
        for (int i = (int)ops.size() - 1; i >= 0; i--)
        {
            auto [op, x, y] = ops[i];
            if (op == 'D') join(x);
            else if (op == 'C')
            {
                t.set(x, y);
            }
            else
            {
                long long k = (long long)t.size(x) - y + 1;
                auto id = t.kth(x, k);
                if (id) sum += t.value(*id);
                count++;
            }
        }
        cout << "Case " << ++tc << ": " << fixed << setprecision(6)
             << (long double)sum / count << '\n';
    }
}
```

应用题补充：Luogu P3224 / 永无乡

题意：n 个岛的重要度为 1..n 排列，初始 m 条桥；B x y 连桥，Q x k 输出连通块中第 k 小重要度的岛编号，不存在输出 −1。merge 以元素原编号为入口，kth 结果加 1 转换为题目编号；不能误输出重要度。n,m≤100000，q≤300000。

所需模板：MergeSplay（第 135 页，9.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3224

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplay t(a);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.merge(u - 1, v - 1);
    }
    int q;
    cin >> q;
    while (q--)
    {
        char op;
        int x, y;
        cin >> op >> x >> y;
        if (op == 'B') t.merge(x - 1, y - 1);
        else
        {
            auto id = t.kth(x - 1, y);
            cout << (id ? *id + 1 : -1) << '\n';
        }
    }
}
```

接口用法演示：可合并有序集合与元素改值（接口演示）

演示约定：输入 n,q 及 n 个 signed64 值，初始各为单点集合，编号 0-based。0 x y 合并集合并输出是否改变；1 x v 改值；2 x k 输出第 k 小元素的原编号和值，非法 k 输出 −1；3 x 输出集合大小与 x 的 1-based 名次。并列按原编号升序；合并与修改之后编号仍有效。

所需模板：MergeSplay（第 135 页，9.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/splay/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplay t(a);
    while (q--)
    {
        int op, x;
        long long y;
        cin >> op >> x;
        if (op <= 2) cin >> y;
        if (op == 0) cout << t.merge(x, y) << '\n';
        else if (op == 1) t.set(x, y);
        else if (op == 2)
        {
            auto id = t.kth(x, y);
            if (id) cout << *id << ' ' << t.value(*id) << '\n';
            else cout << -1 << '\n';
        }
        else cout << t.size(x) << ' ' << t.rank(x) << '\n';
    }
}
```

相关应用：来源专用值域平移/历史计数权值索引，不代表整页已覆盖

9.3.3 按值分裂与整体平移 Splay

空构造，根 0 为空；同一对象内可管理多棵互不相交的树。insert(root,value) 通过引用更新根，返回稳定的 1-based 节点 ID；重复值允许。split(root,key) 消耗旧根，返回值 <key 与 >=key 两棵根。add(root,delta) 全树平移。merge(left,right) 消耗两根，要求左树所有值 <= 右树所有值；不是任意集合合并。value(root,id) 读取本树节点并通过引用更新根，节点 ID 稳定，根会变化。所有根和 ID 必须属于同一对象，id 必须属于给定树；消耗后的旧根不能再当独立树使用。

insert/split/merge/value 均摊 O(log(N+1))，单次最坏 O(N)，add 为 O(1)，空间 O(N)，不回收节点。所有实际值及 val/lazy 每次加法中间结果须适配 signed64；最终值合法不能代替懒标记不溢出的要求。祖先下传递归，需 O(N) 栈空间。对象复制独立，辅助函数与 a 仅供内部实现。

```
struct ShiftSplay
{
    using ll = long long;
    struct Node
    {
        int ch[2]{}, fa = 0;
        ll val = 0, lazy = 0;
    };
    vector<Node> a{Node{}};

    void add(int root, ll delta)
    {
        if (!root)
            return;
        a[root].val += delta;
        a[root].lazy += delta;
    }

    void push(int x)
    {
        add(a[x].ch[0], a[x].lazy);
        add(a[x].ch[1], a[x].lazy);
        a[x].lazy = 0;
    }

    void down(int x)
    {
        if (a[x].fa)
            down(a[x].fa);
        push(x);
    }
```

接上页同一结构体：

```
    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (z)
            a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w)
            a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
    }

    void splay(int x)
    {
        down(x);
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    int insert(int &root, ll value)
    {
        assert(a.size() < INT_MAX);
        int x = a.size();
        a.push_back(Node{});
        a[x].val = value;
        int p = root, parent = 0, side = 0;
        while (p)
        {
            push(p);
            parent = p;
            side = value >= a[p].val;
            p = a[p].ch[side];
        }
        a[x].fa = parent;
        if (parent)
            a[parent].ch[side] = x;
        splay(x);
        root = x;
        return x;
    }
```

接上页同一结构体：

```
    pair<int, int> split(int root, ll key)
    {
        if (!root)
            return {0, 0};
        int p = root, last = root, next = 0;
        while (p)
        {
            push(p);
            last = p;
            if (a[p].val >= key)
            {
                next = p;
                p = a[p].ch[0];
            }
            else
                p = a[p].ch[1];
        }
        splay(last);
        if (!next)
            return {last, 0};
        splay(next);
        int left = a[next].ch[0];
        a[next].ch[0] = 0;
        if (left)
            a[left].fa = 0;
        return {left, next};
    }

    int merge(int left, int right)
    {
        if (!left || !right)
            return left ? left : right;
        int x = left;
        while (a[x].ch[1])
            x = a[x].ch[1];
        splay(x);
        a[x].ch[1] = right;
        a[right].fa = x;
        return x;
    }

    ll value(int &root, int id)
    {
        assert(0 < id && id < (int)a.size());
        splay(id);
        root = id;
        return a[id].val;
    }
};
```

接口用法演示：按值分裂与整体平移 Splay API

演示约定：输入 q，初始树槽 0 为空。1 r v 插入并输出节点 ID；2 r key 把槽 r 保留 <key 部分，新建槽存 >=key 部分并输出槽号；3 r delta 全树平移；4 r s 有序合并到 r 并清空 s（r!=s，且左侧所有值 <= 右侧）；5 r id 输出该槽节点当前值。槽号 0-based，ID 从 1 递增，空槽可复用。仅合法操作，所有中间加法需适配 signed64。本地 API 例子，不是 OJ 判定。

所需模板：ShiftSplay（第 143 页，9.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-SHIFT-SPLAY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    ShiftSplay tree;
    vector<int> roots(1);
    while (q--)
    {
        int op, r;
        long long x;
        cin >> op >> r >> x;
        if (op == 1)
        {
            cout << tree.insert(roots[r], x) << '\n';
        }
        else if (op == 2)
        {
            auto [left, right] = tree.split(roots[r], x);
            roots[r] = left;
            roots.push_back(right);
            cout << roots.size() - 1 << '\n';
        }
        else if (op == 3)
        {
            tree.add(roots[r], x);
        }
        else if (op == 4)
        {
            int s = x;
            roots[r] = tree.merge(roots[r], roots[s]);
            roots[s] = 0;
        }
        else
        {
            int id = x;
            cout << tree.value(roots[r], id) << '\n';
        }
    }
}
```

9.3.4 区间翻转 Splay 与固定编号定位

SequenceSplay(n) 建立原编号 0..n−1 的序列，允许 n=0，要求 n≤INT_MAX−3 且内存足够。reverse(l,r) 翻转当前 0-based 半开区间 [l,r)，允许空区间；pos(id) 返回原编号 id 的当前位置；at(k) 返回当前位置 k 的原编号；order() 按当前顺序返回全部原编号。值由调用方按原编号存储，重复值不会合并。

初建 O(n)，pos/at/reverse 均摊 O(log(n+1))、单次最坏 O(n)，order 为 O(n)，空间 O(n)；不含插入、删除和数值聚合。所有辅助函数与 a/root/n 约定内部只读，pos/at 会伸展并改变树形。两哨兵不暴露给用户；节点编号不随旋转变化。build/祖先下传/遍历均递归，退化时深度 O(n)，调用方保证栈空间。支持独立值拷贝；重置用新建对象赋值。

```
struct SequenceSplay
{
    struct Node
    {
        int ch[2]{}, fa = 0, siz = 0;
        bool rev = false;
    };
    int n, root;
    vector<Node> a;

    // Initial order: original IDs 0, 1, ..., n-1.
    SequenceSplay(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX - 3);
        a.resize(n + 3);
        root = build(1, n + 3, 0);
    }

    void pull(int x)
    {
        a[x].siz = a[a[x].ch[0]].siz + a[a[x].ch[1]].siz + 1;
    }

    int build(int l, int r, int p)
    {
        if (l == r) return 0;
        int x = l + (r - l) / 2;
        a[x].fa = p;
        a[x].ch[0] = build(l, x, x);
        a[x].ch[1] = build(x + 1, r, x);
        pull(x);
        return x;
    }
```

翻转下传与伸展旋转（接上页同一结构体）：

```
    void flip(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        flip(a[x].ch[0]);
        flip(a[x].ch[1]);
        a[x].rev = false;
    }

    void push_path(int x)
    {
        if (a[x].fa) push_path(a[x].fa);
        push(x);
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
        if (z) a[z].ch[a[z].ch[1] == y] = x;
        else root = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }

    void splay(int x, int goal = 0)
    {
        push_path(x);
        while (a[x].fa != goal)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z != goal)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }
```

位置和原编号互查（接上页同一结构体）：

```
    // Internal rank includes the two sentinels; does not splay.
    int select(int k)
    {
        int x = root;
        while (true)
        {
            push(x);
            int l = a[a[x].ch[0]].siz;
            if (k == l) return x;
            if (k < l) x = a[x].ch[0];
            else
            {
                k -= l + 1;
                x = a[x].ch[1];
            }
        }
    }

    // Current 0-based position of an original ID.
    int pos(int id)
    {
        assert(0 <= id && id < n);
        int x = id + 2;
        splay(x);
        return a[a[x].ch[0]].siz - 1;
    }

    // Original ID at the current 0-based position.
    int at(int k)
    {
        assert(0 <= k && k < n);
        int x = select(k + 1);
        splay(x);
        return x - 2;
    }
```

区间翻转与完整顺序（接上页同一结构体）：

```
    void reverse(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        int x = select(l);
        splay(x);
        int y = select(r + 1);
        splay(y, x);
        flip(a[y].ch[0]);
    }

    void collect(int x, vector<int> &v)
    {
        if (!x) return;
        push(x);
        collect(a[x].ch[0], v);
        if (2 <= x && x <= n + 1) v.push_back(x - 2);
        collect(a[x].ch[1], v);
    }

    vector<int> order()
    {
        vector<int> v;
        v.reserve(n);
        collect(root, v);
        return v;
    }
};
```

模板题使用：Luogu P3391 / Splay 区间翻转

题意：初始序列 1..n，m 次翻转 1-based 闭区间 [l,r]，输出最终序列；转为 reverse(l−1,r)，order() 中的原编号加 1。n,m≤100000。

所需模板：SequenceSplay（第 147 页，9.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3391

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SequenceSplay t(n);
    while (m--)
    {
        int l, r;
        cin >> l >> r;
        t.reverse(l - 1, r);
    }
    auto a = t.order();
    for (int i = 0; i < n; i++)
        cout << a[i] + 1 << (i + 1 == n ? '\n' : ' ');
}
```

应用题补充：Luogu P3165 / 排序机械臂

题意：按 (值, 原编号) 排序确定稳定目标次序，第 i 步定位该元素当前 0-based 位置 p，输出 p+1，再翻转 [i,p+1)。重复高度按原编号打破并列，不能按翻转后的次序重新判定。n≤100000；与 kuangbin3.4.1 的固定编号定位、反转操作模型对应。

所需模板：SequenceSplay（第 147 页，9.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3165

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n), id(n);
    for (auto &x : a) cin >> x;
    iota(id.begin(), id.end(), 0);
    sort(id.begin(), id.end(), [&](int u, int v)
    {
        return pair{a[u], u} < pair{a[v], v};
    });
    SequenceSplay t(n);
    for (int i = 0; i < n; i++)
    {
        int p = t.pos(id[i]);
        cout << p + 1 << (i + 1 == n ? '\n' : ' ');
        t.reverse(i, p + 1);
    }
}
```

接口用法演示：原编号与当前位置互查（接口演示）

演示约定：输入 n,q，初始编号 0..n−1；0 l r 翻转半开区间；1 id 查原编号的当前位置；2 k 查当前位置的原编号；3 输出完整顺序。允许空序列与空翻转；空序列不允许 pos/at。

所需模板：SequenceSplay（第 147 页，9.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/splay/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    SequenceSplay t(n);
    while (q--)
    {
        int op, x, y;
        cin >> op;
        if (op == 0)
        {
            cin >> x >> y;
            t.reverse(x, y);
        }
        else if (op == 1)
        {
            cin >> x;
            cout << t.pos(x) << '\n';
        }
        else if (op == 2)
        {
            cin >> x;
            cout << t.at(x) << '\n';
        }
        else
        {
            auto a = t.order();
            for (int i = 0; i < n; i++)
                cout << a[i] << (i + 1 == n ? '\n' : ' ');
            if (n == 0) cout << '\n';
        }
    }
}
```

9.4 替罪羊树

9.4.1 替罪羊树：可重集合与顺序统计

默认空树；insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在。less(x) 计严格小于 x 的元素，less(x,true) 计小于等于；rank(x)=less(x)+1，x 不必存在。kth(k) 要求 1≤k≤size()。prev/next 返回严格前驱/后继，不存在为 nullopt。键支持完整 long long，不使用数值哨兵。节点数过度偏向单侧或有效不同值不足一半时递归重建，复用活节点编号。若 d 为有效不同值数，查询 O(log(d+2))，更新摊 O(log(d+2))、单次最坏 O(d+1)。元素数与累计分配节点数均须小于 INT_MAX。内存 O(K)，K 为累计分配节点数；墓碑从树中清除后不回收到节点池，重建本身不增分配量。a、root 和辅助接口只供内部使用；重新赋值 ScapegoatTree() 可清空。

```
struct ScapegoatTree
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, count = 0, size = 0, nodes = 0, live = 0;
        ll value = 0;
    };

    vector<Node> a{Node{}};
    int root = 0;

    int size() const { return a[root].size; }

    void pull(int p)
    {
        int l = a[p].l, r = a[p].r;
        a[p].size = a[l].size + a[r].size + a[p].count;
        a[p].nodes = a[l].nodes + a[r].nodes + 1;
        a[p].live = a[l].live + a[r].live + (a[p].count > 0);
    }

    void collect(int p, vector<int> &v)
    {
        if (!p) return;
        collect(a[p].l, v);
        if (a[p].count) v.push_back(p);
        collect(a[p].r, v);
    }

    int build(const vector<int> &v, int l, int r)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2, p = v[m];
        a[p].l = build(v, l, m);
        a[p].r = build(v, m + 1, r);
        pull(p);
        return p;
    }
```

重建与插入删除（接上页同一结构体）：

```
    int balance(int p)
    {
        pull(p);
        int heavy = max(a[a[p].l].nodes, a[a[p].r].nodes);
        if (4LL * heavy <= 3LL * a[p].nodes && 2LL * a[p].live >= a[p].nodes) return p;
        vector<int> v;
        v.reserve(a[p].live);
        collect(p, v);
        return build(v, 0, v.size());
    }

    int insert(int p, ll x)
    {
        if (!p)
        {
            a.push_back({0, 0, 1, 1, 1, 1, x});
            return a.size() - 1;
        }
        if (x < a[p].value)
            a[p].l = insert(a[p].l, x);
        else if (x > a[p].value)
            a[p].r = insert(a[p].r, x);
        else
            a[p].count++;
        return balance(p);
    }

    int erase(int p, ll x, bool &found)
    {
        if (!p) return 0;
        if (x < a[p].value)
            a[p].l = erase(a[p].l, x, found);
        else if (x > a[p].value)
            a[p].r = erase(a[p].r, x, found);
        else if (a[p].count)
        {
            a[p].count--;
            found = true;
        }
        return balance(p);
    }

    void insert(ll x) { root = insert(root, x); }

    bool erase(ll x)
    {
        bool found = false;
        root = erase(root, x, found);
        return found;
    }
```

排名、第 k 小与严格前驱后继（接上页同一结构体）：

```
    int less(ll x, bool equal = false) const
    {
        int p = root, answer = 0;
        while (p)
        {
            if (a[p].value < x || (equal && a[p].value == x))
            {
                answer += a[a[p].l].size + a[p].count;
                p = a[p].r;
            }
            else
                p = a[p].l;
        }
        return answer;
    }

    int rank(ll x) const { return less(x) + 1; }

    ll kth(int k) const
    {
        assert(1 <= k && k <= size());
        int p = root;
        while (true)
        {
            int left = a[a[p].l].size;
            if (k <= left)
                p = a[p].l;
            else if (k <= left + a[p].count)
                return a[p].value;
            else
            {
                k -= left + a[p].count;
                p = a[p].r;
            }
        }
    }

    optional<ll> prev(ll x) const
    {
        int k = less(x);
        if (!k) return nullopt;
        return kth(k);
    }

    optional<ll> next(ll x) const
    {
        int k = less(x, true);
        if (k == size()) return nullopt;
        return kth(k + 1);
    }
};
```

模板题使用：Luogu P3369（替罪羊树）

题意：维护可重集合，操作 1..6 分别为插入、删除一个副本、排名、第 k 小、严格前驱、严格后继。排名查询键不一定存在；题目保证第 k 小和前驱后继的答案存在，因此可解引用 optional。该驱动单独调用替罪羊树，不沿用其他平衡树的 AC。

所需模板：ScapegoatTree（第 154 页，9.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    ScapegoatTree g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

9.4.2 在线序列：插入、修改与区间第 k 小

代码依赖：ost（第 104 页，9.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

依赖 ost（GNU PBDS，非 ISO 标准库），C++20。以 vector<long long> 构造，默认空序列。insert(k,x) 在前 k 项后插入，0≤k≤size()；set(k,x) 修改第 k 项。less(l,r,x) 计严格小于 x，末参 true 计 ≤x；kth(l,r,k) 取第 k 小，区间和 k 均 1-based 且非空合法。支持重复值、完整 signed64，midpoint 避免二分溢出；不提供删除。稳定节点编号区分重复值，递归重量重建。秩/修改 $O(\log^2(n+2))$，kth 最多 64 轮秩查询；构建 $O(n \log^2(n+2))$，插入保守均摊 $O(\log^3(n+2))$、单次最坏 $O(n \log^2(n+2))$。空间 $O(n \log(n+2))$，vector 扩容可能复制内树并增大峰值；节点数小于 INT_MAX。a/root 及带节点编号辅助方法内部使用，不外部改写。

```
struct SequenceKth
{
    using ll = long long;
    using Key = pair<ll, int>;

    struct Node
    {
        int l = 0, r = 0, sz = 0;
        ll val = 0;
        ost<Key> keys;
    };

    vector<Node> a;
    int root = 0;

    SequenceKth(const vector<ll> &v = {}) : a(v.size() + 1)
    {
        vector<int> ids;
        for (int i = 0; i < (int)v.size(); i++)
        {
            a[i + 1].val = v[i];
            ids.push_back(i + 1);
        }
        root = build(ids, 0, (int)ids.size());
    }

    int size() const
    {
        return a[root].sz;
    }
```

内树合并与递归重建（接上页同一结构体）：

```
    void pull(int u)
    {
        int l = a[u].l;
        int r = a[u].r;
        a[u].sz = a[l].sz + a[r].sz + 1;
        a[u].keys.clear();
        a[u].keys.insert({a[u].val, u});
        for (auto x : a[l].keys)
            a[u].keys.insert(x);
        for (auto x : a[r].keys)
            a[u].keys.insert(x);
    }

    int build(const vector<int> &v, int l, int r)
    {
        if (l == r)
            return 0;
        int m = l + (r - l) / 2;
        int u = v[m];
        a[u].l = build(v, l, m);
        a[u].r = build(v, m + 1, r);
        pull(u);
        return u;
    }

    void collect(int u, vector<int> &v) const
    {
        if (!u)
            return;
        collect(a[u].l, v);
        v.push_back(u);
        collect(a[u].r, v);
    }

    int balance(int u)
    {
        a[u].sz = a[a[u].l].sz + a[a[u].r].sz + 1;
        if (4LL * max(a[a[u].l].sz, a[a[u].r].sz) <= 3LL * a[u].sz)
            return u;
        vector<int> v;
        v.reserve(a[u].sz);
        collect(u, v);
        return build(v, 0, (int)v.size());
    }
```

按位置插入与修改（接上页同一结构体）：

```
    int insert(int u, int k, int id)
    {
        if (!u)
            return id;
        a[u].keys.insert({a[id].val, id});
        int left = a[a[u].l].sz;
        if (k <= left)
            a[u].l = insert(a[u].l, k, id);
        else
            a[u].r = insert(a[u].r, k - left - 1, id);
        return balance(u);
    }

    void insert(int k, ll x)
    {
        assert(0 <= k && k <= size());
        int id = (int)a.size();
        a.emplace_back();
        a[id].val = x;
        a[id].sz = 1;
        a[id].keys.insert({x, id});
        root = insert(root, k, id);
    }

    Key set(int u, int k, ll x)
    {
        int left = a[a[u].l].sz;
        Key old;
        if (k <= left)
            old = set(a[u].l, k, x);
        else if (k == left + 1)
        {
            old = {a[u].val, u};
            a[u].val = x;
        }
        else
            old = set(a[u].r, k - left - 1, x);
        a[u].keys.erase(old);
        a[u].keys.insert({x, old.second});
        return old;
    }

    void set(int k, ll x)
    {
        assert(1 <= k && k <= size());
        set(root, k, x);
    }
```

区间秩与第 k 小查询（接上页同一结构体）：

```
    int less(int u, int l, int r, ll x, bool equal) const
    {
        if (!u)
            return 0;
        if (l <= 1 && a[u].sz <= r)
            return a[u].keys.order_of_key({x, equal ? (int)a.size() : 0});
        int left = a[a[u].l].sz;
        int ans = 0;
        if (l <= left)
            ans += less(a[u].l, l, r, x, equal);
        if (l <= left + 1 && left + 1 <= r)
            ans += a[u].val < x || (equal && a[u].val == x);
        if (r > left + 1)
            ans += less(a[u].r, l - left - 1, r - left - 1, x, equal);
        return ans;
    }

    int less(int l, int r, ll x, bool equal = false) const
    {
        assert(1 <= l && l <= r && r <= size());
        return less(root, l, r, x, equal);
    }

    ll kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r <= size());
        assert(1 <= k && k <= r - l + 1);
        ll lo = a[root].keys.begin()->first;
        ll hi = a[root].keys.rbegin()->first;
        while (lo < hi)
        {
            ll mid = midpoint(lo, hi);
            if (less(l, r, mid, true) >= k)
                hi = mid;
            else
                lo = mid + 1;
        }
        return lo;
    }
};
```

接口用法演示：带插入区间第 k 小（kuangbin BZOJ3065 模型）

演示约定：来源协议：EOF 多组，每组 n、初值、q 及 Q/M/I 操作，值域 0..100000。所有整数参数与上次查询答案异或；Q x y k 查闭区间第 k 小，M x val 修改第 x 项，I x val 在第 x 项前插入（调用 insert(x-1,val)）。仅 Q 更新 lastans，解码后端点有序且合法。核心可用 signed64，协议答案限定来源值域后转 int。官方题面与资源限制未核验，只登记来源模型/API 用法。

所需模板：ost（第 104 页，9.1.2 节）、SequenceKth（第 158 页，9.4.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://www.lydsy.com/JudgeOnline/problem.php?id=3065

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        vector<long long> a(n);
        for (auto &x : a)
            cin >> x;
        SequenceKth tree(a);
        int q;
        cin >> q;
        int ans = 0;
        while (q--)
        {
            char op;
            int x, y;
            cin >> op >> x >> y;
            x ^= ans;
            y ^= ans;
            if (op == 'Q')
            {
                int k;
                cin >> k;
                k ^= ans;
                ans = (int)tree.kth(x, y, k);
                cout << ans << '\n';
            }
            else if (op == 'M')
                tree.set(x, y);
            else
                tree.insert(x - 1, y);
        }
    }
}
```

9.4.3 序列替罪羊树：按位置插删与区间等值计数

以 vector<long long> 构造，默认空序列。insert(k,x) 在前 k 项后插入，0≤k≤size()；erase(k) 删除第 k 项并返回值；count(l,r,x) 计 1-based 非空闭区间内等于 x 的项数；rotate(l,r) 将区间末项移至开头。键支持 long long 全范围。递归按位置导航，墓碑过多或节点数失衡时重建，回收死节点并删除零频次。map 频次表令查询 $O(\log^2(n+2))$，重建 s 点 $O(s\ \log^2(s+2))$，更新保守均摊 $O(\log^3(N+2))$、单次最坏 $O(N\ \log^2(N+2))$，N 为历史最大长度；初建 $O(n\ \log^2(n+2))$。频次表总空间 $O(n\ \log(n+2))$，vector 池 $O(N)$，节点数须小于 INT_MAX。a/root/spare 及带节点编号辅助函数均内部使用，不能外部改动。

```
struct SequenceScapegoat
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sz = 0, cnt = 0;
        ll val = 0;
        bool live = false;
        map<ll, int> freq;
    };

    vector<Node> a{Node{}};
    vector<int> spare;
    int root = 0;

    SequenceScapegoat(const vector<ll> &v = {})
    {
        vector<int> ids;
        for (ll x : v)
            ids.push_back(make(x));
        root = build(ids, 0, (int)ids.size());
    }

    int size() const
    {
        return a[root].sz;
    }
```

节点复用与频次维护（接上页同一结构体）：

```
    int make(ll x)
    {
        int u;
        if (spare.empty())
        {
            u = (int)a.size();
            a.push_back(Node{});
        }
        else
        {
            u = spare.back();
            spare.pop_back();
        }
        a[u].sz = 1;
        a[u].cnt = 1;
        a[u].val = x;
        a[u].live = true;
        a[u].freq[x] = 1;
        return u;
    }

    void change(int u, ll x, int d)
    {
        int &v = a[u].freq[x];
        v += d;
        if (v == 0)
            a[u].freq.erase(x);
    }

    void pull(int u)
    {
        int l = a[u].l;
        int r = a[u].r;
        a[u].sz = a[l].sz + a[r].sz + a[u].live;
        a[u].cnt = a[l].cnt + a[r].cnt + 1;
        a[u].freq.clear();
        if (a[u].live)
            a[u].freq[a[u].val] = 1;
        for (auto [x, c] : a[l].freq)
            a[u].freq[x] += c;
        for (auto [x, c] : a[r].freq)
            a[u].freq[x] += c;
    }
```

递归收集、重建与平衡（接上页同一结构体）：

```
    void collect(int u, vector<int> &v)
    {
        if (!u)
            return;
        collect(a[u].l, v);
        if (a[u].live)
            v.push_back(u);
        collect(a[u].r, v);
        if (!a[u].live)
        {
            a[u] = Node{};
            spare.push_back(u);
        }
    }

    int build(const vector<int> &v, int l, int r)
    {
        if (l == r)
            return 0;
        int m = l + (r - l) / 2;
        int u = v[m];
        a[u].l = build(v, l, m);
        a[u].r = build(v, m + 1, r);
        pull(u);
        return u;
    }

    int balance(int u)
    {
        int heavy = max(a[a[u].l].cnt, a[a[u].r].cnt);
        if (4LL * heavy <= 3LL * a[u].cnt && 2LL * a[u].sz >= a[u].cnt)
            return u;
        vector<int> v;
        v.reserve(a[u].sz);
        collect(u, v);
        return build(v, 0, (int)v.size());
    }
```

按位置插入与删除（接上页同一结构体）：

```
    int insert(int u, int k, ll x)
    {
        if (!u)
            return make(x);
        int left = a[a[u].l].sz;
        if (k <= left)
            a[u].l = insert(a[u].l, k, x);
        else
            a[u].r = insert(a[u].r, k - left - a[u].live, x);
        a[u].sz++;
        a[u].cnt = a[a[u].l].cnt + a[a[u].r].cnt + 1;
        change(u, x, 1);
        return balance(u);
    }

    void insert(int k, ll x)
    {
        assert(0 <= k && k <= size());
        root = insert(root, k, x);
    }

    int erase(int u, int k, ll &x)
    {
        int left = a[a[u].l].sz;
        if (k <= left)
            a[u].l = erase(a[u].l, k, x);
        else if (a[u].live && k == left + 1)
        {
            x = a[u].val;
            a[u].live = false;
        }
        else
            a[u].r = erase(a[u].r, k - left - a[u].live, x);
        a[u].sz--;
        a[u].cnt = a[a[u].l].cnt + a[a[u].r].cnt + 1;
        change(u, x, -1);
        return balance(u);
    }

    ll erase(int k)
    {
        assert(1 <= k && k <= size());
        ll x = 0;
        root = erase(root, k, x);
        return x;
    }
```

区间等值计数与右移（接上页同一结构体）：

```
    int count(int u, int l, int r, ll x) const
    {
        if (!u || l > r)
            return 0;
        if (l <= 1 && a[u].sz <= r)
        {
            auto it = a[u].freq.find(x);
            return it == a[u].freq.end() ? 0 : it->second;
        }
        int left = a[a[u].l].sz;
        int ans = 0;
        if (l <= left)
            ans += count(a[u].l, l, r, x);
        if (a[u].live && l <= left + 1 && left + 1 <= r && a[u].val == x)
            ans++;
        int offset = left + a[u].live;
        if (r > offset)
            ans += count(a[u].r, l - offset, r - offset, x);
        return ans;
    }

    int count(int l, int r, ll x) const
    {
        assert(1 <= l && l <= r && r <= size());
        return count(root, l, r, x);
    }

    void rotate(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        ll x = erase(r);
        insert(l - 1, x);
    }
};
```

应用题补充：Codeforces 455D / Serega and Fun

题意：n,q≤100000，初值和原始参数均 1..n。操作 1 把解码区间的末项移至开头，操作 2 计区间中等于解码 k 的项数。每个参数先用 (v+lastans-1)%n+1 解码，再交换逆序端点；lastans 只在查询后更新。比赛应用，不代替独立模板题；插入/删除等额外接口另有独立测试，尚无线上 AC 或资源通过声明。

所需模板：SequenceScapegoat（第 163 页，9.4.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/455/D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a)
        cin >> x;
    SequenceScapegoat tree(a);
    int q;
    cin >> q;
    int ans = 0;
    while (q--)
    {
        int op, l, r;
        cin >> op >> l >> r;
        l = (l + ans - 1) % n + 1;
        r = (r + ans - 1) % n + 1;
        if (l > r)
            swap(l, r);
        if (op == 1)
            tree.rotate(l, r);
        else
        {
            int k;
            cin >> k;
            k = (k + ans - 1) % n + 1;
            ans = tree.count(l, r, k);
            cout << ans << '\n';
        }
    }
}
```

9.5 笛卡尔树

9.5.1 笛卡尔树与相等值优先级

cartesian_tree(a) 返回 CartesianTree {root,left,right}，下标 0..n-1，缺失孩子及空树根为-1。中序遍历为原下标顺序，按值为小根堆；相等值保留较小下标在上方，即区间最左最小值为根。输入不修改，n 须能用 int 表示，元素比较须构成严格弱序，浮点 NaN 不适用。

比较 O(1) 时，构造 O(n)，结果数组及辅助单调栈均 O(n)。每点入栈一次、出栈至多一次；单调栈维护树的最右路径，不模拟 DFS。弹出节点中最后一个成为新节点的左孩子，剩余栈顶的右孩子改成新节点。P5854 用 1..n 排列，n≤10^7；P5854 已用 C++14 GCC9 满分通过，排名未核验。

```
struct CartesianTree
{
    int root;
    vector<int> left, right;
};

template <class T>
CartesianTree cartesian_tree(const vector<T> &a)
{
    int n = a.size();
    CartesianTree tree{-1, vector<int>(n, -1), vector<int>(n, -1)};
    vector<int> st;
    st.reserve(n);
    for (int i = 0; i < n; i++)
    {
        int last = -1;
        while (!st.empty() && a[i] < a[st.back()])
        {
            last = st.back();
            st.pop_back();
        }
        tree.left[i] = last;
        if (!st.empty()) tree.right[st.back()] = i;
        st.push_back(i);
    }
    if (n) tree.root = st.front();
    return tree;
}
```

模板题使用：P5854 笛卡尔树

题意：输入 1..n 的排列，构造小根笛卡尔树并输出左右孩子的异或校验和。孩子由 0-based 转为 1-based，缺失孩子为 0；乘法使用 long long。

所需模板：CartesianTree（第 169 页，9.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5854

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto tree = cartesian_tree(a);
    long long left = 0;
    long long right = 0;
    for (int i = 0; i < n; i++)
    {
        left ^= 1LL * (i + 1) * (tree.left[i] + 2);
        right ^= 1LL * (i + 1) * (tree.right[i] + 2);
    }
    cout << left << ' ' << right << '\n';
}
```

第十章　可持久化数据结构

10.1　可持久化线段树

10.1.1　主席树：静态区间第 k 小

构造传入 0-based 数组，kth(l,r,k) 的区间使用 1-based 闭区间；要求 1≤k≤r-l+1。离散化后每个前缀一个版本，单次 O(log n)，总空间 O(n log n)。

```
struct PersistentKth
{
    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    vector<long long> vals;

    PersistentKth(const vector<long long> &a) : vals(a)
    {
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (long long x : a)
        {
            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
        }
    }

    int add(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        ++t[p].sum;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int v = add(t[old].l, l, m, x);
                t[p].l = v;
            }
            else
            {
                int v = add(t[old].r, m + 1, r, x);
                t[p].r = v;
            }
        }
        return p;
```

```
    }

    long long kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
            if (k <= c)
            {
                u = t[u].l;
                v = t[v].l;
                R = m;
            }
            else
            {
                k -= c;
                u = t[u].r;
                v = t[v].r;
                L = m + 1;
            }
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P3834

题意：给定不再修改的数组，查询闭区间 [l,r] 内第 k 小的值，重复值按出现次数计数。构造传入 0-based vector，查询 l、r、k 均从 1 开始。

所需模板：PersistentKth（第 171 页，10.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentKth tree(a);
    while (m--)
    {
        int l, r, k;
        std::cin >> l >> r >> k;
        std::cout << tree.kth(l, r, k) << '\n';
    }
}
```

10.1.2 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0)，a 是非空 0-based 初值数组；第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置，创建并返回新版本编号；copy(version) 以 O(1) 时间复制版本；query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点，旧版本不变；build 为内部函数，不可覆盖历史节点。建树 O(n)，修改及查询 O(log n)，初始 2n-1 个节点，每次修改至多 ceil(log2(n))+1 个；总空间 O(n+ 修改数 *log n+ 版本数)。changes 仅用于预留节点池，低估后可扩容，所有外部引用应使用下标，不保存 vector 元素引用。节点下标用 int，节点总量须可表示。

```
struct PersistentArray
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0;
        ll value = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root;

    PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
    {
        assert(n > 0 && changes >= 0);
        int levels = 1;
        for (int x = n - 1; x; x >>= 1) levels++;
        t.reserve(2ULL * n + 1ULL * changes * levels + 1);
        root.push_back(build(1, n, a));
    }

    int build(int l, int r, const vector<ll> &a)
    {
        int p = t.size();
        t.push_back(Node{});
        if (l == r)
            t[p].value = a[l - 1];
        else
        {
            int m = (l + r) / 2;
            int left = build(l, m, a);
            int right = build(m + 1, r, a);
            t[p].l = left;
            t[p].r = right;
        }
        return p;
    }

    int modify(int old, int l, int r, int pos, ll value)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        if (l == r)
            t[p].value = value;
```

```
        else
        {
            int m = (l + r) / 2;
            if (pos <= m)
            {
                int child = modify(t[old].l, l, m, pos, value);
                t[p].l = child;
            }
            else
            {
                int child = modify(t[old].r, m + 1, r, pos, value);
                t[p].r = child;
            }
        }
        return p;
    }

    // Create a new version from any existing version. Positions are 1-based.
    int set(int version, int pos, ll value)
    {
        assert(0 <= version && version < (int)root.size());
        assert(1 <= pos && pos <= n);
        int p = modify(root[version], 1, n, pos, value);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int copy(int version)
    {
        assert(0 <= version && version < (int)root.size());
        root.push_back(root[version]);
        return (int)root.size() - 1;
    }

    ll query(int version, int pos) const
    {
        assert(0 <= version && version < (int)root.size());
        assert(1 <= pos && pos <= n);
        int p = root[version], l = 1, r = n;
        while (l < r)
        {
            int m = (l + r) / 2;
            if (pos <= m)
            {
                p = t[p].l;
                r = m;
```

```
            }
            else
            {
                p = t[p].r;
                l = m + 1;
            }
        }
        return t[p].value;
    }
};
```

模板题使用：Luogu P3919

题意：从指定历史版本修改单点或查询单点，每次操作都创建新版本。查询后也要 copy 原版本；版本号从 0 开始，位置为 1-based。

所需模板：PersistentArray（第 173 页，10.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3919

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentArray tree(a, m);
    while (m--)
    {
        int version, op, pos;
        std::cin >> version >> op >> pos;
        if (op == 1)
        {
            long long value;
            std::cin >> value;
            tree.set(version, pos, value);
        }
        else
        {
            std::cout << tree.query(version, pos) << '\n';
            tree.copy(version);
        }
    }
}
```

10.1.3 可持久化区间加与版本拼接

PersistentRange(n) 建立长度 n 的全零版本 0，n≥1；使用隐式节点，不预建整棵树。add(v,l,r,x) 在版本 v 的 [l,r) 加 x；splice(dst,src,l,r) 将 dst 的 [l,r) 替换为 src 同一位置的值，其他位置保留 dst；二者均返回新版本编号。query(v,l,r) 返回区间和，const 且不分配节点。下标 0-based、区间半开，允许空区间；空修改及零增量仍创建版本但复用根，整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先，用 offset 修正跨版本拼接；每次修改/拼接/查询 O(log n)，版本别名 O(1)，总节点 O(q log n)。sum、lazy、delta、carry、offset

及其中间和差和长度乘积均须在 long long 内；节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根，不要外部改写。

```
struct PersistentRange
{
    using ll = long long;

    struct Node
    {
        int left = 0, right = 0;
        ll sum = 0, lazy = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    explicit PersistentRange(int size) : n(size) { assert(n > 0); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return int(t.size()) - 1;
    }

    void pull(int p, int len)
    {
        t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
    }
```

区间加与版本拼接（接上页同一结构体）：

```
    int change(int p, int l, int r, int ql, int qr, ll value)
    {
        if (qr <= l || r <= ql) return p;
        int u = clone(p);
        if (ql <= l && r <= qr)
        {
            t[u].lazy += value;
            t[u].sum += value * (r - l);
            return u;
        }
        int m = l + (r - l) / 2;
        int a = change(t[p].left, l, m, ql, qr, value);
        int b = change(t[p].right, m, r, ql, qr, value);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }

    int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
    {
        if (qr <= l || r <= ql) return p;
        if (ql <= l && r <= qr)
        {
            if (offset == 0) return q;
            int u = clone(q);
            t[u].lazy += offset;
            t[u].sum += offset * (r - l);
            return u;
        }
        int u = clone(p), m = l + (r - l) / 2;
        offset += t[q].lazy - t[p].lazy;
        int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
        int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }
```

只读查询与对外接口（接上页同一结构体）：

```
    ll get(int p, int l, int r, int ql, int qr, ll carry) const
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
        carry += t[p].lazy;
        int m = l + (r - l) / 2;
        return get(t[p].left, l, m, ql, qr, carry) +
               get(t[p].right, m, r, ql, qr, carry);
    }

    void check(int version, int l, int r) const
    {
        assert(0 <= version && version < int(root.size()));
        assert(0 <= l && l <= r && r <= n);
    }

    int add(int version, int l, int r, ll value)
    {
        check(version, l, r);
        int p = l == r || value == 0 ? root[version]
                                     : change(root[version], 0, n, l, r, value);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    int splice(int destination, int source, int l, int r)
    {
        check(destination, l, r);
        check(source, l, r);
        int p = l == r ? root[destination]
                       : replace(root[destination], root[source], 0, n, l, r, 0);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    ll query(int version, int l, int r) const
    {
        check(version, l, r);
        return l == r ? 0 : get(root[version], 0, n, l, r, 0);
    }
};
```

应用题补充：SPOJ TTM / Luogu SP11470

题意：C 对当前版本区间加并使时间戳加 1；Q 查当前和，H 查指定时间和，B 回退时间；n,m≤100000，初值绝对值 ≤10^9，增量绝对值 ≤10^4。模板存相对初值的增量，初值用前缀和单独累加。ver[t] 映射逻辑时间到追加版本 ID，回退后 C 覆盖 ver[t+1] 而非使用 t 作为模板版本。闭区间转 [l-1,r]，O(n+m log n)。

所需模板：PersistentRange（第 175 页，10.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP11470

SPOJ TTM / Luogu SP11470：使用代码

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    while (cin >> n >> m)
    {
        vector<long long> pre(n + 1);
        for (int i = 1; i <= n; i++)
        {
            cin >> pre[i];
            pre[i] += pre[i - 1];
        }
        PersistentRange tree(n);
        vector<int> ver(m + 1);
        int t = 0;
        while (m--)
        {
            char op;
            cin >> op;
            if (op == 'B')
            {
                cin >> t;
                continue;
            }
            int l, r;
            cin >> l >> r;
            l--;
            if (op == 'C')
            {
                long long d;
                cin >> d;
                int v = tree.add(ver[t], l, r, d);
                ver[++t] = v;
            }
            else
            {
                int h = t;
                if (op == 'H') cin >> h;
                cout << pre[r] - pre[l] + tree.query(ver[h], l, r) << '\n';
            }
        }
    }
    return 0;
}
```

应用题补充：QOJ 8240 Card Game

题意：ICPC 杭州 2023K：给区间牌列，顺序放牌，遇到已有同号牌就连同其后所有牌及新牌一起删除，问最终剩余数。n,q≤300000，牌号 1..n；端点须与上次答案异或解码。version[i] 的坐标 r 表示 [i,r] 答案；下一同号位置 p 前区间加 1，p 起后缀从 version[p+1] 拼接。nodes 仅估算分配上界，最后按右端点单点查。O((n+q)log n)。

所需模板：PersistentRange（第 175 页，10.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：`https://qoj.ac/problem/8240`

QOJ 8240 Card Game：使用代码

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> a(n), next(n), last(n + 1, n), version(n + 1);
    for (auto &x : a) scanf("%d", &x);
    for (int i = n - 1; i >= 0; i--)
    {
        next[i] = last[a[i]];
        last[a[i]] = i;
    }
    PersistentRange tree(n);
    function<size_t(int, int, int, int)> nodes =
        [&](int l, int r, int ql, int qr) -> size_t
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return 1;
        int m = l + (r - l) / 2;
        return 1 + nodes(l, m, ql, qr) + nodes(m, r, ql, qr);
    };
    size_t capacity = 1;
    for (int i = 0; i < n; i++)
    {
        capacity += nodes(0, n, i, next[i]);
        capacity += nodes(0, n, next[i], n);
    }
    tree.t.reserve(capacity);
    tree.root.reserve(2 * n + 1);
    for (int i = n - 1; i >= 0; i--)
    {
        int p = next[i];
        int v = tree.add(version[i + 1], i, p, 1);
        version[i] = p == n ? v : tree.splice(v, version[p + 1], p, n);
    }
    int answer = 0;
    while (q--)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        l ^= answer;
        r ^= answer;
        answer = tree.query(version[l - 1], r - 1, r);
        printf("%d\n", answer);
    }
}
```

10.1.4 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次；1≤k≤ 路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用 root[u]+root[v]-root[lca]-root[parent(lca)]，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理 O(n log n)，每次查询 O(log n)，空间 O(n log n)。递归深度最坏 O(n)，保持递归写法。

```
struct TreePathKth
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n, lg;
    vector<vector<int>> g, up;
    vector<int> root, dep;
    vector<ll> value, vals;
    vector<Node> t;

    TreePathKth(const vector<ll> &a)
        : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
    {
        assert(n > 0);
        lg = 1;
        while ((1LL << lg) <= n) lg++;
        up.assign(lg, vector<int>(n + 1));
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

版本插入与递归建树（接上页同一结构体）：

```
    int insert(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        t[p].sum++;
        if (l == r) return p;
        int m = (l + r) / 2;
        if (x <= m)
        {
            int child = insert(t[old].l, l, m, x);
            t[p].l = child;
        }
        else
        {
            int child = insert(t[old].r, m + 1, r, x);
            t[p].r = child;
        }
        return p;
    }

    void dfs(int u, int p)
    {
        up[0][u] = p;
        for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
        int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
        root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs(v, u);
        }
    }

    void build(int r = 1)
    {
        assert(1 <= r && r <= n);
        vals = value;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        t.assign(1, Node{});
        dep[r] = 0;
        dfs(r, 0);
    }
```

最近公共祖先与路径第 k 小（接上页同一结构体）：

```
    int lca(int u, int v) const
    {
        if (dep[u] < dep[v]) swap(u, v);
        int diff = dep[u] - dep[v];
        for (int j = 0; j < lg; j++)
            if (diff >> j & 1) u = up[j][u];
        if (u == v) return u;
        for (int j = lg - 1; j >= 0; j--)
            if (up[j][u] != up[j][v])
            {
                u = up[j][u];
                v = up[j][v];
            }
        return up[0][u];
    }

    ll kth(int u, int v, int k) const
    {
        int z = lca(u, v);
        assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
        int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];
        int l = 0, r = (int)vals.size() - 1;
        while (l < r)
        {
            int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
            int m = (l + r) / 2;
            if (k <= count)
            {
                a = t[a].l;
                b = t[b].l;
                c = t[c].l;
                d = t[d].l;
                r = m;
            }
            else
            {
                k -= count;
                a = t[a].r;
                b = t[b].r;
                c = t[c].r;
                d = t[d].r;
                l = m + 1;
            }
        }
        return vals[l];
    }
};
```

模板题使用：Luogu P2633

题意：给点权树，查询路径上第 k 小的点权，包含两个端点。构造传入 0-based 权值数组，点号为 1..n；每次先将输入的第一个端点与上次答案异或解码，初始 last=0。

所需模板：TreePathKth（第 183 页，10.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2633

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    TreePathKth tree(a);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u, v);
    }
    tree.build();
    long long last = 0;
    while (m--)
    {
        long long encoded;
        int v, k;
        std::cin >> encoded >> v >> k;
        int u = encoded ^ last;
        last = tree.kth(u, v, k);
        std::cout << last << '\n';
    }
}
```

10.1.5 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组，query(l,r) 返回闭区间内不同值的个数，1≤l≤r≤n。支持重复值与完整 long long 值域；建成后不支持改值，查询顺序任意。令 prev[i] 为同值上次出现的位置，未出现为 0；root[i] 在值域 0..n-1 中统计前 i 个 prev。区间内每种值恰有一次 prev[i]<l，因此答案为 root[r]-root[l-1] 中位置 0..l-1 的频数和。每个元素只复制一条节点路径，预处理 O(n log n)，查询 O(log n)，空间 O(n log n)。动态版预留 n · (ceil(log2(n))+1)+1 个节点，避免扩容峰值；root 和节点池为内部数据。

```
struct PersistentDistinct
{
    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root;

```

```
    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
    {
        assert(n > 0);
        int height = 1;
        while ((1LL << (height - 1)) < n) height++;
        t.reserve(size_t(n) * height + 1);
        vector<long long> vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        vector<int> last(vals.size());
        for (int i = 1; i <= n; i++)
        {
            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
            last[x] = i;
        }
    }

    int insert(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        t[p].sum++;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int child = insert(t[old].l, l, m, x);
                t[p].l = child;
            }
            else
            {
                int child = insert(t[old].r, m + 1, r, x);
                t[p].r = child;
            }
        }
        return p;
    }

    int query(int l, int r) const
    {
        assert(1 <= l && l <= r && r <= n);
        int u = root[l - 1], v = root[r];
        int L = 0, R = n - 1, answer = 0;
        while (L < R)
        {
            int m = (L + R) / 2;
            if (l - 1 <= m)
            {
                u = t[u].l;
                v = t[v].l;
                R = m;
            }
            else
```

```
            {
                answer += t[t[v].l].sum - t[t[u].l].sum;
                u = t[u].r;
                v = t[v].r;
                L = m + 1;
            }
        }
        return answer + t[v].sum - t[u].sum;
    }
};
```

应用题补充：Luogu P1972

题意：给定固定数组，查询闭区间 [l,r] 内不同值的个数。构造传入 0-based vector，查询使用 1-based 闭区间；重复出现的同一值只计一次。

所需模板：PersistentDistinct（第 186 页，10.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1972

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentDistinct tree(a);
    int m;
    std::cin >> m;
    while (m--)
    {
        int l, r;
        std::cin >> l >> r;
        std::cout << tree.query(l, r) << '\n';
    }
}
```

相关应用：来源专用值域平移/历史计数权值索引，不代表整页已覆盖

10.1.6 持久化计数与权值索引

PersistentWeightedIndex(n)，位置域 [0,n)，n 为正 int，版本 0 为空。add(version,pos,weight) 从任意已有版本分叉，新增一个元素，count 加 1、sum 加 weight，返回新版本号；weight 允许负数。query(version,l,r) 返回半开区间的 {count,sum}，空区间合法。first(before,after,key) 返回计数版本差中 >=key 的最小位置，无结果-1；last(before,after,key) 返回严格 <key 的最大位置，无结果-1；二者 key 均可越界。必须保证 after-before 逐位置计数非负，祖先版本与后代版本满足此前提，任意两个分支不一定满足。后继按 count 判断，不按 sum。

每次插入新增 O(log(n+1)) 节点，query/first/last 时间 O(log(n+1))，查询不分配节点。元素计数、版本数和节点数需可用 int 编号；所有节点权值和及查询累加中间结果须适配 signed64。两次 query 相减可得区间版本差，但减法也需防溢出。内部根与辅助函数不可外部修改，对象复制独立。

```
struct PersistentWeightedIndex
{
    using ll = long long;
    struct Node
    {
        int left = 0, right = 0, count = 0;
        ll sum = 0;
    };
    int n;
    vector<Node> t{Node{}};
    vector<int> roots{0};

    PersistentWeightedIndex(int n) : n(n)
    {
        assert(n > 0);
    }

    int insert(int old, int l, int r, int pos, ll weight)
    {
        assert(t.size() < INT_MAX && t[old].count < INT_MAX);
        int u = t.size();
        Node copy = t[old];
        copy.count++;
        copy.sum += weight;
        t.push_back(copy);
        if (r - l == 1)
            return u;
        int m = l + (r - l) / 2;
        if (pos < m)
            t[u].left = insert(t[u].left, l, m, pos, weight);
        else
            t[u].right = insert(t[u].right, m, r, pos, weight);
        return u;
    }
```

接上页同一结构体：

```
    int add(int version, int pos, ll weight)
    {
        assert(0 <= version && version < (int)roots.size());
        assert(0 <= pos && pos < n && roots.size() < INT_MAX);
        int root = insert(roots[version], 0, n, pos, weight);
        roots.push_back(root);
        return (int)roots.size() - 1;
    }

    pair<int, ll> query_node(int u, int l, int r, int ql, int qr) const
    {
        if (!u || qr <= l || r <= ql)
            return {0, 0};
        if (ql <= l && r <= qr)
            return {t[u].count, t[u].sum};
        int m = l + (r - l) / 2;
        auto a = query_node(t[u].left, l, m, ql, qr);
        auto b = query_node(t[u].right, m, r, ql, qr);
        return {a.first + b.first, a.second + b.second};
    }

    pair<int, ll> query(int version, int l, int r) const
    {
        assert(0 <= version && version < (int)roots.size());
        assert(0 <= l && l <= r && r <= n);
        return query_node(roots[version], 0, n, l, r);
    }

    int find(int before, int after, int l, int r, int key) const
    {
        if (r <= key || t[after].count == t[before].count)
            return -1;
        if (r - l == 1)
            return l;
        int m = l + (r - l) / 2;
        int answer = find(t[before].left, t[after].left, l, m, key);
        if (answer == -1)
            answer = find(t[before].right, t[after].right, m, r, key);
        return answer;
    }

    // The after-before count at every position must be nonnegative.
    int first(int before, int after, int key) const
    {
        assert(0 <= before && before < (int)roots.size());
        assert(0 <= after && after < (int)roots.size());
        return find(roots[before], roots[after], 0, n, key);
    }

    int find_last(int before, int after, int l, int r, int key) const
    {
        if (l >= key || t[after].count == t[before].count)
            return -1;
        if (r - l == 1)
```

```
            return l;
        int m = l + (r - l) / 2;
        int answer = find_last(t[before].right, t[after].right, m, r, key);
        if (answer == -1)
            answer = find_last(t[before].left, t[after].left, l, m, key);
        return answer;
    }

    // Strict predecessor; the same nonnegative count-difference contract applies.
    int last(int before, int after, int key) const
    {
        assert(0 <= before && before < (int)roots.size());
        assert(0 <= after && after < (int)roots.size());
        return find_last(roots[before], roots[after], 0, n, key);
    }
};
```

接口用法演示：持久化计数与权值索引 API

演示约定：输入 n q，位置域 [0,n)、初始版本 0 为空。1 version pos weight 从该版本新增元素并输出新版本号；2 version l r 输出 [l,r) 内 count 和 sum；3 before after key 输出版本差中 >=key 的首位置或-1，4 before after key 输出版本差中严格 <key 的末位置或-1；操作 3/4 要求逐位置计数差非负。旧版本不变，允许分叉和负权。本地 API 例子，不是 OJ 判定。

所需模板：PersistentWeightedIndex（第 189 页，10.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/gsh20040816/CPCtemplate/blob/main/docs/WIDA-SHIFT-SPLAY.md

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    PersistentWeightedIndex tree(n);
    while (q--)
    {
        int op, version;
        cin >> op >> version;
        if (op == 1)
        {
            int pos;
            long long weight;
            cin >> pos >> weight;
            cout << tree.add(version, pos, weight) << '\n';
        }
        else if (op == 2)
        {
            int l, r;
            cin >> l >> r;
            auto [count, sum] = tree.query(version, l, r);
            cout << count << ' ' << sum << '\n';
        }
        else
        {
            int after, key;
            cin >> after >> key;
            if (op == 3)
                cout << tree.first(version, after, key) << '\n';
            else
                cout << tree.last(version, after, key) << '\n';
        }
    }
}
```

模板题使用：Library Checker Rectangle Sum

题意：N 个带权点，Q 次查询 [l,r)×[d,u) 内权值和；1<=N,Q<=200000，坐标及权值在 [0,1e9]。按 x 排序建立前缀版本，y 离散化；两个版本相减，重复点与零权合法，答案用 signed64。预处理 O(NlogN)，每次查询 O(logN)，空间 O(NlogN)。Library Checker C++20 线上 AC 410009，1072ms、101.38MiB。

所需模板：PersistentWeightedIndex（第 189 页，10.1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/rectangle_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<array<int, 3>> a(n);
    vector<int> xs, ys;
    for (auto &[x, y, w] : a)
    {
        cin >> x >> y >> w;
        ys.push_back(y);
    }
    sort(a.begin(), a.end());
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    auto rank = [&](int y)
    {
        return lower_bound(ys.begin(), ys.end(), y) - ys.begin();
    };
    PersistentWeightedIndex tree(ys.size());
    for (int i = 0; i < n; i++)
    {
        auto [x, y, w] = a[i];
        xs.push_back(x);
        tree.add(i, rank(y), w);
    }
    while (q--)
    {
        int l, d, r, u;
        cin >> l >> d >> r >> u;
        int before = lower_bound(xs.begin(), xs.end(), l) - xs.begin();
        int after = lower_bound(xs.begin(), xs.end(), r) - xs.begin();
        long long answer = tree.query(after, rank(d), rank(u)).second;
        answer -= tree.query(before, rank(d), rank(u)).second;
        cout << answer << '\n';
    }
}
```

相关应用：按原始整数奇偶维护历史重数及专用降序选择，不代表整页覆盖

10.1.7 持久化奇偶计数与降序选择

PersistentParityIndex(n)，原始整数位置域 [0,n)，n 为正 int，版本 0 为空。add(version,x,delta) 从任意版本分叉，位置 x 的重数加 signed64 增量 delta，返回新版本号；要求结果重数非负、版本总重数 <=LLONG_MAX。delta=0 也创建版本但不分配节点。奇偶性按原始 x 判断，不能直接把数值离散化后的编号当原值。

difference(before,after) 返回 {总重数差, 奇数位置重数差}，允许有符号差。以下两个查询要求每个位置 after-before 重数非负；有删除操作时，即使祖先与后代版本也不自动满足。stop(before,after,k) 从大到小删除至多 k 个偶数元素，遇到奇数立即停止，返回剩余最大位置，无元素返回-1，查询不改版本。odd_kth(before,after,k) 只按奇数位置元素降序选择 0-based 第 k 项，返回 {位置, 该位置总重数, 在该位置内的 0-based 偏移}；不存在返回 {-1,0,k-奇数总重数}。k 为非负 signed64，位置 0 合法。

每次非零更新 O(log(n+1)) 时间和新增节点；difference 为 O(1)，其余查询 O(log(n+1)) 且不分配节点。总空间 O(版本数 + 更新数 log(n+1))，节点不回收，节点和版本编号须适合 int；递归深度 O(log(n+1))。t/roots 及递归辅助函数仅供内部实现；完整对象复制独立。原稿无结果的 0 哨兵由驱动按需映射，不能把 0 当本接口的空结果。

```
struct PersistentParityIndex
{
    using ll = long long;
    struct Node
    {
        int left = 0, right = 0;
        ll count = 0, odd = 0;
    };
    int n;
    vector<Node> t{Node{}};
    vector<int> roots{0};

    PersistentParityIndex(int n) : n(n)
    {
        assert(n > 0);
    }

    int update(int p, int l, int r, int x, ll delta)
    {
        assert(t.size() < INT_MAX);
        int u = t.size();
        Node node = t[p];
        t.push_back(node);
        if (r - l == 1)
        {
            assert(delta >= -node.count);
            t[u].count += delta;
            t[u].odd = (x % 2) * t[u].count;
            return u;
        }
        int m = l + (r - l) / 2;
        if (x < m)
            t[u].left = update(node.left, l, m, x, delta);
        else
            t[u].right = update(node.right, m, r, x, delta);
        t[u].count = t[t[u].left].count + t[t[u].right].count;
        t[u].odd = t[t[u].left].odd + t[t[u].right].odd;
        return u;
    }

    int add(int version, int x, ll delta)
    {
        assert(0 <= version && version < (int)roots.size());
        assert(0 <= x && x < n && roots.size() < INT_MAX);
        int p = roots[version];
```

```
        assert(delta <= 0 || t[p].count <= LLONG_MAX - delta);
        int root = delta == 0 ? p : update(p, 0, n, x, delta);
        roots.push_back(root);
        return (int)roots.size() - 1;
    }

    array<ll, 2> difference(int before, int after) const
    {
        assert(0 <= before && before < (int)roots.size());
        assert(0 <= after && after < (int)roots.size());
        auto a = t[roots[before]], b = t[roots[after]];
        return {b.count - a.count, b.odd - a.odd};
    }

    int stop_node(int a, int b, int l, int r, ll k) const
    {
        if (r - l == 1)
            return l;
        int m = l + (r - l) / 2;
        int x = t[a].right, y = t[b].right;
        ll count = t[y].count - t[x].count;
        ll odd = t[y].odd - t[x].odd;
        if (odd > 0 || count > k)
            return stop_node(x, y, m, r, k);
        return stop_node(t[a].left, t[b].left, l, m, k - count);
    }

    // Queries require nonnegative after-before counts at every position.
    // Remove up to k largest even elements, stopping at the first odd element.
    int stop(int before, int after, ll k) const
    {
        assert(k >= 0);
        auto [count, odd] = difference(before, after);
        if (odd == 0 && count <= k)
            return -1;
        return stop_node(roots[before], roots[after], 0, n, k);
    }

    array<ll, 3> odd_node(int a, int b, int l, int r, ll k) const
    {
        if (r - l == 1)
            return {l, t[b].odd - t[a].odd, k};
        int m = l + (r - l) / 2;
        int x = t[a].right, y = t[b].right;
        ll count = t[y].odd - t[x].odd;
        if (count > k)
            return odd_node(x, y, m, r, k);
        return odd_node(t[a].left, t[b].left, l, m, k - count);
    }
```

```
95
96      // Descending zero-based rank; result is {position, multiplicity, offset}.
97      array<ll, 3> odd_kth(int before, int after, ll k) const
98      {
99          assert(k >= 0);
100         auto [count, odd] = difference(before, after);
101         if (k >= odd)
102             return {-1, 0, k - odd};
103         return odd_node(roots[before], roots[after], 0, n, k);
104     }
105 };
```

接口用法演示：持久化奇偶计数与降序选择 API

演示约定：读 n q，初始版本 0。1 v x delta 在 v 上增减位置 x 重数，输出新版本号；2 before after k 输出 stop；3 before after k 输出 odd_kth 三元组；4 before after 输出总重数差与奇数重数差。操作 2/3 要求逐位置版本差非负，k>=0；原值域 [0,n)，下标 0-based，重数与 k 使用 signed64。自定义 API 协议，非 OJ 判定。

所需模板：PersistentParityIndex（第 193 页，10.1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    PersistentParityIndex tree(n);
    while (q--)
    {
        int op, a, b;
        cin >> op >> a >> b;
        if (op == 1)
        {
            long long delta;
            cin >> delta;
            cout << tree.add(a, b, delta) << '\n';
        }
        else if (op == 2)
        {
            long long k;
            cin >> k;
            cout << tree.stop(a, b, k) << '\n';
        }
        else if (op == 3)
        {
            long long k;
            cin >> k;
            auto result = tree.odd_kth(a, b, k);
            cout << result[0] << ' ' << result[1] << ' ' << result[2] << '\n';
        }
        else
        {
            auto result = tree.difference(a, b);
            cout << result[0] << ' ' << result[1] << '\n';
        }
    }
}
```

10.1.8 可持久化并查集

PersistentDSU(n)，n>0，点号 0..n-1，版本 0 为单点集合。merge(ver,x,y) 从任意版本合并并返回新版本号，已连通也创建版本；copy(ver) 复制版本。find、same、size 只读，不压缩路径；按大小合并，等大时第一根挂到第二根。区别于普通 dsu 的固定方向合并。稀疏可持久化线段树的叶子存父亲或负大小，空节点表示-1。初始化 O(1)，copy 均摊 O(1)，其余 $O(\log^2(n+1))$；成功合并增加 O(log(n+1)) 节点。节点数/版本数须可用 int 索引。root、t 只读，get/set 内部函数勿直接调用；旧版本永不被改写。

```
struct PersistentDSU
{
    struct Node
    {
        int l = 0, r = 0, v = -1;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentDSU(int n) : n(n)
    {
        assert(n > 0);
    }

    int get(int p, int l, int r, int x) const
    {
        if (!p) return -1;
        if (r - l == 1) return t[p].v;
        int m = l + (r - l) / 2;
        if (x < m) return get(t[p].l, l, m, x);
        return get(t[p].r, m, r, x);
    }

    int set(int old, int l, int r, int x, int v)
    {
        Node node = t[old];
        int p = t.size();
        t.push_back(node);
        if (r - l == 1)
            t[p].v = v;
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = set(node.l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = set(node.r, m, r, x, v);
                t[p].r = q;
            }
        }
        return p;
    }
```

可持久化并查集（接上页同一结构体）：

```
    // Queries never compress paths or allocate nodes. Vertices are 0-based.
    int find(int ver, int x) const
    {
        assert(0 <= ver && ver < (int)root.size());
        assert(0 <= x && x < n);
        int p = get(root[ver], 0, n, x);
        while (p >= 0)
        {
            x = p;
            p = get(root[ver], 0, n, x);
        }
        return x;
    }

    int size(int ver, int x) const
    {
        return -get(root[ver], 0, n, find(ver, x));
    }

    bool same(int ver, int x, int y) const
    {
        return find(ver, x) == find(ver, y);
    }

    int copy(int ver)
    {
        assert(0 <= ver && ver < (int)root.size());
        root.push_back(root[ver]);
        return (int)root.size() - 1;
    }

    // Always create one version; ties attach the first root to the second.
    int merge(int ver, int x, int y)
    {
        x = find(ver, x);
        y = find(ver, y);
        if (x == y) return copy(ver);
        int a = get(root[ver], 0, n, x);
        int b = get(root[ver], 0, n, y);
        if (a < b)
        {
            swap(x, y);
            swap(a, b);
        }
        int p = set(root[ver], 0, n, x, y);
        p = set(p, 0, n, y, a + b);
        root.push_back(p);
        return (int)root.size() - 1;
    }
};
```

模板题使用：Luogu P3402 / 可持久化并查集

题意：n≤100000，m≤200000。1 a b 合并；2 k 回到第 k 次操作后的状态，可回到 0；3 a b 输出是否连通。每种操作均产生版本，操作号等于版本号，点号转 0-based，无异或。

所需模板：PersistentDSU（第 198 页，10.1.8 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3402

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    PersistentDSU d(n);
    for (int i = 1; i <= m; i++)
    {
        int op, x, y;
        cin >> op >> x;
        if (op == 2)
            d.copy(x);
        else
        {
            cin >> y;
            if (op == 1)
                d.merge(i - 1, x - 1, y - 1);
            else
            {
                cout << d.same(i - 1, x - 1, y - 1) << '\n';
                d.copy(i - 1);
            }
        }
    }
}
```

10.2 可持久化平衡树

10.2.1 可持久化有序 Treap：分叉版本与顺序统计

版本 0 为空。insert(v,x)、erase(v,x)、copy(v) 追加并返回新版本；删除不存在的值也追加相同集合的版本。size/less/rank/kth/prev/next 均显式传版本且只读，不创建版本或树节点。重复值计数，rank 为严格小于 x 的副本数加一；kth 为 1-based 且要求合法，严格前后继缺失返回 nullopt。支持 long long 全范围，不用 x±1。随机优先级下期望 O(log(d+1))，d 为版本不同键数；copy 摊还 O(1)，没有最坏对数保证。递归复制更新路径，节点不回收，空间按全部历史分配节点与版本数计。节点/版本/各版副本数须小于 INT_MAX；t、root 及带节点编号的辅助函数属内部状态。当前常见 64 位布局每节点 32 字节；500000 次随机不同键插入实测约 1200 万节点，不预留的 vector 扩容可能超过 512MiB 峰值。预留 26*n+1 仅是明确预算而非上界，先核对内存限制；重新构造清空全部版本。

```
struct PersistentOrderedTreap
{
    using ll = long long;

    struct Node
    {
        ll val = 0;
        unsigned long long pri = 0;
        int l = 0, r = 0, siz = 0, cnt = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    mt19937_64 rng;

    PersistentOrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void check(int v) const { assert(0 <= v && v < (int)root.size()); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return (int)t.size() - 1;
    }

    void pull(int p) { t[p].siz = t[t[p].l].siz + t[t[p].r].siz + t[p].cnt; }
```

复制路径插入与旋转（接上页同一结构体）：

```
    // Only rotate freshly copied nodes; the promoted child was copied by add.
    int add(int p, ll x)
    {
        if (!p)
        {
            t.push_back({x, rng(), 0, 0, 1, 1});
            return (int)t.size() - 1;
        }
        int u = clone(p);
        if (x == t[u].val)
            t[u].cnt++;
        else if (x < t[u].val)
        {
            int q = add(t[u].l, x);
            t[u].l = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].l = t[q].r;
                t[q].r = u;
                pull(u);
                u = q;
            }
        }
        else
        {
            int q = add(t[u].r, x);
            t[u].r = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].r = t[q].l;
                t[q].l = u;
                pull(u);
                u = q;
            }
        }
        pull(u);
        return u;
    }
```

复制路径合并与删除（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (t[l].pri > t[r].pri)
        {
            int u = clone(l);
            int q = merge(t[u].r, r);
            t[u].r = q;
            pull(u);
            return u;
        }
        int u = clone(r);
        int q = merge(l, t[u].l);
        t[u].l = q;
        pull(u);
        return u;
    }

    int remove(int p, ll x)
    {
        if (!p) return 0;
        if (x == t[p].val)
        {
            if (t[p].cnt == 1) return merge(t[p].l, t[p].r);
            int u = clone(p);
            t[u].cnt--;
            pull(u);
            return u;
        }
        bool left = x < t[p].val;
        int old = left ? t[p].l : t[p].r;
        int q = remove(old, x);
        if (q == old) return p;
        int u = clone(p);
        if (left)
            t[u].l = q;
        else
            t[u].r = q;
        pull(u);
        return u;
    }
```

版本接口、大小与排名（接上页同一结构体）：

```
    // Updates append versions; erase removes one occurrence, or changes nothing.
    int insert(int v, ll x)
    {
        check(v);
        int p = add(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int erase(int v, ll x)
    {
        check(v);
        int p = remove(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int copy(int v)
    {
        check(v);
        root.push_back(root[v]);
        return (int)root.size() - 1;
    }

    int size(int v) const
    {
        check(v);
        return t[root[v]].siz;
    }

    int less(int v, ll x, bool equal = false) const
    {
        check(v);
        int p = root[v], ans = 0;
        while (p)
        {
            if (t[p].val < x || (equal && t[p].val == x))
            {
                ans += t[t[p].l].siz + t[p].cnt;
                p = t[p].r;
            }
            else
                p = t[p].l;
        }
        return ans;
    }

    int rank(int v, ll x) const { return less(v, x) + 1; }
```

第 k 小与严格前驱后继（接上页同一结构体）：

```
    // Ranks and k are 1-based. No sentinel values are reserved.
    ll kth(int v, int k) const
    {
        assert(1 <= k && k <= size(v));
        int p = root[v];
        while (true)
        {
            int left = t[t[p].l].siz;
            if (k <= left)
                p = t[p].l;
            else if (k <= left + t[p].cnt)
                return t[p].val;
            else
            {
                k -= left + t[p].cnt;
                p = t[p].r;
            }
        }
    }

    optional<ll> prev(int v, ll x) const
    {
        int k = less(v, x);
        if (!k) return nullopt;
        return kth(v, k);
    }

    optional<ll> next(int v, ll x) const
    {
        int k = less(v, x, true);
        if (k == size(v)) return nullopt;
        return kth(v, k + 1);
    }
};
```

模板题使用：Luogu P3835 【模板】可持久化平衡树

题意：维护可重整数集合的历史版本；每条插入、删除或排名/第 k 小/严格前后继查询都从指定旧版本生成版本 i。删除缺失值忽略但仍产生版本；查询输出后 copy(v)，避免版本编号错位。第 k 小保证合法；缺失前后继分别输出-2147483647、2147483647，仅由驱动映射 optional。n≤500000、键绝对值 ≤10^9。reserve(26*n+1) 是约 32 字节节点的预留预算，不是分配上界；最大 n 预留约 397MiB，需先核对内存限制，不能将本地通过当作线上 AC。

所需模板：PersistentOrderedTreap（第 201 页，10.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3835

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    PersistentOrderedTreap tree;
    tree.root.reserve(n + 1);
    tree.t.reserve(26ULL * n + 1);
    for (int i = 1; i <= n; i++)
    {
        int v, op;
        long long x;
        cin >> v >> op >> x;
        if (op == 1)
            tree.insert(v, x);
        else if (op == 2)
            tree.erase(v, x);
        else
        {
            if (op == 3) cout << tree.rank(v, x) << '\n';
            if (op == 4) cout << tree.kth(v, (int)x) << '\n';
            if (op == 5) cout << tree.prev(v, x).value_or(-2147483647LL) << '\n';
            if (op == 6) cout << tree.next(v, x).value_or(2147483647LL) << '\n';
            tree.copy(v);
        }
    }
}
```

10.3 可持久化字典树

10.3.1 前缀可持久化 01 字典树

PersistentXorTrie(bits,cap)，1≤bits≤64，0≤cap≤(INT_MAX−1)/(bits+1)，构造时预留最多 cap 个键。append(x) 追加一个 uint64_t 原值，重复值按出现次数保留；满容量或 x 不满足位宽时返回 false 且逻辑状态不变。size() 为键数；max_xor(l,r,x) 从 0-based 半开区间 [l,r) 选择一个键 y，返回最大 x xor y，空区间 nullopt。查询 x 允许完整 uint64；不是子集异或、任意分支版本或字符串 Trie。构造 O(cap*bits) 容量空间，追加/查询 O(bits)；复制后首次追加为恢复预留空间可花 O(size*bits)。append 分配失败传播异常，逻辑内容不变；构造参数非法抛异常，查询下标是调用者前置条件，assert 只作调试检查。内部成员只读；不要复用已移动对象。

```
struct PersistentXorTrie
{
    using U = uint64_t;
    struct Node
    {
        int ch[2] = {}, cnt = 0;
    };
    int bits, cap;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentXorTrie(int bits, int cap) : bits(bits), cap(cap)
    {
        if (bits < 1 || bits > 64 || cap < 0)
            throw invalid_argument("PersistentXorTrie domain");
        if (cap > (INT_MAX - 1) / (bits + 1))
            throw length_error("PersistentXorTrie capacity");
        reserve();
    }

    void reserve()
    {
        t.reserve(1 + size_t(cap) * (bits + 1));
        root.reserve(size_t(cap) + 1);
    }

    int size() const { return (int)root.size() - 1; }
```

前缀追加与区间单值最大异或（接上页同一结构体）：

```
    bool append(U x)
    {
        if (size() == cap || (bits < 64 && (x >> bits))) return false;
        reserve(); // A copied vector need not preserve its original capacity.
        int old = root.back(), p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        root.push_back(p);
        t[p].cnt++;
        for (int d = bits - 1; d >= 0; d--)
        {
            int b = x >> d & 1, q = t.size();
            old = t[old].ch[b];
            copy = t[old];
            t.push_back(copy);
            t[p].ch[b] = q;
            p = q;
            t[p].cnt++;
        }
        return true;
    }

    optional<U> max_xor(int l, int r, U x) const
    {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return nullopt;
        int a = root[l], b = root[r];
        U y = 0;
        for (int d = bits - 1; d >= 0; d--)
        {
            int k = (x >> d & 1) ^ 1;
            if (t[t[b].ch[k]].cnt == t[t[a].ch[k]].cnt) k ^= 1;
            y |= U(k) << d;
            a = t[a].ch[k];
            b = t[b].ch[k];
        }
        return x ^ y;
    }
};
```

应用题补充：Luogu P4735 最大异或和

题意：N,M≤300000，序列元素 0..10^7；追加 A x 或查询 Q l r x，1≤l≤r≤ 当前 N。令 S 为前缀异或，答案为 (S_N xor x) 与 S_[l−1,r−1] 中一个数异或的最大值。先插 S0，再插后续前缀，调用 max_xor(l−1,r,S_N xor x)。前缀可能超过 10^7 但小于 2^24；库储存 24 位，查询 x 按 uint64_t 读取并保留高位，题面未单列查询 x 上界。此为组合应用；最多 600001 个键，12 字节节点平台数组负载约 174MiB，不含进程开销，不声称通过 OJ 时间限制。

所需模板：PersistentXorTrie（第 207 页，10.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4735

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    PersistentXorTrie trie(24, n + m + 1);
    uint64_t sum = 0, x;
    trie.append(0);
    for (int i = 0; i < n; i++)
    {
        cin >> x;
        sum ^= x;
        trie.append(sum);
    }
    while (m--)
    {
        char op;
        cin >> op;
        if (op == 'A')
        {
            cin >> x;
            sum ^= x;
            trie.append(sum);
        }
        else
        {
            int l, r;
            cin >> l >> r >> x;
            cout << *trie.max_xor(l - 1, r, sum ^ x) << '\n';
        }
    }
}
```

第十一章　树套树

11.1　树状数组套权值线段树

11.1.1　动态区间第 k 小（树状数组套权值线段树）

DynamicKth(a,future) 传入非空 0-based 初值，future 包含所有将来可能赋入的新值；内部自动合并初值并离散化。set(pos,value) 单点赋值，kth(l,r,k) 查询闭区间第 k 小，下标均为 1-based；1≤k≤r-l+1。允许重复值与完整 long long 权值；新值必须已在离散化集合内，操作可按原顺序执行，不保留历史版本。每个树状数组节点对应一棵可变权值线段树；删除旧值后插入新值。空节点回收到 free_head 链，重复修改不会按操作次数累计泄漏节点。设 D 为不同值数，S=n+future.size()，预处理 O(S log S+n log n log(D+1))，单次修改及查询 O(log n log(D+1))。节点数上界 B=(ceil(log2(D))+1) · $\sum$ lowbit(i)，求和 i=1..n；空间 O(S+n+B)。update、modify、id 和节点池均为内部接口；不能手动减去不存在的值。

```
struct DynamicKth
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n, free_head = 0;
    vector<int> bit, rank;
    vector<ll> vals;
    vector<Node> t{Node{}};

    DynamicKth(const vector<ll> &a, vector<ll> future = {})
        : n(a.size()), bit(n + 1), rank(n + 1)
    {
        assert(n > 0);
        vals = future;
        vals.insert(vals.end(), a.begin(), a.end());
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (int i = 1; i <= n; i++)
        {
            rank[i] = id(a[i - 1]);
            modify(i, rank[i], 1);
        }
    }

    int id(ll value) const
    {
        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
        assert(p < (int)vals.size() && vals[p] == value);
        return p;
    }

    int new_node()
    {
```

```
        if (!free_head)
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free_head;
        free_head = t[p].l;
        t[p] = Node{};
        return p;
    }

    int update(int p, int l, int r, int x, int delta)
    {
        if (!p) p = new_node();
        t[p].sum += delta;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int child = update(t[p].l, l, m, x, delta);
                t[p].l = child;
            }
            else
            {
                int child = update(t[p].r, m + 1, r, x, delta);
                t[p].r = child;
            }
        }
        if (!t[p].sum)
        {
            t[p].l = free_head;
            free_head = p;
            return 0;
        }
        return p;
    }

    void modify(int pos, int x, int delta)
    {
        for (int i = pos; i <= n; i += i & -i)
            bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
    }
```

```
    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        int x = id(value);
        if (rank[pos] == x) return;
        modify(pos, rank[pos], -1);
        modify(pos, x, 1);
        rank[pos] = x;
    }

    ll kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
        vector<int> a, b;
        for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
        for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
        int L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int count = 0;
            for (int p : a) count += t[t[p].l].sum;
            for (int p : b) count -= t[t[p].l].sum;
            bool left = k <= count;
            if (!left) k -= count;
            for (int &p : a) p = left ? t[p].l : t[p].r;
            for (int &p : b) p = left ? t[p].l : t[p].r;
            int m = (L + R) / 2;
            if (left)
                R = m;
            else
                L = m + 1;
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P2617

题意：支持单点赋值及闭区间第 k 小。先读完操作，把所有修改的新值收集到 future，再构造并按原顺序执行；初值 vector 为 0-based，位置、查询区间和 k 为 1-based。Query 仅保存输入字段。

所需模板：DynamicKth（第 210 页，11.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2617

```
int main()
{
    struct Query
    {
        char op;
        int x, y, k;
        long long value;
    };

    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n), future;
    for (auto &x : a) std::cin >> x;
    std::vector<Query> queries(m);
    for (auto &q : queries)
    {
        std::cin >> q.op;
        if (q.op == 'Q')
            std::cin >> q.x >> q.y >> q.k;
        else
        {
            std::cin >> q.x >> q.value;
            future.push_back(q.value);
        }
    }
    DynamicKth tree(a, future);
    for (const auto &q : queries)
    {
        if (q.op == 'Q')
            std::cout << tree.kth(q.x, q.y, q.k) << '\n';
        else
            tree.set(q.x, q.value);
    }
}
```

第十二章　K-D Tree

12.1　动态二维 K-D Tree：单点累加与矩形和

KDTreeSum<Coord=int>(reserve_points=0) 初始为空；Coord 取 int 或 long long。add(x,y,delta) 累加有符号 long long 权值，相同坐标只占一个节点；零权节点仍保留，不支持删除坐标。query(x1,y1,x2,y2) 查询闭矩形和，左下端超过右上端则返回 0。坐标只比较，不相减，不用有限哨兵；可用所选整数类型极值。当前各点绝对权值总和及更新中间量须在 long long 内。不同点数 d、预留容量和节点编号须小于 INT_MAX。按轴交替分割，子树过重时递归收集编号并原位重建；树高 O(log(d+1))。nth_element 按平均线性选择计，插入摊还 $O(\log^2(d+1))$，单次大重建可 O(d log(d+1))；矩形查询最坏 O(d)，不能保证对数。空间 O(d)，预留容量另计，默认 int 节点常见为 56 字节；紧内存题预留最大不同点数，避免扩容峰值。a/root 及带节点编号的函数均为内部状态；可重新赋值空对象重置。

```
template <class Coord = int> struct KDTreeSum
{
    using ll = long long;
    using Point = array<Coord, 2>;

    struct Node
    {
        Point point{}, low{}, high{};
        array<int, 2> child{};
        int size = 1;
        ll value = 0, sum = 0;
    };

    vector<Node> a;
    int root = 0;

    KDTreeSum(int reserve_points = 0)
    {
        a.reserve(reserve_points + 1);
        a.push_back(Node{});
        a[0].size = 0;
    }

    bool less(Point x, Point y, int axis) const
    {
        if (x[axis] != y[axis]) return x[axis] < y[axis];
        return x[axis ^ 1] < y[axis ^ 1];
    }
```

包围盒维护、递归收集与重建（接上页同一结构体）：

```
    void pull(int p)
    {
        a[p].low = a[p].high = a[p].point;
        a[p].size = 1;
        a[p].sum = a[p].value;
        for (int q : a[p].child)
        {
            if (!q) continue;
            a[p].size += a[q].size;
            a[p].sum += a[q].sum;
            for (int d = 0; d < 2; d++)
            {
                a[p].low[d] = min(a[p].low[d], a[q].low[d]);
                a[p].high[d] = max(a[p].high[d], a[q].high[d]);
            }
        }
    }

    void collect(int p, vector<int> &v)
    {
        if (!p) return;
        collect(a[p].child[0], v);
        v.push_back(p);
        collect(a[p].child[1], v);
    }

    int build(vector<int> &v, int l, int r, int axis)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2;
        nth_element(v.begin() + l,
                    v.begin() + m,
                    v.begin() + r,
                    [&](int x, int y)
                    {
                        return less(a[x].point, a[y].point, axis);
                    });
        int p = v[m];
        a[p].child[0] = build(v, l, m, axis ^ 1);
        a[p].child[1] = build(v, m + 1, r, axis ^ 1);
        pull(p);
        return p;
    }
```

点权累加与闭矩形查询（接上页同一结构体）：

```
    int add(int p, Point point, ll delta, int axis)
    {
        if (!p)
        {
            p = a.size();
            a.push_back(Node{});
            a[p].point = a[p].low = a[p].high = point;
            a[p].value = a[p].sum = delta;
            return p;
        }
        if (a[p].point == point)
            a[p].value += delta;
        else
        {
            int side = !less(point, a[p].point, axis);
            a[p].child[side] = add(a[p].child[side], point, delta, axis ^ 1);
        }
        pull(p);
        int heavy = max(a[a[p].child[0]].size, a[a[p].child[1]].size);
        if (4LL * heavy <= 3LL * a[p].size) return p;
        vector<int> v;
        v.reserve(a[p].size);
        collect(p, v);
        return build(v, 0, v.size(), axis);
    }

    void add(Coord x, Coord y, ll delta)
    {
        root = add(root, {x, y}, delta, 0);
    }

    ll query(int p, Point low, Point high) const
    {
        if (!p) return 0;
        bool covered = true, inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (a[p].high[d] < low[d] || high[d] < a[p].low[d]) return 0;
            covered &= low[d] <= a[p].low[d] && a[p].high[d] <= high[d];
            inside &= low[d] <= a[p].point[d] && a[p].point[d] <= high[d];
        }
        if (covered) return a[p].sum;
        ll answer = inside ? a[p].value : 0;
        for (int q : a[p].child) answer += query(q, low, high);
        return answer;
    }

    ll query(Coord x1, Coord y1, Coord x2, Coord y2) const
    {
        if (x1 > x2 || y1 > y2) return 0;
        return query(root, {x1, y1}, {x2, y2});
    }
};
```

应用题补充：Luogu P4148 简单题

题意：在线单点累加与闭矩形求和，每条命令除操作码外都异或上次答案，3 结束。答案保证在 int 范围内，因此 last 使用 int。预留 200000 个不同点，减少 vector 扩容峰值；本地验证不等于通过该题 20 MB 平台内存限制。

所需模板：KDTreeSum（第 214 页，12.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4148

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, op, last = 0;
    cin >> n;
    KDTreeSum<> s(200000);
    while (cin >> op && op != 3)
    {
        int x, y;
        cin >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 1)
        {
            int value;
            cin >> value;
            value ^= last;
            s.add(x, y, value);
        }
        else
        {
            int x2, y2;
            cin >> x2 >> y2;
            x2 ^= last;
            y2 ^= last;
            last = s.query(x, y, x2, y2);
            cout << last << '\n';
        }
    }
}
```

接口用法演示：动态二维偏序计数（WIDA 接口适配）

演示约定：输入操作数 m。操作 1 x y 插入一个二维整数点，允许重复；操作 2 x y 统计已经插入且两坐标分别 ≤x、≤y 的点数。每次插入权值 1，查询从 INT_MIN 到 (x,y) 的闭矩形，边界坐标也计入，不对极小值减 1。

所需模板：KDTreeSum（第 214 页，12.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    KDTreeSum<> tree(m);
    while (m--)
    {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1) tree.add(x, y, 1);
        else cout << tree.query(INT_MIN, INT_MIN, x, y) << '\n';
    }
}
```

12.2 静态 K-D Tree 最近与最远点

KDNearest<D>(points)，D>0，Point=array<int,D>，坐标为有符号 32 位整数，输入编号 0..n-1 且允许重合点。query(q,k,far=false) 返回最近 k 个原编号；far=true 改为最远 k 个。结果按距离优先、等距编号小者优先排列，0≤k≤n；k=0 返回空，包含与查询重合的点，不自动排除自身。平方距离用 128 位整数精确计算。p 保留输入顺序；p/a/root 仅供读取，重建请重新赋值。递归中位数建树，平均 O(Dn log(n+1))，空间 O(Dn)；包围盒剪枝没有对任意数据的次线性保证，单次最坏 O(Dn+n log(k+1))，额外 O(k+log(n+1)) 空间。search/build/bound 为内部辅助，不直接调用。

```
template <int D = 2> struct KDNearest
{
    static_assert(D > 0);
    using Point = array<int, D>;
    using Wide = __int128_t;
    using Key = pair<Wide, int>;

    struct Node
    {
        Point low, high;
        array<int, 2> child{-1, -1};
        int first;
    };

    vector<Point> p;
    vector<Node> a;
    int root;

    KDNearest(const vector<Point> &points = {}) : p(points), a(p.size())
    {
        vector<int> order(p.size());
        iota(order.begin(), order.end(), 0);
        root = build(order, 0, order.size(), 0);
    }
```

中位数建树与距离界（接上页同一结构体）：

```
    int build(vector<int> &order, int l, int r, int axis)
    {
        if (l == r) return -1;
        int m = (l + r) / 2;
        nth_element(order.begin() + l, order.begin() + m, order.begin() + r,
            [&](int x, int y)
            {
                return pair(p[x][axis], x) < pair(p[y][axis], y);
            });
        int u = order[m];
        a[u].low = a[u].high = p[u];
        a[u].first = u;
        a[u].child[0] = build(order, l, m, (axis + 1) % D);
        a[u].child[1] = build(order, m + 1, r, (axis + 1) % D);
        for (int v : a[u].child)
        {
            if (v == -1) continue;
            a[u].first = min(a[u].first, a[v].first);
            for (int d = 0; d < D; d++)
            {
                a[u].low[d] = min(a[u].low[d], a[v].low[d]);
                a[u].high[d] = max(a[u].high[d], a[v].high[d]);
            }
        }
        return u;
    }

    Wide distance(const Point &x, const Point &y) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide t = Wide(x[d]) - y[d];
            ans += t * t;
        }
        return ans;
    }

    Key bound(int u, const Point &q, bool far) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide l = Wide(a[u].low[d]) - q[d];
            Wide r = Wide(a[u].high[d]) - q[d];
            if (far) ans -= max(l * l, r * r);
            else if (l > 0) ans += l * l;
            else if (r < 0) ans += r * r;
        }
        return {ans, a[u].first};
    }
```

剪枝搜索与查询入口（接上页同一结构体）：

```
    void search(int u, const Point &q, int k, bool far,
                priority_queue<Key> &heap) const
    {
        if (u == -1) return;
        if ((int)heap.size() == k && bound(u, q, far) >= heap.top()) return;
        Wide dist = distance(p[u], q);
        Key cur{far ? -dist : dist, u};
        if ((int)heap.size() < k || cur < heap.top())
        {
            heap.push(cur);
            if ((int)heap.size() > k) heap.pop();
        }
        int x = a[u].child[0];
        int y = a[u].child[1];
        if (x == -1 || (y != -1 && bound(y, q, far) < bound(x, q, far)))
        {
            swap(x, y);
        }
        search(x, q, k, far, heap);
        search(y, q, k, far, heap);
    }

    vector<int> query(const Point &q, int k, bool far = false) const
    {
        assert(0 <= k && k <= (int)p.size());
        if (k == 0) return {};
        priority_queue<Key> heap;
        search(root, q, k, far, heap);
        vector<int> ans;
        while (!heap.empty())
        {
            ans.push_back(heap.top().second);
            heap.pop();
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

接口用法演示：多维最近 / 最远前 k 点（接口演示）

演示约定：输入 n,d,m（1≤d≤10），n 个 d 维整数点，随后 m 组查询坐标、k、far。far=0 查询最近 k 点，far=1 查询最远 k 点；0≤k≤n。每行输出从优到劣的 0-based 输入编号，等距编号小者优先。其余维补 0。固定维度时直接用 KDNearest<d> 以节省常数。

所需模板：KDNearest（第 219 页，12.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, d, m;
    cin >> n >> d >> m;
    vector<KDNearest<10>::Point> p(n);
    for (auto &x : p)
    {
        for (int j = 0; j < d; j++) cin >> x[j];
    }
    KDNearest<10> tree(p);
    while (m--)
    {
        KDNearest<10>::Point q{};
        for (int j = 0; j < d; j++) cin >> q[j];
        int k, far;
        cin >> k >> far;
        auto ids = tree.query(q, k, far);
        for (int id : ids) cout << id << ' ';
        cout << '\n';
    }
}
```

应用题补充：Luogu P2093 / JZPFAR

题意：给 n 个互异二维整数点，每次查询 (px,py,k) 的第 k 远点；等距时编号小者算更远。点和查询坐标绝对值至 10^9，n 至 10^5，m 至 10^4，k 至 20 且不超过 n。原题有随机数据条件；本算法不承诺对任意输入次线性。输出 1-based 输入编号。

所需模板：KDNearest（第 219 页，12.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2093

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<KDNearest<>::Point> p(n);
    for (auto &x : p) cin >> x[0] >> x[1];
    KDNearest<> tree(p);
    int m;
    cin >> m;
    while (m--)
    {
        KDNearest<>::Point q;
        int k;
        cin >> q[0] >> q[1] >> k;
        auto ids = tree.query(q, k, true);
        cout << ids.back() + 1 << '\n';
    }
}
```

12.3 动态 K-D Tree 矩形取点与删除

KDRange(points={}), Item 含 Point p=array<long long,2> 和 int id。add({{x,y},id}) 新增一条记录; extract(low,high) 取出并删除闭矩形内所有记录, 返回 vector<Item> 且顺序不保证, 空矩形返回空。允许坐标和 id 重复, 每条插入独立计数; 原编号由调用者提供, 需要唯一编号时自行保证。返回后可改坐标再 add; 一次 extract 完成后再插入, 避免同轮重复移动。a[root].alive 为当前记录数, a/root/free 只读; 其余方法是内部辅助。坐标仅比较, 支持完整有符号 64 位; 活跃及物理节点数须小于 INT_MAX。递归维护物理节点 3/4 平衡, 删除做标记, 存活数少于物理数一半时清理根; 重建回收槽位。树高 O(log(n+1)), 其中 n 为操作前后最大存活数; 按 nth_element 平均线性选择计, 构建 O(n log(n+1)), 单次插入或取点保守上界 O(n $\log^2$(n+1)), 查询可能扫描全部点, 不能承诺对数; 空间 O(历史最大存活数), 空对象重新赋值可释放历史容量。

```
struct KDRange
{
    using Point = array<long long, 2>;

    struct Item
    {
        Point p;
        int id;
    };

    struct Node
    {
        Item item;
        Point low, high;
        array<int, 2> child{};
        int size = 0, alive = 0;
        bool present = false;
    };

    vector<Node> a{Node{}};
    vector<int> free;
    int root = 0;

    KDRange(vector<Item> points = {})
    {
        a.reserve(points.size() + 1);
        root = build(points, 0, points.size(), 0);
    }
```

节点复用与子树维护（接上页同一结构体）：

```
    bool less(const Item &x, const Item &y, int axis) const
    {
        if (x.p[axis] != y.p[axis]) return x.p[axis] < y.p[axis];
        if (x.p[axis ^ 1] != y.p[axis ^ 1]) return x.p[axis ^ 1] < y.p[axis ^ 1];
        return x.id < y.id;
    }

    int node(const Item &item)
    {
        int u;
        if (free.empty())
        {
            u = a.size();
            a.emplace_back();
        }
        else
        {
            u = free.back();
            free.pop_back();
        }
        a[u] = Node{};
        a[u].item = item;
        a[u].low = a[u].high = item.p;
        a[u].size = a[u].alive = 1;
        a[u].present = true;
        return u;
    }

    void pull(int u)
    {
        a[u].low = a[u].high = a[u].item.p;
        a[u].size = 1;
        a[u].alive = a[u].present;
        for (int v : a[u].child)
        {
            if (!v) continue;
            a[u].size += a[v].size;
            a[u].alive += a[v].alive;
            for (int d = 0; d < 2; d++)
            {
                a[u].low[d] = min(a[u].low[d], a[v].low[d]);
                a[u].high[d] = max(a[u].high[d], a[v].high[d]);
            }
        }
    }
```

建树、回收与插入（接上页同一结构体）：

```
    int build(vector<Item> &v, int l, int r, int axis)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2;
        nth_element(v.begin() + l, v.begin() + m, v.begin() + r,
            [&](const Item &x, const Item &y)
            {
                return less(x, y, axis);
            });
        int u = node(v[m]);
        a[u].child[0] = build(v, l, m, axis ^ 1);
        a[u].child[1] = build(v, m + 1, r, axis ^ 1);
        pull(u);
        return u;
    }

    void collect(int u, vector<Item> &v)
    {
        if (!u) return;
        collect(a[u].child[0], v);
        collect(a[u].child[1], v);
        if (a[u].present) v.push_back(a[u].item);
        free.push_back(u);
    }

    int rebuild(int u, int axis)
    {
        vector<Item> v;
        v.reserve(a[u].alive);
        collect(u, v);
        return build(v, 0, v.size(), axis);
    }

    int add(int u, const Item &item, int axis)
    {
        if (!u) return node(item);
        int side = !less(item, a[u].item, axis);
        a[u].child[side] = add(a[u].child[side], item, axis ^ 1);
        pull(u);
        int heavy = max(a[a[u].child[0]].size, a[a[u].child[1]].size);
        if (4LL * heavy > 3LL * a[u].size) u = rebuild(u, axis);
        return u;
    }

    void add(Item item)
    {
        root = add(root, item, 0);
    }
```

矩形取出与删除入口（接上页同一结构体）：

```
    void take(int u, Point low, Point high, vector<Item> &out)
    {
        if (!u || !a[u].alive) return;
        bool inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (a[u].high[d] < low[d] || high[d] < a[u].low[d]) return;
            inside &= low[d] <= a[u].item.p[d] && a[u].item.p[d] <= high[d];
        }
        if (a[u].present && inside)
        {
            out.push_back(a[u].item);
            a[u].present = false;
        }
        for (int v : a[u].child) take(v, low, high, out);
        pull(u);
    }

    vector<Item> extract(Point low, Point high)
    {
        vector<Item> out;
        if (low[0] > high[0] || low[1] > high[1]) return out;
        take(root, low, high, out);
        if (2LL * a[root].alive < a[root].size) root = rebuild(root, 0);
        return out;
    }
};
```

接口用法演示：动态矩形取点与删除（接口演示）

演示约定：输入 n,m 和 n 条 (x,y,id)。操作 1 x y id 新增记录；操作 2 x1 y1 x2 y2 取出并删除闭矩形中的全部记录，输出数量和递增排序后的编号。允许坐标及编号重复，重复记录分别保留；空矩形输出 0。坐标为完整有符号 64 位整数。

所需模板：KDRange（第 224 页，12.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<KDRange::Item> points(n);
    for (auto &v : points) cin >> v.p[0] >> v.p[1] >> v.id;
    KDRange tree(points);
    while (m--)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            KDRange::Item v;
            cin >> v.p[0] >> v.p[1] >> v.id;
            tree.add(v);
        }
        else
        {
            KDRange::Point low, high;
            cin >> low[0] >> low[1] >> high[0] >> high[1];
            auto removed = tree.extract(low, high);
            vector<int> ids;
            for (auto v : removed) ids.push_back(v.id);
            sort(ids.begin(), ids.end());
            cout << ids.size();
            for (int id : ids) cout << ' ' << id;
            cout << '\n';
        }
    }
}
```

接口用法演示：曼哈顿邻域同步搬移（kuangbin 源码模型）

演示约定：输入 n,q,W,H 及 n 个点；每次给 X,Y,E,a,b,c,d,f,g，把操作开始时曼哈顿距离 ≤E 的点同步改为 ((a*x+b*y+(id+1)*c)%W,(d*x+f*y+(id+1)*g)%H)。输入点在 [0,W)×[0,H)，W,H 为 1..10^9，系数和 E 为 0..10^9，中心坐标绝对值 ≤10^9，n≤10^5；编号为输入次序 0-based。最后按原序输出坐标。使用 (x+y,x-y) 转闭矩形；128 位乘加。只核对源码模型，不冒充 UVALive6045 原题协议；原文小于与源码闭边界存在差异，待原题确认。

所需模板：KDRange（第 224 页，12.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w, h;
    cin >> n >> q >> w >> h;
    vector<KDRange::Point> p(n);
    vector<KDRange::Item> items;
    for (int i = 0; i < n; i++)
    {
        cin >> p[i][0] >> p[i][1];
        auto [x, y] = p[i];
        items.push_back({{x + y, x - y}, i});
    }
    KDRange tree(items);
    while (q--)
    {
        long long x, y, e, a, b, c, d, f, g;
        cin >> x >> y >> e >> a >> b >> c >> d >> f >> g;
        auto removed = tree.extract({x + y - e, x - y - e},
                                    {x + y + e, x - y + e});
        for (auto v : removed)
        {
            int id = v.id;
            auto [u, t] = p[id];
            long long nx = ((__int128)u * a + (__int128)t * b + (__int128)(id + 1) * c) % w;
            long long ny = ((__int128)u * d + (__int128)t * f + (__int128)(id + 1) * g) % h;
            p[id] = {nx, ny};
            tree.add({{nx + ny, nx - ny}, id});
        }
    }
    for (auto [x, y] : p) cout << x << ' ' << y << '\n';
}
```

12.4 静态二维矩形最小权与删点

KDMin(points) 批量建树；Item 含二维 signed64 坐标 p 与 signed64 权 key，原输入位置为 0-based 编号。query(low, high) 查闭矩形内尚存记录，返回 (key, 编号) 最小的原编号；空集或反向矩形返回-1。erase(id) 删除该编号，重复删除无影响；相同坐标与权的记录彼此独立。p 保留原序可只读访问，其他成员也是只读约定，内部 build/pull 不直接调用；不支持插入、改权或恢复。没有数值哨兵，坐标与权只比较，均支持完整 signed64；n≤INT_MAX 且内存可分配。按 nth_element 平均线性选择计，构建 O(n log(n+1))，erase 为 O(log(n+1))，query 保守最坏 O(n)，空间 O(n)，递归深度 O(log(n+1))。包围盒保持建树时范围，删除后可能偏松但不会漏掉活点；best 只汇总活点。

```
struct KDMin
{
    using Point = array<long long, 2>;

    struct Item
    {
        Point p;
        long long key;
    };

    struct Node
    {
        Point low, high;
        int ch[2] = {-1, -1}, fa = -1, best = -1;
        bool alive = true;
    };

    vector<Item> p;
    vector<Node> a;
    int root = -1;

    KDMin(const vector<Item> &points) : p(points), a(points.size())
    {
        assert(points.size() <= INT_MAX);
        vector<int> order(p.size());
        iota(order.begin(), order.end(), 0);
        root = build(order, 0, order.size(), 0);
    }

    int better(int x, int y) const
    {
        if (x == -1) return y;
        if (y == -1) return x;
        if (p[x].key != p[y].key) return p[x].key < p[y].key ? x : y;
        return min(x, y);
    }

    void pull(int u)
    {
        a[u].best = a[u].alive ? u : -1;
        for (int v : a[u].ch)
        {
            if (v != -1) a[u].best = better(a[u].best, a[v].best);
        }
    }
```

中位数建树与固定包围盒（接上页同一结构体）：

```
    int build(vector<int> &order, int l, int r, int d)
    {
        if (l == r) return -1;
        int mid = l + (r - l) / 2;
        nth_element(order.begin() + l, order.begin() + mid, order.begin() + r,
                    [&](int x, int y)
        {
            if (p[x].p[d] != p[y].p[d]) return p[x].p[d] < p[y].p[d];
            return x < y;
        });
        int u = order[mid];
        a[u].low = a[u].high = p[u].p;
        a[u].ch[0] = build(order, l, mid, d ^ 1);
        a[u].ch[1] = build(order, mid + 1, r, d ^ 1);
        for (int v : a[u].ch)
        {
            if (v == -1) continue;
            a[v].fa = u;
            for (int j = 0; j < 2; j++)
            {
                a[u].low[j] = min(a[u].low[j], a[v].low[j]);
                a[u].high[j] = max(a[u].high[j], a[v].high[j]);
            }
        }
        pull(u);
        return u;
    }
```

按编号删除与矩形查询（接上页同一结构体）：

```
    void erase(int id)
    {
        assert(0 <= id && id < (int)p.size());
        a[id].alive = false;
        for (int u = id; u != -1; u = a[u].fa) pull(u);
    }

    int query(int u, const Point &low, const Point &high) const
    {
        if (u == -1 || a[u].best == -1) return -1;
        bool covered = true, inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (high[d] < a[u].low[d] || low[d] > a[u].high[d]) return -1;
            if (low[d] > a[u].low[d] || high[d] < a[u].high[d]) covered = false;
            if (p[u].p[d] < low[d] || p[u].p[d] > high[d]) inside = false;
        }
        if (covered) return a[u].best;
        int ans = inside && a[u].alive ? u : -1;
        for (int v : a[u].ch) ans = better(ans, query(v, low, high));
        return ans;
    }

    int query(const Point &low, const Point &high) const
    {
        if (low[0] > high[0] || low[1] > high[1]) return -1;
        return query(root, low, high);
    }
};
```

接口用法演示：矩形最小权与按编号删除（接口演示）

演示约定：输入 n,q 和 n 条 x,y,key 记录；操作 1 id 删除 0-based 原编号，操作 2 xl yl xr yr 查询闭矩形内 (key, 编号) 最小的活记录编号，无解输出-1。支持重复坐标/权、重复删除、反向矩形与空点集。

所需模板：KDMin（第 230 页，12.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/ds/kdt/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<KDMin::Item> p(n);
    for (auto &v : p) cin >> v.p[0] >> v.p[1] >> v.key;
    KDMin kd(p);
    while (q--)
    {
        int op;
        cin >> op;
        if (op == 1)
        {
            int id;
            cin >> id;
            kd.erase(id);
        }
        else
        {
            KDMin::Point low, high;
            cin >> low[0] >> low[1] >> high[0] >> high[1];
            cout << kd.query(low, high) << '\n';
        }
    }
}
```

应用题补充：Codeforces 44G Shooting Gallery

题意：按顺序发射子弹，每发击碎最近尚存靶子，边界也算命中；输出每发击中的 1-based 靶子编号或 0。n,m≤10^5，坐标 0..10^7，矩形严格非退化，z 互异且为 1..10^7。把子弹时间作为 key，按 z 从近到远处理靶子，查询靶面内最早未用子弹并删除。比赛应用，非正式模板题；本地验证不等于在线 AC。

所需模板：KDMin（第 230 页，12.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://codeforces.com/problemset/problem/44/G

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 5>> targets(n);
    for (auto &t : targets)
    {
        for (auto &x : t) cin >> x;
    }
    vector<int> order(n);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int x, int y)
    {
        return targets[x][4] < targets[y][4];
    });
    int m;
    cin >> m;
    vector<KDMin::Item> shots(m);
    for (int i = 0; i < m; i++)
    {
        cin >> shots[i].p[0] >> shots[i].p[1];
        shots[i].key = i;
    }
    KDMin kd(shots);
    vector<int> ans(m);
    for (int id : order)
    {
        auto [xl, xr, yl, yr, z] = targets[id];
        int shot = kd.query({xl, yl}, {xr, yr});
        if (shot == -1) continue;
        ans[shot] = id + 1;
        kd.erase(shot);
    }
    for (int x : ans) cout << x << '\n';
}
```

第十三章　动态树

13.1　Link Cut Tree

13.1.1　Link-cut tree：动态森林与路径异或

构造 LinkCutTree(n)，点号 1..n，初始权值为零。set(x,value) 修改点权；link(x,y) 仅在不同连通块之间连边并返回是否成功；cut(x,y) 只删除真实存在的边，否则返回 false。connected 查询连通性；query(x,y) 返回路径点权异或，要求两点连通，支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 O(log n)，O(n) 空间，splay 下传使用显式栈，支持长链。权值为 unsigned long long。仅维护路径异或，不包含虚子树聚合或动态直径。

```
struct LinkCutTree
{
    using U = unsigned long long;

    struct Node
    {
        int ch[2]{}, fa = 0;
        U val = 0, sum = 0;
        bool rev = false;
    };

    vector<Node> a;
    vector<int> st;

    LinkCutTree(int n) : a(n + 1), st(n + 1) {}

    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }

    void reverse(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        reverse(a[x].ch[0]);
        reverse(a[x].ch[1]);
        a[x].rev = false;
    }
```

```
40      void rotate(int x)
41      {
42          int y = a[x].fa, z = a[y].fa;
43          int k = a[y].ch[1] == x;
44          int w = a[x].ch[k ^ 1];
45          if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46          a[x].fa = z;
47          a[y].ch[k] = w;
48          if (w) a[w].fa = y;
49          a[x].ch[k ^ 1] = y;
50          a[y].fa = x;
51          pull(y);
52          pull(x);
53      }
54
55      void splay(int x)
56      {
57          int top = 0, y = x;
58          st[top++] = y;
59          while (!is_root(y))
60          {
61              y = a[y].fa;
62              st[top++] = y;
63          }
64          while (top) push(st[--top]);
65          while (!is_root(x))
66          {
67              int y = a[x].fa, z = a[y].fa;
68              if (!is_root(y))
69              {
70                  bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71                  rotate(same ? y : x);
72              }
73              rotate(x);
74          }
75      }
```

```
77      void access(int x)
78      {
79          int last = 0;
80          for (int y = x; y; y = a[y].fa)
81          {
82              splay(y);
83              // The old right child's fa remains y: it becomes a path-parent.
84              a[y].ch[1] = last;
85              if (last) a[last].fa = y;
86              pull(y);
87              last = y;
88          }
89          splay(x);
90      }
91
92      void make_root(int x)
93      {
94          access(x);
95          reverse(x);
96      }
97
98      int find_root(int x)
99      {
100         access(x);
101         push(x);
102         while (a[x].ch[0])
103         {
104             x = a[x].ch[0];
105             push(x);
106         }
107         splay(x);
108         return x;
109     }
```

```
    bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }

    bool link(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x) return false;
        a[x].fa = y;
        return true;
    }

    bool cut(int x, int y)
    {
        make_root(x);
        access(y);
        if (a[y].ch[0] != x || a[x].ch[1]) return false;
        a[y].ch[0] = 0;
        a[x].fa = 0;
        pull(y);
        return true;
    }

    void set(int x, U value)
    {
        access(x);
        a[x].val = value;
        pull(x);
    }

    // x and y must be connected. This is a path aggregate, not a subtree sum.
    U query(int x, int y)
    {
        make_root(x);
        access(y);
        return a[y].sum;
    }
};
```

模板题使用：Luogu P3690

题意：维护点权森林，支持路径异或、连边、删边和单点赋值。初始没有边；0 查询、1 连边、2 删边、3 赋值。题目保证查询两点连通，成环的连边和不存在的删边由接口拒绝。点号 1..n。

所需模板：LinkCutTree（第 235 页，13.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3690

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LinkCutTree g(n);
    for (int i = 1; i <= n; i++)
    {
        unsigned long long x;
        cin >> x;
        g.set(i, x);
    }
    while (m--)
    {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 0)
            cout << g.query(x, y) << '\n';
        else if (op == 1)
            g.link(x, y);
        else if (op == 2)
            g.cut(x, y);
        else
            g.set(x, y);
    }
}
```

13.1.2 动态换根路径乘积总和

代码依赖：ModInt（第 263 页，A.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

TreePathProducts<mod>(g,weight) 要求 g 是非空无向树，点号 0..n-1、每条边双向存储，weight 为等长 vector<ModInt<mod»。set(u,w) 修改点权；query(u) 返回从 u 到所有顶点 v 的唯一路径上点权乘积之和，含 u=v 的单点路径。固定拓扑，不提供 link/cut。建树 O(n)，修改和查询摊 O(log n)，空间 O(n)，递归建树和 splay 下传最坏深度 O(n)。允许零和负权，mod>1 且不要求素数，不用除法或逆元。a 与全部辅助接口仅供内部使用，内部点号 1..n；不能把公开接口的 0-based 点号直接传给 make_root。依赖 ModInt；附件爱莲说的输入 x 由主程序显式转为权值 1-x。

```
template <int mod> struct TreePathProducts
{
    using Z = ModInt<mod>;

    struct Node
    {
        int ch[2]{}, fa = 0;
        Z weight = 0, product = 1, left = 0, right = 0, virt = 0;
        bool rev = false;
    };

    vector<Node> a;

    // g is a nonempty tree, with 0-based vertices and one weight per vertex.
    TreePathProducts(const vector<vector<int>> &g, const vector<Z> &weight)
        : a(g.size() + 1)
    {
        assert(!g.empty() && g.size() == weight.size());
        auto dfs = [&](auto &&self, int u, int parent) -> void
        {
            int x = u + 1;
            a[x].fa = parent + 1;
            a[x].weight = weight[u];
            for (int v : g[u])
                if (v != parent)
                {
                    self(self, v, u);
                    a[x].virt += a[v + 1].left;
                }
            pull(x);
        };
        dfs(dfs, 0, -1);
    }
```

聚合、翻转、递归下传与旋转（接上页同一结构体）：

```
    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x)
    {
        auto &t = a[x];
        const auto &l = a[t.ch[0]], &r = a[t.ch[1]];
        t.product = l.product * t.weight * r.product;
        t.left = l.left + l.product * t.weight * (Z(1) + t.virt + r.left);
        t.right = r.right + r.product * t.weight * (Z(1) + t.virt + l.right);
    }

    void reverse(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        swap(a[x].left, a[x].right);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        reverse(a[x].ch[0]);
        reverse(a[x].ch[1]);
        a[x].rev = false;
    }

    void push_path(int x)
    {
        if (!is_root(x)) push_path(a[x].fa);
        push(x);
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
        if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

Splay、虚实边切换与公开接口（接上页同一结构体）：

```
    void splay(int x)
    {
        push_path(x);
        while (!is_root(x))
        {
            int y = a[x].fa, z = a[y].fa;
            if (!is_root(y))
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    void make_root(int x)
    {
        int last = 0;
        for (int y = x; y; y = a[y].fa)
        {
            splay(y);
            a[y].virt += a[a[y].ch[1]].left - a[last].left;
            a[y].ch[1] = last;
            if (last) a[last].fa = y;
            pull(y);
            last = y;
        }
        splay(x);
        reverse(x);
    }

    void set(int u, Z weight)
    {
        int x = u + 1;
        make_root(x);
        a[x].weight = weight;
        pull(x);
    }

    Z query(int u)
    {
        int x = u + 1;
        make_root(x);
        return a[x].left;
    }
};
```

应用题补充：附件爱莲说接口（本地应用示例）

题意：沿用附件输入：n,q，n 个 x，n-1 条 1-based 无向边；1 u x 修改参数，2 u 查询。主程序把每个 x 转为点权 1-x，并把顶点转为 0-based。输出所有从 u 出发路径的点权乘积总和。原题外部地址和线上结果尚未核验。

所需模板：ModInt（第 263 页，A.1.2 节）、TreePathProducts（第 240 页，13.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://github.com/gsh20040816/CPCtemplate/issues/11

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Z = ModInt<998244353>;
    int n, q;
    cin >> n >> q;
    vector<Z> weight(n);
    for (auto &w : weight)
    {
        int x;
        cin >> x;
        w = Z(1) - Z(x);
    }
    vector<vector<int>> g(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    TreePathProducts<998244353> tree(g, weight);
    while (q--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 1)
        {
            int x;
            cin >> x;
            tree.set(u - 1, Z(1) - Z(x));
        }
        else
            cout << tree.query(u - 1).v << '\n';
    }
}
```

相关应用：固定拓扑的虚子树聚合与点权修改后全树路径异或和，不表示通用动态森林接口

13.1.3 点权修改与全树路径异或和

XorPathSum(parent,value)，两数组 0-based 且等长，1≤n≤200000，parent 构成一棵固定有根树，唯一根的父亲为-1；根和节点编号任意。点权为 [0,2^27) 整数。sum() 返回所有无序端点对 u≤v 的简单路径上点权异或值之和，包含 n 条单点路径；set(x,value) 修改一个点权并返回新总和。不是路径异或查询，也不支持 link/cut 或改变拓扑。

逐位维护首尾路径异或计数及虚子树之间的配对数，计数相乘先提升为 long long。27 位固定，n≤200000 时总和至多 n(n+1)/2 乘 (2^27-1)，可由 signed64 表示。初建 O(27n)，set 摊还 O(27logn)，sum 为 O(1)，空间 O(27n) 且常数较大：每个节点保存两份 Info、虚子树计数及配对数；按实际 sizeof(Node) 估算内存。初建 DFS 保持递归，最坏深度 n；splay 无翻转标记。t 和其他函数仅供内部实现，调用方只用构造、set、sum；完整对象复制后相互独立。

WIDA07C 原稿的 cnt 乘法在两个 5 万点分支时会出现 int 中间值溢出，本版显式提升；原题完整约束尚未核实，不据此断言原题实现有错。当前为独立路径枚举、原稿小规模对照及大树公式验证，非线上 AC。

```
struct XorPathSum
{
    using ll = long long;
    static constexpr int B = 27;
    struct Info
    {
        array<array<int, 2>, B> up{}, down{};
        int path = 0;
        ll sum = 0;
    };
    struct Node
    {
        array<int, 2> ch{};
        int p = 0;
        Info val, tot;
        array<array<int, 2>, B> cnt{};
        array<array<ll, 2>, B> pairs{};
        ll sum = 0;
    };
    vector<Node> t;
    ll ans = 0;

    static Info join(const Info &a, const Info &b)
    {
        Info c;
        c.path = a.path ^ b.path;
        c.sum = a.sum + b.sum;
        for (int i = 0; i < B; i++)
            for (int j = 0; j < 2; j++)
            {
                c.sum += (1LL << i) * a.down[i][j] * b.up[i][j ^ 1];
                c.up[i][j] = a.up[i][j] + b.up[i][j ^ (a.path >> i & 1)];
                c.down[i][j] = b.down[i][j] + a.down[i][j ^ (b.path >> i & 1)];
            }
        return c;
    }

    void pull(int u)
    {
        t[u].tot = join(join(t[t[u].ch[0]].tot, t[u].val), t[t[u].ch[1]].tot);
    }
```

全树路径异或和（接上页同一结构体）：

```
    // sign = +1 adds a virtual child; -1 removes its old aggregate.
    void branch(int u, const Info &s, int sign)
    {
        auto &a = t[u];
        for (int i = 0; i < B; i++)
        {
            if (sign < 0)
                for (int j = 0; j < 2; j++)
                    a.cnt[i][j] -= s.up[i][j];
            for (int x = 0; x < 2; x++)
            {
                ll ways = s.up[i][1 ^ x];
                for (int j = 0; j < 2; j++)
                    ways += 1LL * a.cnt[i][j] * s.up[i][j ^ 1 ^ x];
                a.pairs[i][x] += sign * ways;
            }
            if (sign > 0)
                for (int j = 0; j < 2; j++)
                    a.cnt[i][j] += s.up[i][j];
        }
        a.sum += sign * s.sum;
    }

    void update(int u)
    {
        auto &a = t[u];
        a.val.sum = a.val.path + a.sum;
        for (int i = 0; i < B; i++)
        {
            int bit = a.val.path >> i & 1;
            a.val.sum += (1LL << i) * a.pairs[i][bit];
            for (int j = 0; j < 2; j++)
            {
                a.val.up[i][j] = a.cnt[i][j ^ bit];
                a.val.down[i][j] = a.cnt[i][j ^ bit];
            }
            a.val.up[i][bit]++;
            a.val.down[i][bit]++;
        }
        pull(u);
    }

    int init(int u, const vector<vector<int>> &g)
    {
        int size = 1;
        for (int v : g[u])
        {
            size += init(v, g);
            branch(u, t[v].tot, 1);
        }
        update(u);
        return size;
    }
```

全树路径异或和（接上页同一结构体）：

```
    // parent/value are 0-based; exactly one parent is -1, topology stays fixed.
    XorPathSum(const vector<int> &parent, const vector<int> &value)
        : t(parent.size() + 1)
    {
        int n = parent.size();
        assert(n > 0 && n <= 200000 && (int)value.size() == n);
        vector<vector<int>> g(n + 1);
        int root = 0;
        for (int i = 0; i < n; i++)
        {
            assert(value[i] >= 0 && value[i] < (1 << B));
            assert(parent[i] >= -1 && parent[i] < n && parent[i] != i);
            t[i + 1].val.path = value[i];
            t[i + 1].p = parent[i] + 1;
            if (parent[i] == -1)
            {
                assert(root == 0);
                root = i + 1;
            }
            else
                g[parent[i] + 1].push_back(i + 1);
        }
        assert(root > 0);
        int size = init(root, g);
        assert(size == n);
        (void)size;
        ans = t[root].tot.sum;
    }

    bool isroot(int u) const
    {
        int p = t[u].p;
        return t[p].ch[0] != u && t[p].ch[1] != u;
    }

    void rotate(int u)
    {
        int p = t[u].p, q = t[p].p;
        int k = t[p].ch[1] == u;
        int v = t[u].ch[k ^ 1];
        if (!isroot(p))
            t[q].ch[t[q].ch[1] == p] = u;
        t[u].p = q;
        t[p].ch[k] = v;
        if (v)
            t[v].p = p;
        t[u].ch[k ^ 1] = p;
        t[p].p = u;
        pull(p);
    }
```

全树路径异或和（接上页同一结构体）：

```
    void splay(int u)
    {
        while (!isroot(u))
        {
            int p = t[u].p, q = t[p].p;
            if (!isroot(p))
                rotate((t[p].ch[1] == u) == (t[q].ch[1] == p) ? p : u);
            rotate(u);
        }
        pull(u);
    }

    ll set(int x, int value)
    {
        assert(x >= 0 && x + 1 < (int)t.size());
        assert(value >= 0 && value < (1 << B));
        int u = x + 1;
        Info old;
        for (int i = u, q = 0; i; q = i, i = t[i].p)
        {
            splay(i);
            if (t[i].ch[1])
                branch(i, t[t[i].ch[1]].tot, 1);
            t[i].ch[1] = q;
            if (q)
                branch(i, old, -1);
            else
                t[i].val.path = value;
            old = t[i].tot;
            update(i);
        }
        splay(u);
        ans = t[u].tot.sum;
        return ans;
    }

    ll sum() const
    {
        return ans;
    }
};
```

接口用法演示：点权修改与全树路径异或和 API

演示约定：读 n q，接着 n 个 0-based 父亲编号（唯一根为-1）与 n 个 27 位非负点权；输出初始所有无序端点简单路径的点权异或和，包含单点路径。随后 q 行 x value，每次修改点 x 并输出新总和。固定树，1≤n≤200000；输出 signed64。自定义 API 协议，非原比赛题判定。

所需模板：XorPathSum（第 243 页，13.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> parent(n), value(n);
    for (int &p : parent)
        cin >> p;
    for (int &x : value)
        cin >> x;
    XorPathSum tree(parent, value);
    cout << tree.sum() << '\n';
    while (q--)
    {
        int x, value;
        cin >> x >> value;
        cout << tree.set(x, value) << '\n';
    }
}
```

13.1.4 动态森林路径加与最大值

以 0-based 点权 vector 构造，点号 1..n，初始无边，n<INT_MAX。link(x, y) 只连接不同树；cut(x, y) 仅删直接边；cut_parent(r, x) 以 r 为根删除 x 的父边，r 与 x 可不相邻。三者成功返回 true，非法操作返回 false。add(x, y, w) 对路径全部点加 w，不连通返回 false；query(x, y) 返回 optional<long long> 路径最大值，不连通为空，支持同点及负权。失败保持森林和点权，可改变辅助树。所有点权、最大值及累计懒加标记的 64 位加法均须不溢出。make_root 与 find_root 改变或查询表示树根；其余方法及 a 属内部。操作均摊 O(log n)，空间 O(n)，递归下传辅助树路径，单次深度可达 O(n)，需足够栈。不维护虚子树聚合。

```
struct DynamicPathMax
{
    using ll = long long;

    struct Node
    {
        int ch[2]{}, fa = 0;
        ll val = 0, mx = 0, add = 0;
        bool rev = false;
    };

    vector<Node> a;

    DynamicPathMax(const vector<ll> &v) : a(v.size() + 1)
    {
        for (int i = 0; i < (int)v.size(); i++)
            a[i + 1].val = a[i + 1].mx = v[i];
    }

    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x)
    {
        a[x].mx = a[x].val;
        for (int y : a[x].ch)
            if (y)
                a[x].mx = max(a[x].mx, a[y].mx);
    }

    void flip(int x)
    {
        if (!x)
            return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }
```

递归下传与旋转（接上页同一结构体）：

```
    void apply(int x, ll w)
    {
        if (!x)
            return;
        a[x].val += w;
        a[x].mx += w;
        a[x].add += w;
    }

    void push(int x)
    {
        if (a[x].rev)
        {
            flip(a[x].ch[0]);
            flip(a[x].ch[1]);
            a[x].rev = false;
        }
        if (a[x].add)
        {
            apply(a[x].ch[0], a[x].add);
            apply(a[x].ch[1], a[x].add);
            a[x].add = 0;
        }
    }

    void push_path(int x)
    {
        if (!is_root(x))
            push_path(a[x].fa);
        push(x);
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (!is_root(y))
            a[z].ch[a[z].ch[1] == y] = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w)
            a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

伸展、访问与换根（接上页同一结构体）：

```
    void splay(int x)
    {
        push_path(x);
        while (!is_root(x))
        {
            int y = a[x].fa, z = a[y].fa;
            if (!is_root(y))
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        int last = 0;
        for (int y = x; y; y = a[y].fa)
        {
            splay(y);
            a[y].ch[1] = last;
            if (last)
                a[last].fa = y;
            pull(y);
            last = y;
        }
        splay(x);
    }

    void make_root(int x)
    {
        access(x);
        flip(x);
    }

    int find_root(int x)
    {
        access(x);
        push(x);
        while (a[x].ch[0])
        {
            x = a[x].ch[0];
            push(x);
        }
        splay(x);
        return x;
    }
```

连边、两种删边与路径操作（接上页同一结构体）：

```
    bool expose(int x, int y)
    {
        make_root(x);
        if (find_root(y) != x)
            return false;
        access(y);
        return true;
    }

    bool link(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x)
            return false;
        a[x].fa = y;
        return true;
    }

    bool cut(int x, int y)
    {
        if (!expose(x, y) || a[y].ch[0] != x || a[x].ch[1])
            return false;
        a[y].ch[0] = 0;
        a[x].fa = 0;
        pull(y);
        return true;
    }

    bool cut_parent(int root, int x)
    {
        if (root == x || !expose(root, x))
            return false;
        int left = a[x].ch[0];
        a[left].fa = 0;
        a[x].ch[0] = 0;
        pull(x);
        return true;
    }

    bool add(int x, int y, ll w)
    {
        if (!expose(x, y))
            return false;
        apply(y, w);
        return true;
    }

    optional<ll> query(int x, int y)
    {
        if (!expose(x, y))
            return nullopt;
        return a[y].mx;
    }
};
```

接口用法演示：动态森林路径加最大值（kuangbin HDU4010 模型）

演示约定：EOF 多组：读 n、n-1 条边、点权和 q。1 x y 连边；2 x y 以 x 为根删 y 的父边；3 w x y 路径加；4 x y 路径最大值。非法输出-1，成功修改不输出，组末空行。来源模型/API，官方题面、资源限制及线上 AC 未核验。

所需模板：DynamicPathMax（第 249 页，13.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=4010

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    while (cin >> n)
    {
        vector<pair<int, int>> edges(n - 1);
        for (auto &[x, y] : edges)
            cin >> x >> y;
        vector<long long> v(n);
        for (auto &x : v)
            cin >> x;
        DynamicPathMax t(v);
        for (auto [x, y] : edges)
            t.link(x, y);
        int q;
        cin >> q;
        while (q--)
        {
            int op, x, y;
            long long w = 0;
            cin >> op;
            if (op == 3)
                cin >> w;
            cin >> x >> y;
            bool ok = true;
            if (op == 1)
                ok = t.link(x, y);
            else if (op == 2)
                ok = t.cut_parent(x, y);
            else if (op == 3)
                ok = t.add(x, y, w);
            else
            {
                auto ans = t.query(x, y);
                cout << (ans ? *ans : -1) << '\n';
            }
            if (!ok)
                cout << -1 << '\n';
        }
        cout << '\n';
    }
    return 0;
}
```

第十四章　配套知识与验证范围

14.1　可持久化区间拼接的懒标记

代码见第 175 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足 $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为 $\text{offset} + \text{source.lazy} - \text{destination.lazy}$。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为 $O(\log n)$ 新节点，查询为 $O(\log n)$ 时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 `INT_MAX`。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 `INT_MAX` 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的 vector 应用已有 QOJ8240 AC，其余通用接口范围仍由本地验证覆盖。来源需求是2023 杭州站题解 第 24–26 页的 K 题，完整驱动及验证见下节。

回退时间与追加版本编号。 SP11470 To the moon 完整用法见第 180 页。模板 add 返回不断追加的版本编号，而题目的时间戳在 B 操作后可能减小，二者不能直接混用。用 ver[t] 保存时间戳 t 对应的模板版本；C 从 ver[t] 更新，再令 ver[t+1] 指向新版本并推进时间。再次经过同一时间戳时替换映射，不能写入原来版本的节点。Q、H 读取对应映射；B 只移动当前时间。初值无需逐点建树：把初始前缀和单独保留，树中仅存后续区间增量，每次查询将两部分相加。这样初始构造 $O(n)$，之后每次 C/Q/H 为 $O(\log n)$，B 为 $O(1)$；不调用 splice。本地以完整数组快照独立核验分支与历史，并检查正负大初值、反复回退、时间戳覆盖和最大规模部分区间更新。

14.2 杭州 2023 K：跨版本拼接应用

Card Game 完整用法见第 182 页，使用前节的可持久化区间模板。

将下标转为从 0 开始，设 $F_i(r)$ 为区间 $[i,r]$ 的剩余牌数，p 为 i 之后第一张与 a_i 相同的牌，若不存在则 $p=n$。当 $r<p$ 时，第一张牌始终保留，剩余牌的过程与从 $i+1$ 开始相同；当 $r\ge p$ 时，到达 p 会将此前区间全部清空。因此

$$F_i(r)=\begin{cases}1+F_{i+1}(r), & r<p,\\ F_{p+1}(r), & r\ge p.\end{cases}$$

每个版本的坐标是右端点 r，值是 $F_i(r)$。从 $i=n-1$ 向前处理：复制版本 $i+1$ 并在 $[i,p)$ 加 1，再把 $[p,n)$ 拼接自版本 $p+1$。$p=n$ 时省去后一步。询问原题的 $[l,r]$ 时，从版本 $l-1$ 读取坐标 $r-1$ 的单点值。端点先与上次答案异或解码，不能将输入查询预先离线排序。

$n,q\le 300000$，牌号位于 $[1,n]$。构建及查询时间分别为 $O(n\log n)$、$O(q\log n)$。驱动另计算各次操作可能访问的相交节点数作为分配上界，一次 reserve 足够容纳整个构建过程，避免复制扩容的临时峰值。该计数仅依据区间结构，不影响牌局答案，且同样为 $O(n\log n)$。

本地参照按题意逐张放牌，遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间，并打乱询问顺序、真实编码异或输入。另有随机牌局、$n=q=300000$ 的全异、全同、周期序列闭式检查，以及稀疏重复大数组的短区间逐张模拟。vector 驱动的历史 AC通过 41 个正式测试和 3 个额外测试，最慢 789 ms、452688 KB。当前展开程序与该快照仅格式不同，本轮未重新提交；传统版已退役。此应用仅调用区间加 1、后缀拼接、单点读取；一般有符号增量与区间求和仍由前节本地测试支持。

14.3 并查集模板题的接口范围

P3367 输入点号为 1-based，驱动统一减一后调用 dsu。记录 297668102 通过全部 20 个测试，最慢单点 196 ms、峰值内存 2.27 MB。该输出验证 merge、same 及其调用的 find，不验证 size、groups；后两者已由独立标签参考和 50 万点长链测试核对。

14.4 动态树路径乘积：虚子树与双向信息

附件“爱莲说”对应的量是

$$F(u)=\sum_{v\in V}\prod_{x\in \mathrm{path}(u,v)} w_x.$$

包括单点路径，所以一个孤立点的答案是其点权，而不是 1。固定根后，递推为 $F(u)=w_u(1+\sum_{v\ \text{为子节点}}F(v))$。使用第 240 页的 `TreePathProducts`，可以单点修改并动态选取查询根。它维护固定树，不包含连边、删边。

辅助树按实链顺序维护三项：整条链点权积 P、从链首出发的贡献 L、从链尾出发的贡献 R。每个点另记虚儿子贡献之和 G。对于左、右辅助子树，合并式为

$$P=P_l w P_r,\qquad L=L_l+P_l w(1+G+L_r),\qquad R=R_r+P_r w(1+G+R_l).$$

空辅助树取 $P=1, L=R=0$。翻转时必须同时交换左右儿子和 L,R；只翻儿子会破坏换根后的方向。

`access` 把旧右链变为虚儿子，将它的 L 加入 G；把新右链变为实儿子，从 G 减去它的 L。旧右链的父指针保留为路径父亲。修改点权前先将该点换为根，确保旧贡献已经沿访问路径正确移出虚子树项；然后修改权值并更新合并信息。换根结束时根处的 L 就是所求总和。整个过程只用加减乘，没有除法，零权、负权与合数模数同样适用。

接口使用 0-based 顶点；内部辅助树为 1-based，内部函数不可直接作为外部 API。附件输入参数 x 对应 $w=1-x$，这个转换在使用程序中显式完成。从源码可确定上述递推和输入协议，但原题外部地址尚未核验，因此只登记本地应用用法。建树 $O(n)$，修改和查询摊还 $O(\log n)$，总空间 $O(n)$；建树和辅助树下传保持递归，最坏深度 $O(n)$。本地以逐终点路径枚举、模 8、零权、20 万点链与星形及反复换根验证，不将普通 LCT 的线上记录视为这个变体的 AC。

14.5 隐式 Treap 位翻转与顺序反转

附件 FHQ Treap 中名为 `Reverse` 的操作是逐位异或 1；它没有交换左右儿子，不是区间顺序反转。

`flip_bits(l,r)` 明确表达前者，要求区间元素均为 0 或 1；原有 `reverse(l,r)` 表达后者。两者均使用 1-based 闭区间。例如 0011 的前三位取反得到 1101，而前三位顺序反转得到 1001。二进制序列的区间和就是 1 的个数，仍可直接调用 `query`。

取反可写作 $x\mapsto 1-x$。用符号标记与加法标记表示 $x\mapsto sx+b$，其中 $s\in\{1,-1\}$。取负时同时令 $s\leftarrow -s, b\leftarrow -b$，再加 1；下传时必须先符号、再加法。若把先后顺序颠倒，已有加法会变成错误的符号。顺序反转则只交换儿子并翻转其标记，它与逐元素的取负、加法可交换。结构体公开位翻转接口仅用于二进制值；所有标记和取负中间量仍须在 `long long` 范围内，不能对 `LLONG_MIN` 直接取负。

附件 `Build` 按中点建立树后给节点随机优先级，未修复优先级堆序；这不能直接继承随机 Treap 的平衡复杂度证明。本库逐次 `insert` 构建，期望 $O(n\log n)$，每次分裂合并均维持堆序；不能把这种构建成本写为线性。随机优先级下单次操作期望 $O(\log n)$，不保证针对已知种子的自适应对抗操作的最坏界。分裂、合并及中序输出保持递归。附件缺少完整牛客题号和主程序，因此新增使用例只作为本地接口示例。

14.6 替罪羊树：重复计数与墓碑清理

第 154 页使用三个不同的量：`size` 计元素副本总数，`nodes` 计物理节点数，`live` 计仍有副本的不同键数。重复值增加计数，删除最后一个副本后保留墓碑。查询排名只累计副本数，不能用物理节点数代替。

更新回溯时，若某孩子占节点数的比例超过 3/4，或者有效键不足物理节点的一半，就递归中序收集活节点，按中点重建。重建复用活节点编号，保留重复次数。重建后的中序和计数均不变，故集合与排名不变；所有新子树均满足平衡与有效比例。令 d 为当前不同键数，树高为 $O(\log(d+2))$，因为每层节点数缩小到至多 3/4，根的物理节点数至多为 $2d$。查询最坏对数，更新摊还对数；一次重建仍可能线性。删除清理后的墓碑不回收到节点池，空间按累计分配量计算，不按当前集合大小计算。

附件的 `Rank` 只有命中相等键才加一，未命中时可能少一；仅插入 1 后查询 2 的排名就是一个反例。本版统一定义为严格较小副本数加一，空树中的排名也为 1。严格前驱后继使用 `optional`，避免数值哨兵与合法键冲突；完整 `long long` 键无需做减一或加一。P3369 保证第 k 小及邻居存在，使用程序才能直接解引用结果。所有递归保持原样；本地排序向量对拍、重建不变量及 20 万键规模均已验证，线上待补。

14.7 线段树维护单调栈：双向扫描与接雨水

`MonotoneStackSeg` 维护区间加后的运行最大值贡献。每段的 F_0, F_1 分别为从左、从右扫描的最高值减原值之和。若外部高度 x 不低于整段最大值，贡献为 $x \cdot len - sum$。否则，将孩子按扫描顺序记为 A, B：若 $x \geq \max A$，递归 B；否则递归 A，而 B 的贡献已经是 $F_{parent} - F_A$。每次只递归一个孩子，因此单段带边界扫描为 $O(\log n)$。区间平移不改变这些差值，懒标记只改最大值、原值和与标记本身。区间修改回溯及任意子区间扫描最坏 $O(\log^2(n+1))$，建树 $O(n)$。

对全局前缀和后缀最大值 L_i, R_i，有 $\max(L_i, R_i) = M$，其中 M 为全局最大值。因此

$$\sum_i \big(\min(L_i, R_i) - a_i\big) = F_0 + F_1 - \big(nM - \sum_i a_i\big).$$

根上即可 $O(1)$ 返回雨水体积，不必查询最大点位置。整体平移不改变体积，负高度也成立；带外部高度的 scan 则明确使用调用者的边界。所有乘法、和、标记及中间运算须在 long long 内，不能只检查最终答案。附件原题为 NERC 2023 G，使用例属于比赛应用，不计正式模板题覆盖；小规模有符号穷举、三十万次随机更新和二十万点规模本地验证通过，线上待补。

14.8 动态 K-D Tree：矩形和与重建边界

`KDTreeSum` 按深度交替坐标轴，以另一轴打破平局。重复坐标累加权值，其他点沿轴全序递归插入；孩子过重时收集编号并用 `nth_element` 重建，复用原节点。包围盒从本点初始化，仅合并非空孩子，不用有限正负无穷；重建直接返回根编号，不用坐标 -1 作为状态标志。权值为零时仍保留节点，负权不影响几何剪枝。

每个孩子大小至多为父节点的 3/4，所以树高为 $O(\log(d+1))$；矩形部分相交却可能递归两边，查询最坏仍为 $O(d)$。按平均线性选择计，一次大小 s 的重建需 $O(s\log(s+1))$，插入摊还 $O(\log^2(d+1))$；不是每次操作的最坏对数界。矩形使用闭边界，只比较坐标、不做坐标减法；各点绝对权值和及更新中间量须在 long long 内。

P4148 的在线输入需逐参数 XOR 上次答案，驱动预留二十万个节点以避免扩容峰值。本地默认 int 坐标节点为 56 字节，另有重建编号和运行环境开销；该分配量不能代替线上 20 MB 内存通过记录。独立 map 对拍、分割与包围盒审计、坐标极值及二十万规模已本地验证，线上待补。

14.9 压位小波矩阵与区间顺序统计

`WaveletMatrix` 先将值离散化，再从高位到低位稳定地把当前序列的零位元素放前面、一位元素放后面。同一路径下的原区间始终对应当前层的连续区间。记左右端点的一位前缀数为 a, b，则区间内零位数为 $z = r - l - b + a$。第 k 小若 $k < z$，下一层区间为 $[l - a, r - b)$；否则减去 z，进入 $[mid + a, mid + b)$，并设置答案的当前位。每层按 64 位存储，配一份整字前缀计数；末尾多留一个字，保证 $r = n$ 且 n 为 64 的倍数时仍可查询。掩码移位量用位置模 64，不会移位 64。

`kth(l,r,k)` 的区间和排名都为 0-based；P3834 的闭区间调用应为 `kth(l-1,r,k-1)`。第 k 大（k 从零开始）为 `kth(l,r,r-l-1-k)`；两者都要求 $0 \le k < r - l$。`less(l,r,x)` 严格小于 x；`freq(l,r,x)` 直接沿 x 所在路径计数，不能用 $x + 1$ 来规避等值查询，否则在最大有符号整数处溢出。空数组、空区间可计数，但不能求第 k 小；单一值域的层数为零，直接返回唯一值或区间长度。核心支持完整有符号 64 位值，三份正式题驱动按各题实际非负值范围验证。

该结构适合静态查询，不支持修改或历史版本。与持久化线段树的功能重叠不意味着来源中的每份划分树实现都已完成审计。WIDA 条目的压位查询功能已落实；kuangbin 划分树已另行实现为 DivisionTree，逐层保存值和前缀，空间为 $O(n \log(n+1))$。重复中位数按左半容量分配，详见 `docs/DIVISION-TREE.md`。独立子区间排序、直接计数、字边界和最大规模验证，以及两个 Library Checker 题目的官方本地 checker 报告见 `docs/WAVELET-MATRIX.md`。这些记录不是线上 AC，也不是最快提交榜排名。

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　动态模整数与合数模逆元

mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，乘法提升到 long long，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。所有中间乘法受 int 模数上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
    }

    mint(long long x = 0) : v((x % mod + mod) % mod) {}

    mint operator+(mint b) const { return mint((long long)v + b.v); }

    mint operator-(mint b) const { return mint((long long)v - b.v); }

    mint operator*(mint b) const { return mint(1LL * v * b.v); }

    mint &operator+=(mint b) { return *this = *this + b; }

    mint &operator-=(mint b) { return *this = *this - b; }

    mint &operator*=(mint b) { return *this = *this * b; }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        for (; k; k >>= 1, a *= a)
            if (k & 1) r *= a;
        return r;
    }
```

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1) return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const { return *this * b.inv(); }

    mint &operator/=(mint b) { return *this = *this / b; }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；传 move(a) 调用 batch_units，只求一次总积逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄；move 后不再读取 a。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：mint（第 260 页，A.1.1 节）、batch_units（第 265 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_units(move(a));
    if (!inverse) return 1;
    mint<> answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

A.1.2 静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0) : v((x % mod + mod) % mod) {}

    ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }

    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }

    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }

    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        for (; e; e >>= 1, a = a * a)
            if (e & 1) r = r * a;
        return r;
    }
```

求逆与除法（接上页同一结构体）：

```
    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1) return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue

    ModInt operator/(ModInt b) const { return *this * b.inv(); }

    ModInt &operator+=(ModInt b) { return *this = *this + b; }

    ModInt &operator-=(ModInt b) { return *this = *this - b; }

    ModInt &operator*=(ModInt b) { return *this = *this * b; }

    ModInt &operator/=(ModInt b) { return *this = *this / b; }
};
```

联合使用示例见第 269 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 模逆元

A.2.1 模整数类型的批量逆元

batch_units(a) 返回 optional<vector<Z»，保持输入顺序；任一元素不可逆返回 nullopt，空输入返回空数组。Z 须可从 1 构造，支持乘法、复合乘法及 try_inv，且元素来自同一交换模整数环；目前可与 mint<tag> 直接组合。先累计前缀乘积，只对总乘积求一次逆，再倒序求各项逆元。总乘积可逆当且仅当所有因子可逆，因此合数模数也可使用。O(n) 次乘法加一次求逆，空间 O(n)，要求大小可表示为 int；建议传 move(a) 避免保留额外输入副本。返回前不能切换动态模数。现有 long long 版 batch_inverse 保留独立接口与历史证据，不由本函数替换。

```
template <class Z> optional<vector<Z>> batch_units(vector<Z> a)
{
    int n = a.size();
    vector<Z> pre(n + 1, Z(1));
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * a[i];
    auto x = pre[n].try_inv();
    if (!x) return nullopt;
    Z inv = *x;
    for (int i = n - 1; i >= 0; i--)
    {
        Z old = a[i];
        a[i] = inv * pre[i];
        inv *= old;
    }
    return a;
}
```

联合使用示例见第 262 页（Luogu P5431 【模板】模意义下的乘法逆元 2），其中直接调用本模板。

A.3 卢卡斯定理

A.3.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 266 页，A.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

附录 B　组合数学

B.1　排列组合

B.1.1　阶乘组合数

代码依赖：ModInt（第 263 页，A.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }

    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

应用题补充：Luogu P1313 计算系数

题意：求 (by+ax) 的 k 次方中指定项的系数，模 10007。x、y 的指数分别为 n、m，均非负且 n+m=k。k 至多 1000，a、b 均为不超过 1000000 的非负数。组合 C(k,n) 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 263 页，A.1.2 节）、Binomial（第 268 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1313

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Binomial<10007> c(k);
    auto x = ModInt<10007>(a).pow(n);
    auto y = ModInt<10007>(b).pow(m);
    cout << (c.choose(k, n) * x * y).v << '\n';
}
```

算法索引

AffineSegTree, 59
AffineSequenceTreap, 115

batch_units, 265
Binomial, 268

CartesianTree, 169
cc_hash_table, 1

DivisionTree, 94
dsu, 4
DynamicKth, 210
DynamicPathMax, 249

Fenwick, 43
Fenwick2D, 38

GcdSequenceTreap, 120
gp_hash_table, 1
gp_map, 1

KDMin, 230
KDNearest, 219
KDRange, 224
KDTreeSum, 214

lazy_segtree, 48
LazySeg, 58
LCT
    虚子树, 256
LeftistHeap, 21
LiChao, 85
LinkCutTree, 235
Lucas, 266

MergeSplay, 135
MergeSplitTree, 68
mint, 260
ModInt, 263
MonotoneStackSeg, 63

offline_rectangle_sum, 45
OrderedSplay, 129
OrderedTreap, 106
ost, 104

PersistentArray, 173

PersistentDistinct, 186
PersistentDSU, 198
PersistentKth, 171
PersistentOrderedTreap, 201
PersistentParityIndex, 193
PersistentRange, 175
PersistentWeightedIndex, 189
PersistentXorTrie, 207
pheap, 17
PotentialDSU, 8

RectangleFenwick, 41
RollbackDSU, 6
rp, 102

ScapegoatTree, 154
SegmentBeats, 88
SegmentLiChao, 79
segtree, 53
SequenceKth, 158
SequenceMaxTreap, 125
SequenceScapegoat, 163
SequenceSplay, 147
SequenceTreap, 110
ShiftSplay, 143
StaticRMQ, 30
StaticRMQ2D, 34

TimeConnectivity, 11
Treap
    位翻转, 256
TreeBlockDistinct, 23
TreePathKth, 183
TreePathProducts, 240

WaveletMatrix, 97

XorPathSum, 243

动态树 DP, 256

替罪羊树, 256